

RWLang for Web Application Developers

From model-centered thinking to a first application and production operation

RWLang project

V1 – September 2026

Preface

This book is for web application developers who are meeting RWLang for the first time and may also be new to model-centered application design. It does not assume that the reader naturally thinks in terms of ORMs, domain models, or capability-based systems. The first chapter therefore does not begin with syntax. It begins with the question *what an application consists of*: what business data it handles, what operations may be performed on that data, who may perform them, and where all of this meets the HTTP world.

The recommended reading order is deliberately gradual. We first build the model and capability mental model, then compile and install the RWLang programs. After that we walk through a deliberately small A-to-Z application, from the model through a form and the database to the HTTP response. Only then do we break the language, database, authentication, file, and browser-facing topics into deeper chapters.

Once the reader can write an ordinary application, we move on to organizing larger projects and then to a wiki/CMS example. External integration and testing come next, because they make more sense once there is a complete application to reason about. The production part then introduces shared Redis state and caching, security hardening, observability, deployment, and disaster recovery. At the end of the book, the CLI and full `server.toml` reference are presented as searchable operator references rather than as introductory teaching material.

How to read this book

On a first pass, do not try to memorize every configuration key or production limit. The first goal is to be able to answer three questions consistently:

1. *What typed data does this feature operate on?*
2. *What operation reads or modifies that data?*
3. *What authority or capability permits that operation?*

Once those three questions are clear, the route, form, SQL, authentication, and configuration become different boundaries of the same model rather than unrelated tricks.

Five recurring terms

The book uses a few technical terms consistently. It is useful to establish the same meaning for them from the beginning:

model a named typed data shape known to the compiler; it is not an ORM class and not an automatically generated database schema;

typed the shape and type of the data are known to the compiler instead of being discovered only at runtime;

capability an explicit ability to access a resource or perform a sensitive operation, such as `Db`, `Transaction`, or `AppFs`;

policy a rule enforced at an external boundary, such as authentication, rate limiting, CORS, or trusted-proxy policy;

runtime the server-side environment that executes compiled RWLang programs and enforces limits and capability boundaries.

Later chapters refine these definitions, but these meanings are enough for a first reading.

Book conventions

- `main.rw` is the application entry point.
- The trusted TOML file is the primary production configuration surface.
- Secret values do not go on the command line or in plaintext TOML fields; configuration uses `*_file` references.
- SQL text is compiler-owned; user values may enter queries only as typed bind parameters.
- In HTML, `{ ... }` interpolation is escaped; there is no raw-HTML escape hatch.
- Business mutations run inside transactions.

The book targets the V1 surface. If a capability is not part of that surface, it should not be implemented through an ad-hoc bypass; it should become an explicit language/runtime feature.

Contents

Preface	ii
I Getting started: a mental model and a working application	1
1 How to think about an RWLang application	2
1.1 Concepts first, pages second	2
1.2 What does <code>model</code> mean?	3
1.3 The model is a contract between layers	4
1.4 Business concepts have operations	4
1.5 Capability: access is part of the model too	5
1.6 A route is not the business model	5
1.7 A simple design method	6
1.8 What to carry into the next chapters	6
2 Installation, binaries, and runtime topologies	7
2.1 Build from source	7
2.2 Where to install the binaries	8
2.3 Your first application	9
2.4 At first, learn only the minimum configuration	9
2.5 Production process and reverse proxy	10
2.6 Runtime topologies in detail	10
2.7 Direct single-domain HTTPS	10
2.8 Behind an Apache reverse proxy	11
2.9 Behind an Nginx reverse proxy	12
2.10 Recommended production directory structure	12
2.11 First validation	13
3 A to Z: a minimal web application	14
3.1 Create the project directory	14
3.2 The database schema	14
3.3 The model and queries	15
3.4 The GET page	15
3.5 The POST action	16
3.6 Routes and validation	16
3.7 The complete application in one place	17
3.8 SQLite connection for development	17
3.9 Source validation and migration	17
3.10 Minimal development configuration	18
3.11 The first HTTP request	18

3.12	What happens with invalid input?	18
3.13	From development toward production	19
3.14	What did this one example teach us?	19
II	Core web application development	20
4	Language essentials and HTTP boundaries	21
4.1	Source structure and statement boundaries	21
4.2	Typed computation for web application code	23
4.3	Domain values and handler code	25
4.4	Routing, typed input, and HTML	26
5	Data entry: typed SQL, CRUD, forms, and PRG	29
5.1	Typed SQL and CRUD	29
5.2	Forms, validation, and PRG	30
6	Database deep dive: transactions, migrations, and concurrency	32
6.1	Three separate responsibilities	32
6.2	The query return type is a contract	33
6.3	Bind values; do not build SQL	34
6.4	Read and mutation authority boundaries	34
6.5	Optimistic locking: do not overwrite someone else's work	35
6.6	Design indexes for access patterns	36
6.7	Migration: schema is part of the release	36
6.8	Expand/contract: preserve rollback options	37
6.9	Handling database errors	38
6.10	SQLite, PostgreSQL, or MariaDB?	38
6.11	Backup from a database developer's perspective	38
6.12	A sample CMS database layer	38
6.13	Chapter checklist	39
7	Authentication, sessions, and authorization	40
7.1	The complete login path	40
7.2	Local auth: a separate identity database	41
7.3	Account lifecycle through the operator CLI	41
7.4	Session invalidation: auth_generation	42
7.5	Session store: memory or Redis	42
7.6	Cookie policy	42
7.7	Logout	42
7.8	LDAP / LDAPS for enterprise identity	43
7.9	TOTP and recovery codes	43
7.10	Route authorization	44
7.11	Using the principal on a page	44
7.12	A route role is not always enough	44
7.13	Login rate limiting	45
7.14	CSRF, Origin, and Fetch Metadata	45
7.15	Authentication audit	45
7.16	Local auth or LDAP?	46
7.17	Rich text in an authenticated CMS	46
7.18	Production checklist	46

8	Files, uploads, and the media library	47
8.1	The mental model: three different things	47
8.2	General file upload: <code>Upload</code>	47
8.3	AppFs: the application's writable filesystem	48
8.4	What happens during an upload?	48
8.5	Image upload: prefer the <code>Image</code> type	49
8.6	Byte limits and pixel limits are different	49
8.7	Image references in the database	49
8.8	Safe rendering in HTML	50
8.9	The built-in media endpoint	50
8.10	CMS example: uploading a hero image	50
8.11	Database and AppFs may form one business state	51
8.12	Static assets and uploaded files are different	51
8.13	What not to do	51
8.14	Current media-surface limits	51
8.15	Developer upload checklist	51
9	JSON API, CORS, and browser client integration	52
9.1	The first typed JSON endpoint	52
9.2	JSON lists from the database	53
9.3	Typed JSON POST body	54
9.4	Content negotiation and 406	54
9.5	API errors	55
9.6	Same-origin: the default and simplest model	55
9.7	CORS: exact-origin allowlist	56
9.8	Preflight	56
9.9	Credentialed CORS with a session cookie	56
9.10	CSRF on a JSON API	57
9.11	Complete browser POST flow	57
9.12	Reverse proxy and origin	58
9.13	What not to do	58
9.14	Which architecture should you choose?	58
9.15	Runnable example	58
10	Static assets and frontend integration	59
10.1	Three separate file lifecycles	59
10.2	Configuring the static root	60
10.3	Fingerprints: the key to long caching	60
10.4	A minimal asset build pipeline	61
10.5	ETag and conditional GET	61
10.6	Precompressed Brotli and gzip	61
10.7	MIME, nosniff, and path confinement	62
10.8	CSS, images, and fonts in RWLang HTML	62
10.9	Important V1 boundary: JavaScript may be served, but not executed in RWLang HTML	62
10.10	Should RWLang serve assets, or Apache/Nginx?	63
10.11	Nginx example for direct static serving	63
10.12	Apache example for direct static serving	64
10.13	Deploy application code and assets as one release	64
10.14	Checking with curl	65
10.15	Practical decision rule	65

11	Organizing larger applications	66
11.1	Three separate questions	66
11.2	<code>mod</code> is not textual include	66
11.3	When should you switch to feature-based structure?	67
11.4	Domain object: a business namespace, not an OOP class	67
11.5	Dependencies remain explicit	69
11.6	Why should routes remain top-level?	70
11.7	Recommended naming convention	70
11.8	One possible CMS structure	70
11.9	What not to do	71
11.10	Refactoring path: from one file to a larger application	71
III	Practical applications	72
12	Practical applications: wiki and CMS	73
12.1	A simple wiki	73
12.2	CMS and news site	74
IV	Advanced integration and quality	77
13	External services and egress policy	78
13.1	First, the V1 boundary	78
13.2	Why is an arbitrary URL dangerous?	78
13.3	Named target: configuring a network capability	79
13.4	Policy checks multiple layers	79
13.5	IPv4 and IPv6 belong to the same policy	80
13.6	Size and time limits	80
13.7	A secret is not a configuration string	80
13.8	GET and JSON POST in the trusted adapter	81
13.9	Error model	81
13.10	Retries and idempotency	81
13.11	Synchronous request or background work?	82
13.12	What does this mean for an RWLang application developer?	82
13.13	Minimum testing	82
13.14	The essence of this chapter	82
14	Testing, error models, and developer diagnostics	83
14.1	The first gate: <code>rwlang-cli check</code>	83
14.2	A negative test is as valuable as a positive test	84
14.3	Four different worlds of failure	84
14.4	Test configuration separately	84
14.5	Migration: status, verify, apply, verify	85
14.6	Business tests: prove state, not implementation	85
14.7	Database tests: SQLite does not prove everything	86
14.8	Authentication and authorization test matrix	86
14.9	File, media, API, and cache tests	86
14.10	An HTTP error code is not the same thing as a program bug	87
14.11	Debugging with request IDs	88
14.12	Deployment smoke test	88

14.13	What does the repository <code>verify.sh</code> mean?	88
14.14	Recommended development cycle	89
14.15	Debugging decision order	89
14.16	Minimum application release checklist	89
V	Production and operations	91
15	Cache, Redis, rate limiting, and multi-instance operation	92
15.1	Why not a general Redis client?	92
15.2	Production Redis configuration	93
15.3	Shared sessions across server instances	93
15.4	Public page cache	94
15.5	What may go into public cache?	94
15.6	Operator-controlled cache boundaries	94
15.7	Cache invalidation: generations, not key search	95
15.8	Stampede protection and its limits	95
15.9	Redis failure during cached requests	95
15.10	Route rate limiting	95
15.11	Rate-limit scopes	96
15.12	Fixed-window model across instances	96
15.13	Memory backends are development exceptions	97
15.14	A typical multi-instance topology	97
15.15	Rolling restart and cache generation	97
15.16	What should you monitor?	98
15.17	Redis in recovery	98
15.18	Practical example: cached news front page with publishing	98
15.19	Checklist	99
16	Security hardening: connecting the trust boundaries	100
16.1	The five most important trust boundaries	100
16.2	HTTP edge: Host, proxy, and parser	101
16.3	TLS and secret handling	101
16.4	Identity: authentication is not authorization	102
16.5	Session, CSRF, and CORS together	102
16.6	SQL and business mutation	103
16.7	Uploads, media, and AppFs	103
16.8	Redis and shared runtime state	103
16.9	Outbound egress and SSRF	103
16.10	Resource limits are security controls too	104
16.11	Log hygiene and audit integrity	104
16.12	Process and filesystem hardening	105
16.13	Dependency and build supply chain	105
16.14	Production security gate	105
16.15	A code-review thinking pattern	106
17	Logging, audit, metrics, and troubleshooting	107
17.1	The role of the three log classes	107
17.2	Request ID: following one request end to end	108
17.3	What may and may not be logged?	108
17.4	Security audit and HTTP denials	109

17.5	Business audit: durable business history	109
17.6	Prometheus metrics on a separate management socket	110
17.7	Who scrapes metrics behind Apache or Nginx?	111
17.8	Health: liveness and readiness are different	111
17.9	Logrotate and SIGHUP	112
17.10	What if the log file is not writable?	112
17.11	Practical production troubleshooting workflow	113
17.12	Minimum monitoring baseline	113
17.13	Retention, SIEM, and audit protection	113
17.14	Checklist	114
18	Production deployment and service lifecycle	115
18.1	Release path	115
18.2	systemd service	115
18.3	Controlled restart and readiness	116
18.4	Migration and rollback are not the same operation	116
18.5	Recovery gate before deployment	116
18.6	Minimum post-deploy checks	117
19	Backup, restore, and disaster recovery	118
19.1	Start with a state inventory	118
19.2	What is not business source of truth?	118
19.3	RPO and RTO are business decisions before they are technical settings	119
19.4	Safe V1 baseline: a controlled maintenance backup	119
19.5	Application DB: use the backend's native backup tool	119
19.6	The local-auth DB is a security-sensitive backup	120
19.7	AppFs: the entire data root may be durable state	120
19.8	Every recovery point needs a manifest	121
19.9	Restore drill: prove the backup	121
19.10	Make a recovery decision before an upgrade	122
19.11	Expand/contract keeps the fast rollback window open	122
19.12	Rollback and restore are different operations	122
19.13	Forbidden shortcuts	123
19.14	Production recovery checklist	123
VI	Reference and operator quick guide	124
20	The RWLang CLI and the daily developer/operator workflow	125
20.1	Which tool for which task?	125
20.2	The CLI is not an alternative server	126
20.3	The first command: <code>rwlang-cli check</code>	126
20.4	Running from the source tree versus using the installed CLI	126
20.5	Migration workflow	126
20.6	Local-auth administration	127
20.7	TOTP administration is especially sensitive	128
20.8	A secret file is also an input contract	128
20.9	Server preflight follows CLI checks	129
20.10	Daily development cycle	129
20.11	Application release workflow	129
20.12	The RWLang platform release workflow is different	130

20.13	Automation: separate read-only and mutating steps	130
20.14	Common failures	131
20.15	One-page operator runbook	131
21	The <code>server.toml</code> configuration reference	133
21.1	Loading, precedence, and fail-closed rules	133
21.2	How to read the tables	134
21.3	[server] and [tls]	134
21.4	[database], [redis], and [auth]	135
21.5	[web]	135
21.6	[storage] and [static_assets]	136
21.7	[lifecycle], [observability], and [logging]	136
21.8	[rate_limit] and [cache]	137
21.9	[limits]	137
21.10	[cgroup]	138
21.11	Production baseline	138
21.12	Automatic source reload	139
21.13	Change workflow	139
22	Developer reference and where to go next	140
22.1	Where to look for each topic	140
22.2	Daily development checklist	141
A	Appendix and production checklist	142
A.1	Recommended production directory layout	142
A.2	Complete minimal Apache reverse-proxy example	143
A.3	Complete minimal Nginx reverse-proxy example	143
A.4	systemd service example	144
A.5	logrotate example	145
A.6	Pre-release production gate	146
A.7	Quick troubleshooting map	146
A.8	What to carry into production	147

Part I

Getting started: a mental model and a working application

Chapter 1

How to think about an RWLang application

Before installing anything, writing routes, or connecting a database, it is worth settling one design question. In many web stacks the natural starting point is a page, endpoint, or controller: “*we need a `/products` URL, so let’s write a handler.*” In RWLang it is more useful to ask first: “*what business data exists, what shape does it have, what operations may be performed on it, and within what boundaries?*”

This is not an academic distinction. Several of the language’s security and correctness guarantees come from making important data shapes, inputs, queries, and resource access explicit contracts.

1.1 Concepts first, pages second

Consider a simple product list. One possible way to start is:

We need a GET page, a POST endpoint, and an SQL INSERT.

In RWLang it is better to think in this order:

1. There is a business concept: *Product*.
2. The program knows its data shape: `id`, `name`, `price`.
3. There are read and mutation operations: `list` and `create`.
4. Mutation requires a database transaction.
5. The HTTP layer supplies typed input to the operation and attaches policy to it.
6. Rendering then turns the typed result into HTML or JSON.

The direction of thought is therefore approximately:

```
business concept
  -> typed data shape
    -> operations
      -> capabilities and policies
        -> HTTP route
          -> HTML / JSON
```

You do not need to write this chain down for every application. The order is nevertheless useful because it prevents business rules from being scattered accidentally across routes, templates, and hand-built SQL strings.

1.2 What does `model` mean?

The book uses two closely related ideas:

- **data model or domain model:** the human description of the business concepts and relationships in the application;
- **model declaration:** RWLang's concrete data shape known to the compiler.

For example:

```
model Product {  
  id: Int  
  name: String  
  price: Int  
}
```

This tells the compiler that a `Product` record has exactly these fields and types. A typed SQL query that returns a product must match this contract, and the same typed value can later be used in HTML or JSON output.

A `model` is not an automatic database schema

The declaration above does not create a `products` table. The persistent database schema is created by a migration, for example:

```
CREATE TABLE products (  
  id INTEGER PRIMARY KEY,  
  name TEXT NOT NULL,  
  price INTEGER NOT NULL  
);
```

The two layers are related, but they are not the same thing. This separation is intentional:

- the migration owns the lifecycle of the persistent database schema;
- the `model` is the application's typed data contract;
- the query states explicitly how database rows become model records.

RWLang therefore does not assume that every model is automatically an ORM entity or a table with the same name.

A `model` is not a mutable object

`Product` is not a traditional object-oriented class. It has no hidden `save()` method, implicit database connection, or mutable object lifecycle. To change data, the program uses an explicit operation and an explicit resource.

Reading, for example:

```
query fn listProducts(db: Db) -> Result<List<Product>, DbError> sql {  
  SELECT id, name, price  
  FROM products  
  ORDER BY id DESC  
  LIMIT 50  
}
```

Mutation is separate and transaction-bound:

```

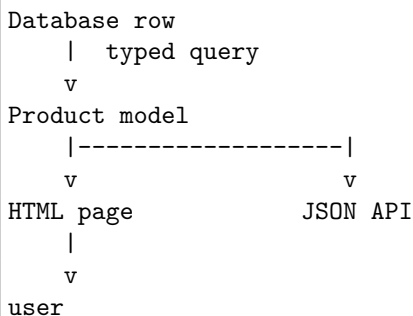
query fn createProduct(
  tx: Transaction,
  name: String,
  price: Int
) -> Result<Void, DbError> sql {
  INSERT INTO products(name, price)
  VALUES (:name, :price)
}

```

The function signatures make it visible which operation only reads and which one receives a transaction that can mutate state.

1.3 The model is a contract between layers

It is useful to think of a **model** not merely as a database-oriented structure, but as a typed contract that moves between layers.



This is why the compiler can catch errors early that a dynamic system might discover only at runtime. If a query promises to return **Product**, the selected/returned columns must match the model shape. The goal is not for the program to “figure out somehow” what the result means later.

The same idea reappears in object authorization and the JSON API: the system works with known typed records instead of passing arbitrary runtime-shaped objects around.

1.4 Business concepts have operations

A model by itself is rarely interesting. Real applications attach operations to it:

- `Product.list`;
- `Product.byId`;
- `Product.create`;
- in a CMS, for example, `Article.publish`;
- in a wiki, `WikiPage.update`.

In a small program these may be top-level functions. In a larger program the **object** domain namespace introduced later groups them around the business concept they belong to.

A useful distinction is:

Element	Question it answers
<code>model</code>	What shape does <code>Product</code> data have?
<code>query fn</code>	How do we read or mutate persistent data?
<code>object</code>	Which business concept owns these operations?
<code>page/action</code>	How does the business operation connect to a web handler?
<code>route</code>	On what HTTP path, with what input and policy, is it reachable?

1.5 Capability: access is part of the model too

RWLang makes not only data types explicit but also the resources an operation may access. A function does not obtain the database, filesystem, or current user from an invisible global source.

```
page fn home(ctx: PageContext, db: Db)
  -> Result<Html, PageError> {
    let products = listProducts(db)?;
    return Ok(html {
      <ul>
        @for product in products {
          <li>{{ product.name }}</li>
        }
      </ul>
    });
  }
```

The `db: Db` capability is visible in the signature. File handling uses an `AppFs`'s capability, mutating database work uses a `Transaction`, and authenticated operations depend on `auth` policy and the corresponding typed context.

This leads to a useful design rule:

If an operation needs a sensitive resource, that requirement should be visible in the code and policy.

1.6 A route is not the business model

A route matters, but it is the external boundary of the system. For example:

```
route create POST "/products"
  form name<String> price<Int>
  validate name length 1 100 price range 0 100000000
  => create;
```

This states how an HTTP request enters the system: method, path, typed input, validation, and later authentication/cache/rate-limit policy.

The business concept, however, is not the `/products` URL. The same `Product` can later be used by an HTML page, JSON API, admin interface, or another route.

For larger applications, the following boundary is especially useful:

<code>domain/model</code>	: what is the data and what operations make sense on it?
<code>HTTP route</code>	: who may call it, how, and under what policy?
<code>runtime config</code>	: within what infrastructure and resource limits may it run?

1.7 A simple design method

When starting a new feature, these five questions are a good default:

1. **What is the business noun?** – Product, Article, Invoice, WikiPage.
2. **What typed data is actually needed?** – do not automatically mirror the full database row.
3. **Which operations read, and which mutate?**
4. **Which capabilities do they need?** – Db, Transaction, AppFs, auth, and later network egress.
5. **Through what HTTP/policy surface do we expose them?** – route, input, validation, auth, cache, rate limit.

For a simple CRUD feature this is a few minutes of thought. For a CMS or line-of-business application it prevents the system from turning, a few months later, into a difficult-to-follow collection of routes, SQL fragments, and implicit authorization conditions.

1.8 What to carry into the next chapters

You do not need to design every application using full “domain-driven design” vocabulary, and a three-field record does not need an elaborate object hierarchy. The model-centered approach in this book is simpler:

- name the important business data;
- define its typed shape;
- describe data movement with explicit queries;
- bind mutation to explicit transactions;
- make resource access visible as capabilities;
- treat HTTP and security policy as an external boundary.

In the next chapters we compile, install, and minimally configure the system, then follow the same approach through a working **Product** application from end to end. After that, each part of the model is explored in depth.

Chapter 2

Installation, binaries, and runtime topologies

2.1 Build from source

The release source package is a Rust workspace. The normal release build that produces the two installable programs is:

```
cargo build --locked --release
```

The repository's own full self-check is `./verify.sh`; run it when you validate or release the RWLang source tree itself. It is not the first step required merely to use an application.

`-locked` ensures that the build uses the dependency graph captured in the release's `Cargo.lock`. If you only need the two executable programs:

```
cargo build --locked --release -p rwlang-server -p rwlang-cli
```

After the build, the two important programs are:

```
target/release/rwlang-server
target/release/rwlang-cli
```

- **rwlang-server**: the long-running service process that compiles RWLang applications and serves HTTP(S);
- **rwlang-cli**: an operator/administrative program for tasks such as authentication administration and migrations.

Their roles are intentionally different. A reverse proxy connects only to the **rwlang-server** process. **rwlang-cli** is not a web server and does not need a public or proxied listener.

During development, these may be sufficient:

```
cargo check --workspace
cargo test --workspace
```

2.2 Where to install the binaries

On a production host, do not run the service permanently out of Cargo’s `target/` directory. RWLang is locally installed software rather than a program managed by the operating system’s package manager, so this book consistently uses the `/usr/local` hierarchy: executables go under `/usr/local/bin`, and machine-local configuration goes under `/usr/local/etc/rwlang`. This keeps the RWLang installation separate from distribution-managed `/usr/bin` and `/etc` files.

A simple, auditable layout is:

```
/usr/local/bin/rwlang-server
/usr/local/bin/rwlang-cli
/usr/local/etc/rwlang/server.toml
/srv/rwlang/current/
/srv/rwlang/data/
/run/secrets/rwlang/
/var/log/rwlang/
```

For example:

```
sudo install -d -m 0755 /usr/local/bin
sudo install -d -m 0755 /usr/local/etc/rwlang
sudo install -m 0755 target/release/rwlang-server /usr/local/bin/rwlang-server
sudo install -m 0755 target/release/rwlang-cli /usr/local/bin/rwlang-cli
sudo install -m 0644 config/server.toml.sample \
  /usr/local/etc/rwlang/server.toml
```

`/usr/local/bin` is normally already part of the system `PATH`, so no additional symlink is necessary. The examples in this book use the actual binary names, `rwlang-server` and `rwlang-cli`.

Debian package installation is intentionally different

The layout above is for manual/source installation. If RWLang is installed through a Debian package, the files are owned by `dpkg`, so Debian filesystem conventions apply instead of the local-administrator `/usr/local` prefix:

```
/usr/bin/rwlang-server
/usr/bin/rwlang-cli
/etc/rwlang/server.toml
/usr/lib/systemd/system/rwlang.service
```

Build the package on a Debian/Ubuntu build host with:

```
make deb
sudo dpkg -i dist/rwlang_1.0.0-1_amd64.deb
```

The package deliberately does not start the service automatically: deploy the application and secrets, edit `/etc/rwlang/server.toml`, validate it, and only then enable `rwlang.service`. This is not a contradiction with the manual layout; `/usr/local` is correct for administrator-owned local installs, while `/usr` and `/etc` are correct for distribution/package-manager-owned files. The repository’s `docs/36-debian-package.md` gives the complete packaging workflow.

2.3 Your first application

A minimal recommended project is:

```
myapp/  
|-- main.rw  
|-- pages.rw  
`-- public/
```

main.rw:

```
mod pages;
```

pages.rw:

```
page fn home(ctx: PageContext) -> Result<Html, PageError> {  
    return Ok(html {  
        <main><h1>Hello RWLang</h1></main>  
    });  
}  
  
route home GET "/" => home;
```

Start it directly from the workspace during development:

```
cargo run -p rwlang-server -- \  
  --app /abs/path/myapp/main.rw \  
  --listen 127.0.0.1:8080 \  
  --insecure-dev-cookies
```

The same command using the installed release binary is:

```
/usr/local/bin/rwlang-server \  
  --app /abs/path/myapp/main.rw \  
  --listen 127.0.0.1:8080 \  
  --insecure-dev-cookies
```

`-insecure-dev-cookies` is for local development only.

2.4 At first, learn only the minimum configuration

At this point in the book the goal is not to learn the entire `server.toml`. As a beginner, it is enough to distinguish three things:

- the *application code* defines the models, queries, pages, and routes that exist;
- the *server configuration* defines where the process listens, which external resources it may access, and what operator-controlled limits apply;
- *secret files* hold sensitive values that must not live in application source or plaintext TOML.

The A-to-Z chapter provides a minimal development configuration. Production-specific keys are introduced only after there is an application worth operating; the complete key/default/sample reference is near the end of the book. On a first reading, do not try to memorize the full

configuration surface.

2.5 Production process and reverse proxy

Behind Apache or Nginx, the typical request path is:

```
Internet
|
| HTTPS :443
v
Apache or Nginx
|
| HTTP 127.0.0.1:8080
v
/usr/local/bin/rwlang-server
|
+-- /usr/local/etc/rwlang/server.toml
+-- /srv/rwlang/current/main.rw
+-- DB / Redis / AppFs
```

The proxy neither starts nor replaces the RWLang process. Run `rwlang-server` as a separate service, for example under `systemd`, and let the proxy forward requests to its loopback listener. `rwlang-cli` runs only for operator commands.

A simple `systemd` `ExecStart` fragment is:

```
ExecStart=/usr/local/bin/rwlang-server \
  --config /usr/local/etc/rwlang/server.toml
```

The process user should not be `root`; grant access only to the required application, secret, data, and log files. If TLS is terminated by Apache or Nginx, keep the RWLang listener on loopback, for example `127.0.0.1:8080`.

2.6 Runtime topologies in detail

The binary locations and process roles stay the same whether RWLang terminates TLS itself or runs behind Apache or Nginx. The difference is the listener, TLS termination, and trusted-proxy boundary configuration.

2.7 Direct single-domain HTTPS

When RWLang terminates TLS itself, a public listener may use port 443:

```
[server]
app = "/srv/rwlang/current/main.rw"
listen = "0.0.0.0:443"
insecure_dev_cookies = false

[tls]
cert_file = "/run/secrets/rwlang/tls-fullchain.pem"
key_file = "/run/secrets/rwlang/tls-key.pem"
handshake_timeout_ms = 5000
http_redirect_listen = "0.0.0.0:80"
```

```
public_host = "www.example.com"
```

`public_host` comes from trusted configuration. Only GET/HEAD requests may be redirected from HTTP to HTTPS.

Because this direct configuration binds ports 80/443, an unprivileged systemd service needs the narrow `CAP_NET_BIND_SERVICE` capability. Do not run the service as root merely to bind these ports. Behind a reverse proxy, with a listener such as `127.0.0.1:8080`, that capability is unnecessary.

2.8 Behind an Apache reverse proxy

In this topology Apache handles public TLS and RWLang listens only on loopback. Example RWLang configuration:

```
[server]
app = "/srv/rwlang/current/main.rw"
listen = "127.0.0.1:8080"
insecure_dev_cookies = false

[tls]
public_host = "www.example.com"

[web]
trusted_proxy_cidrs = ["127.0.0.1/32", "::1/128"]
allow_missing_origin = false
cors_origins = []
cors_allow_credentials = false
```

In reverse-proxy mode there is no RWLang-side certificate/key, but `tls.public_host` is still required. It is the trusted source for the canonical host visible to the browser and anchors Host and Origin/Referer validation. Production plain-HTTP upstream is allowed only on a loopback listener with an explicit trusted-proxy list and such a public host. Every normal proxied request must carry trusted metadata such as `X-Forwarded-Proto: https` or the equivalent standard `Forwarded: proto=https` from a trusted proxy.

A minimal Apache VirtualHost is:

```
<VirtualHost *:443>
    ServerName www.example.com

    SSLEngine on
    SSLCertificateFile /etc/letsencrypt/live/www.example.com/fullchain.pem
    SSLCertificateKeyFile /etc/letsencrypt/live/www.example.com/privkey.pem

    ProxyPreserveHost On
    ProxyPass          / http://127.0.0.1:8080/
    ProxyPassReverse   / http://127.0.0.1:8080/

    RequestHeader set X-Forwarded-Proto "https"
</VirtualHost>
```

This typically requires the `proxy`, `proxy_http`, `headers`, and `ssl` modules. RWLang accepts forwarding headers only from ranges listed in `trusted_proxy_cidrs`; do not configure an

overly broad range such as the entire Internet.

The port-80 VirtualHost may exist solely to redirect to HTTPS:

```
<VirtualHost *:80>
    ServerName www.example.com
    Redirect permanent / https://www.example.com/
</VirtualHost>
```

2.9 Behind an Nginx reverse proxy

The Nginx topology uses the same trust model: Nginx handles public TLS and RWLang listens on loopback. The RWLang-side `server.toml` may be the same as in the Apache example.

A minimal Nginx server block is:

```
server {
    listen 443 ssl;
    server_name www.example.com;

    ssl_certificate      /etc/letsencrypt/live/www.example.com/fullchain.pem;
    ssl_certificate_key  /etc/letsencrypt/live/www.example.com/privkey.pem;

    location / {
        proxy_pass http://127.0.0.1:8080;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-Proto https;
        proxy_set_header X-Forwarded-For $remote_addr;
    }
}
```

This example has a single direct reverse proxy, so `X-Forwarded-For` is the client address as seen by the proxy and `X-Forwarded-Proto` is `https`. Do not let arbitrary forwarding headers arriving from the Internet become authority. RWLang only trusts forwarding headers received from the exact `trusted_proxy_cidrs` ranges.

The HTTP listener may exist only to redirect to HTTPS:

```
server {
    listen 80;
    server_name www.example.com;
    return 301 https://$host$request_uri;
}
```

2.10 Recommended production directory structure

```
/usr/local/bin/rwlang-server
/usr/local/bin/rwlang-cli
/srv/rwlang/releases/2026-09-04.1/
/srv/rwlang/current -> /srv/rwlang/releases/2026-09-04.1
/srv/rwlang/data/
/usr/local/etc/rwlang/server.toml
/run/secrets/rwlang/
```

```
/var/log/rwlang/
```

The binary, release tree, and configuration should be runtime read-only. The application should write only to declared data/log locations.

2.11 First validation

Before sending traffic to the server:

```
/usr/local/bin/rwlang-server \  
  --config /usr/local/etc/rwlang/server.toml \  
  --check-config
```

This validates configuration, secret-file references, application compilation, and TLS files, among other checks, without opening a listener.

Chapter 3

A to Z: a minimal web application

The previous chapters established RWLang’s model and capability mental model, then compiled and installed the system. We still do not need to know the entire server configuration surface: this example uses only minimal development settings. Before examining each language feature in isolation, let us build one complete but intentionally small application. The goal is not a finished webshop, but to show how an RWLang web application fits together from source file through database and HTTP request to production deployment.

The example is a miniature product list. The home page lists products from the database and contains a form for adding a new product. In one small program we therefore see a model, typed SQL, GET page, POST action, form validation, CSRF protection, transaction, migration, configuration, and deployment behind a reverse proxy.

The finished example is available in the repository at:

```
examples/a-z-products/  
  main.rw  
  migrations/0001_create_products.sql  
  server.toml.example  
  README.md
```

3.1 Create the project directory

Create a separate application directory for development:

```
mkdir -p my-products/migrations  
cd my-products
```

For this example all RWLang code stays in a single `main.rw`. Larger applications will later be split into mod files.

3.2 The database schema

For SQLite, `migrations/0001_create_products.sql` contains:

```
CREATE TABLE products (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  name TEXT NOT NULL,
```



```
    price INTEGER NOT NULL CHECK (price >= 0)
);
```

This migration creates the table used by the application's typed SQL queries. In production, migration is a separate deployment step; the server does not modify the schema automatically at startup.

3.3 The model and queries

At the top of `main.rw`, following the model-centered approach from Chapter 1, define the typed business data shape first:

```
model Product {
  id: Int
  name: String
  price: Int
}
```

The list query is:

```
query fn listProducts(db: Db) -> Result<List<Product>, DbError> sql {
  SELECT id, name, price
  FROM products
  ORDER BY id DESC
  LIMIT 50
}
```

`List<Product>` states that the query may return zero or more `Product` rows. The selected columns exactly follow the model field names and order.

The INSERT receives a transaction capability:

```
query fn createProduct(
  tx: Transaction,
  name: String,
  price: Int
) -> Result<Void, DbError> sql {
  INSERT INTO products(name, price)
  VALUES (:name, :price)
}
```

`:name` and `:price` are bind parameters. We do not construct SQL from user input with string concatenation.

3.4 The GET page

The home page fetches the products, displays a form, and renders the result list:

```
page fn home(ctx: PageContext, db: Db) -> Result<Html, PageError> {
  let products = listProducts(db)?;

  return Ok(html {
    <main>
```

```

    <h1>Mini product list</h1>

    <form method="post" @action(create)>
      <input type="hidden" name="_csrf" value="{{ csrfToken }}">
      <label>
        Name
        <input name="name" maxlength="100">
      </label>
      <label>
        Price
        <input name="price">
      </label>
      <button type="submit">Add</button>
    </form>

    <ul>
      @for product in products {
        <li>
          #{{ product.id }} - {{ product.name }} - {{ product.price }}
        </li>
      }
    </ul>
  </main>
});
}

```

Dynamic text is HTML-escaped before it enters the response. The form action is not a hand-written URL; it is generated by the `@action(create)` helper. The hidden `_csrf` field protects the POST request against CSRF.

3.5 The POST action

A successful form submission writes to the database in a transaction, then redirects to the home page:

```

action fn create(
  ctx: ActionContext,
  db: Db,
  name: String,
  price: Int
) -> Result<Redirect, PageError> {
  transaction db {
    createProduct(tx, name, price)?;
  }
  return Ok(redirect("/"));
}

```

This is the Post/Redirect/Get pattern. If the user refreshes the resulting page, the browser does not automatically repeat the INSERT.

3.6 Routes and validation

Two routes at the end of the file connect HTTP to the handlers:

```
route home GET "/" => home;

route create POST "/products"
  form name<String> price<Int>
  validate name length 1 100 price range 0 100000000
  => create;
```

The POST route turns both fields into typed input before the handler runs. An empty or over-100-character name, a negative or excessively large price, or a non-integer value never reaches the INSERT as a valid business operation.

3.7 The complete application in one place

The preceding pieces form the complete application: one `model`, two `query fn` declarations, one `page fn`, one `action fn`, and two `route` declarations. We do not repeat the entire long listing here; the exact single-file version is available directly in the repository:

```
examples/a-z-products/main.rw
```

This is also a deliberate editorial choice. Later chapters examine the individual language features in depth; here the goal is to follow the entire application path once.

3.8 SQLite connection for development

Do not put the database URL into source code. Create a separate file containing an absolute SQLite path:

```
printf '%s\n' 'sqlite:///tmp/rwlang-a-z-products.db' > dev-db-url
chmod 600 dev-db-url
```

This file does not contain a real secret in this example, but it uses the same `*_file` pattern that production uses for password-bearing database URLs.

3.9 Source validation and migration

If the release binaries are already installed under `/usr/local/bin`, for example:

```
/usr/local/bin/rwlang-cli check main.rw

/usr/local/bin/rwlang-cli migrate verify \
  --dir migrations \
  --db-url-file "$PWD/dev-db-url"

/usr/local/bin/rwlang-cli migrate apply \
  --dir migrations \
  --db-url-file "$PWD/dev-db-url"
```

`check` verifies the language and static contracts. The migration CLI is a separate process; the application server is still not running.

3.10 Minimal development configuration

For example, `server.toml` may be:

```
[server]
app = "/ABS/PATH/my-products/main.rw"
listen = "127.0.0.1:8080"
insecure_dev_cookies = true

[database]
url_file = "/ABS/PATH/my-products/dev-db-url"
```

Replace both `/ABS/PATH` values with real absolute paths. `insecure_dev_cookies` is acceptable only for loopback development; never enable it in production.

Validate the configuration first:

```
/usr/local/bin/rwlang-server --config "$PWD/server.toml" --check-config
```

Then start the server:

```
/usr/local/bin/rwlang-server --config "$PWD/server.toml"
```

3.11 The first HTTP request

From another terminal:

```
curl -i http://127.0.0.1:8080/
```

Open the same URL in a browser. After the form is submitted, the flow is:

1. the browser sends POST to the `/products` route;
2. the runtime validates the request schema, typed fields, validation rules, and CSRF token;
3. the `create` action opens a transaction;
4. `createProduct` runs an INSERT with bind parameters;
5. success commits the transaction;
6. the handler redirects to `/`;
7. the new GET renders the row read back from the database.

The same basic chain appears later in the wiki and CMS examples; they simply add more models, queries, routes, authorization, and components around it.

3.12 What happens with invalid input?

Try a negative price or a non-numeric value. The important rule is that browser-side HTML attributes are only a usability aid; the real contract is the server-side route schema and validation. The SQL query receives only already-typed values.

It is also useful to try the following:

- submit a request without the `name` field;
- submit an unknown extra form field;
- modify the migration after applying it, then run `migrate verify` again;

- make the database unavailable and observe readiness and server-log behavior.

3.13 From development toward production

The application code stays the same in a release environment; it does not need to be rewritten merely because the server now runs from installed binaries. After the build and installation described in Chapter 2, the server typically starts as:

```
/usr/local/bin/rwlang-server \  
--config /usr/local/etc/rwlang/server.toml
```

The application entry point may be `/srv/rwlang/current/main.rw`. With Apache or Nginx, the public TLS connection terminates at the proxy and RWLang listens on loopback. The full Apache/Nginx configuration is not repeated here: the installation chapter explains the trust boundary, while the appendix provides near-copy/paste production configurations.

3.14 What did this one example teach us?

A few dozen lines already show the entire basic RWLang path:

```
HTTP GET  
-> route  
-> page  
-> typed query  
-> database  
-> escaped HTML  
  
HTTP POST  
-> route schema + validation + CSRF  
-> action  
-> transaction  
-> typed SQL bind  
-> commit  
-> redirect  
-> GET
```

The next chapters decompose this picture. Whenever a language or runtime detail seems overly technical, return to two questions: *where does data enter the system, and through what typed/capability boundaries does it pass before becoming persistent state or an HTTP response?*

Part II

Core web application development

Chapter 4

Language essentials and HTTP boundaries

The A-to-Z example already showed the main building blocks in action. This chapter turns those pieces into a day-to-day development model for web work: organize source explicitly, terminate statements unambiguously, compute with typed values, normalize text at boundaries, keep domain logic in handlers, and cross into HTTP/HTML only through typed routes and safe rendering.

A useful working rule is: *parse and normalize at the boundary, compute in typed values, persist through explicit queries, and render as late as possible*. The sections below follow that order.

4.1 Source structure and statement boundaries

Modules

`main.rw` is the entry point of the module graph:

```
mod models;
mod queries;
mod components;
mod pages;
mod actions;
```

In V1, `mod` loads an application-root-relative module without injecting its declarations into the global namespace. For example, declarations from `models.rw` are addressed as `models::Article` across module boundaries. Local declarations may use their short name inside their own module. There is no general Rust-like `pub` export system in V1.

Module namespaces and cross-module calls

A module declaration adds one application-root-relative source module to the explicit source graph; it does not copy that module's declarations into the current scope. The canonical mapping is direct:

<code>models.rw</code>	<code>-> models</code>
<code>catalog/queries.rw</code>	<code>-> catalog::queries</code>
<code>catalog/pages.rw</code>	<code>-> catalog::pages</code>

For example, an entry point may load several modules:

```
mod models;
mod catalog::queries;
mod catalog::pages;

route catalogIndex GET "/catalog"
  => catalog::pages::index;
```

Across a module boundary, use the qualified name. A declaration in `catalog/queries.rw` is referenced as `catalog::queries::recent(...)`, and a type in `models.rw` as `models::Article`. Inside the declaration's own module, the short local name remains valid. This keeps local code concise while making dependencies between source modules visible.

Module paths are always relative to the application root. There is no `mod.rw` fallback, automatic directory discovery, `../`, `./`, `self::`, `super::`, `crate::`, or wildcard import. Declare every source module explicitly, before ordinary top-level declarations.

Language builtins such as `trim(...)`, `sin(...)`, and `len(...)` are not application-module declarations and therefore remain unqualified.

Statement terminators

RWLang uses an explicit semicolon as the terminator for simple statements. Newlines are whitespace, not statement boundaries; there is no automatic semicolon insertion. This keeps multiline expressions deterministic and makes generated/audited code unambiguous.

```
let subtotal = price
  * quantity
  + shipping;
set retries = retries + 1;
authorize article owner authorUsername or role Publisher;
return Ok(json(subtotal));
```

Block statements and block declarations are closed by their brace and do not take a trailing semicolon:

```
if ready {
  set state = 1;
}
while state < 3 {
  set state = state + 1;
}
```

The same rule applies inside a transaction: a standalone mutating query call and a business-audit statement are simple statements and therefore end in `;`. Top-level block declarations such as `model`, `page fn`, `action fn`, and control-flow blocks do not use `;`. Non-block top-level declarations do: both `mod path;` and `route ... => handler;` are terminated explicitly with `;`. A multiline route is complete only at that final terminator.

4.2 Typed computation for web application code

Expressions and arithmetic

RWLang expressions use explicit precedence. From tighter to looser: unary `!`; multiplication, division and remainder; addition and subtraction; shifts; comparisons; bitwise AND/XOR/OR; logical AND; logical OR. Parentheses should be used whenever they make intent clearer.

```
let total = 12 + 3 * 4;
let remainder = 17 % 5;
let shifted = 3 << 2;
let masked = 12 & 10;
let allowed = ready && authorized;
let fallback = cached || fresh;
let denied = !allowed;
```

The operator contract is deliberately typed:

Operands	Operators	Result
Int, same type	<code>+ - * / \%</code>	checked Int.
F32, same type	<code>+ - * / \%</code>	F32; result must remain finite.
Decimal, same type	<code>+ - * / \%</code>	checked Decimal.
Int, same type	<code>< > \% \^{} </code>	shift/bitwise Int.
Bool, same type	<code>\&\& </code>	Bool; short-circuit evaluation.
Bool	<code>!</code>	logical negation.
String, same type	<code>+</code>	String concatenation, allocation-budgeted.
Numeric, same type	<code>< <= > >=</code>	Bool.
Same supported scalar type	<code>== !=</code>	Bool.

There is no implicit mixing of Int, F32, and Decimal. Convert an integer explicitly with `toF32(value)` when floating-point computation is required. Numeric overflow, division/remainder by zero, invalid shifts, or a non-finite F32 result fail closed rather than silently wrapping or propagating NaN/Infinity.

The numeric builtins include:

Function	Type	Purpose
<code>sin(x), cos(x)</code>	F32 -> F32	Trigonometric functions.
<code>sqrt(x)</code>	F32 -> F32	Square root; invalid domain fails.
<code>abs(x)</code>	Int -> Int or F32 -> F32	Absolute value.
<code>ln(x), log10(x)</code>	F32 -> F32	Natural/base-10 logarithm.
<code>log(x, base)</code>	F32, F32 -> F32	Logarithm with explicit base.
<code>exp(x)</code>	F32 -> F32	Exponential function.
<code>pow(x, y)</code>	F32, F32 -> F32	Floating-point power.
<code>round/floor/ceil(x)</code>	F32 -> F32	Rounding operations.
<code>toF32(x)</code>	Int -> F32	Explicit integer-to-F32 conversion.
<code>monotonicNanos()</code>	() -> Int	Process-local monotonic nanosecond counter.

All F32 builtin results must remain finite. Invalid-domain operations fail instead of creating NaN or infinity.

`monotonicNanos()` is for elapsed-time measurement, not wall-clock timestamps; public-cache output that depends on it is rejected.

Practical arithmetic patterns

Web code rarely needs advanced mathematics everywhere, but a few numeric patterns recur constantly. Keep them typed and visible rather than hiding them in string conversions or SQL fragments.

For pagination, compute an offset from validated integers:

```
let pageSize = 25;
let offset = (page - 1) * pageSize;
```

For flags and compact integer masks, bitwise operations are available, but they should represent a real integer-domain contract rather than replace readable booleans. Use `&&` and `||` for business conditions because they short-circuit:

```
let mayPublish = authenticated
    && ownsArticle
    && !locked;
```

Use `Decimal` for money. Use `F32` only when floating-point behavior is part of the problem, such as geometry, signal processing, statistics, or explicit math functions. Never convert monetary values to `F32` merely to reuse a math builtin.

String and text functions

String operations are typed, Unicode-aware builtins executed inside the bounded runtime:

```
let cleaned = trim(text);
let left = trimStart(text);
let right = trimEnd(text);
let normalized = lower(cleaned);
let chars = stringLen(cleaned);
let rewritten = replace(normalized, " ", "-");
let part = substring(cleaned, 1, 3);
let first = indexOf(cleaned, "rw");
let last = lastIndexOf(cleaned, "a");
let ch = charAt("RWLang", 1);
let repeated = repeat("rw", 3);
let parts = split(rewritten, "-");
```

Function	Signature
<code>stringLen(text)</code>	<code>String -> Int</code>
<code>trim/trimStart/trimEnd(text)</code>	<code>String -> String</code>
<code>lower/upper(text)</code>	<code>String -> String</code>
<code>contains(text, needle)</code>	<code>String, String -> Bool</code>
<code>startsWith/endsWith(text, part)</code>	<code>String, String -> Bool</code>
<code>replace(text, search, replacement)</code>	<code>String, String, String -> String</code>
<code>split(text, delimiter)</code>	<code>String, String -> List<String></code>
<code>substring(text, start[, length])</code>	<code>String, Int[, Int] -> String</code>
<code>indexOf/lastIndexOf(text, needle)</code>	<code>String, String -> Int</code>
<code>charAt(text, index)</code>	<code>String, Int -> String</code>
<code>repeat(text, count)</code>	<code>String, Int -> String</code>

`stringLen`, `substring`, `indexOf`, `lastIndexOf`, and `charAt` use Unicode scalar-value positions rather than UTF-8 byte offsets. Missing searches return `-1`; invalid negative/out-of-range indices fail closed. `replace` rejects an empty search string, `split` rejects an empty delimiter and is capped at 4096 items, while `repeat` is allocation-budgeted before materializing its result. `List<String>` remains a typed compute collection with `len(parts)` and checked `parts[index]` access.

For bounded regular-expression work, RWLang also provides `regexMatch`, `regexReplace`, and `regexCaptures`. Regex patterns and runtime work remain subject to validation and resource limits.

Practical text-handling patterns

Treat text manipulation as boundary work. Normalize user-facing search/filter input before comparing it, but keep the original value when the exact spelling matters to the domain.

```
let queryText = trim(q);
let normalizedQuery = lower(queryText);
let hasRwlang = contains(normalizedQuery, "rwlang");
```

Use `replace` for deterministic substitutions, not for general parsing:

```
let displayTag = replace(trim(tag), "_", " ");
```

When slicing or locating text, remember that RWLang uses Unicode scalar-value positions rather than UTF-8 byte offsets. This makes `substring`, `charAt`, and the index functions suitable for application text without exposing byte-level encoding details. For structured identifiers such as URLs, slugs, emails, dates, or UUIDs, prefer the dedicated RWLang types and validators instead of rebuilding their rules with string functions.

Choosing the right tool in application code

Web-development task	Preferred RWLang approach
Normalize search/filter text	<code>trim</code> , then <code>lower/upper</code> only when comparison should ignore case.
Replace a known token	<code>replace</code> ; do not use regex unless pattern matching is actually required.
Parse email, URL, slug, date, UUID	Use the dedicated typed value and validation path, not ad-hoc string parsing.
Money/totals	<code>Decimal</code> .
Pagination/counters	<code>Int</code> with checked arithmetic.
Business conditions	<code>Bool</code> with short-circuit <code>&&</code> , <code> </code> , and <code>!</code> .
Scientific/statistical math	<code>F32</code> and explicit math builtins; check that floating point is appropriate for the domain.
Elapsed-time measurement	<code>monotonicNanos()</code> , never as a wall-clock timestamp.

4.3 Domain values and handler code

Models and business types

```
model Article {
  id: Int
```

```

    slug: Slug
    title: String
    body: String
}

```

Common types include `String`, `Int`, and `Bool`, as well as `Date`, `DateTime`, `Uuid`, `Decimal`, and `Slug`. Use `Decimal` for money rather than floating-point values.

Page and action

A page contains GET-like presentation logic:

```

page fn show(ctx: PageContext, db: Db, id: Int)
-> Result<Html, PageError> {
  let article = articleById(db, id)?;
  return Ok(html { <h1>{{ article.title }}</h1> });
}

```

An action handles state-changing requests:

```

action fn create(ctx: ActionContext, db: Db, title: String)
-> Result<Redirect, PageError> {
  transaction db {
    insertArticle(tx, title)?;
  }
  return Ok(redirect("/admin/articles"));
}

```

Variables and error propagation

`let` introduces a typed local value. V1 has no general-purpose `mut`; compute code uses explicit `set name = expr` reassignment, with the original static type preserved and execution charged to the instruction budget. The `?` operator propagates an error from a typed `Result`.

Enum

Use an enum for a closed set of choices rather than free-form text. This gives a stronger contract in forms, routes, JSON, and domain logic.

Object

Related model/query/page operations can be grouped under a domain object. This is particularly useful in larger applications because it defines a clear domain API boundary.

4.4 Routing, typed input, and HTML

Route

```

route article GET "/article/:slug<Slug>" => article;

```

The path parameter type is declared in the route. Invalid input never reaches the handler as a typed value.

HTML escaping

```
return Ok(html {
    <h1>{{ article.title }}</h1>
});
```

{ ... } is HTML-escaped. A string read from the database is no exception. There is no raw-HTML API.

Lists and optional values

A list must be iterated, and an optional model may be dereferenced only after an explicit presence check. The compiler does not allow a `List<Model>` or an unchecked `Model?` to behave like a single record.

```
@for article in articles {
    <li>{{ article.title }}</li>
}

@if article {
    <h1>{{ article.title }}</h1>
}
```

The database chapter explains the exact runtime query-cardinality contract, including the fact that a `Model?` result is an error if the query returns more than one row.

Typed URLs

```
<a @href(article, article.slug)>Open</a>
<form method="post" @action(deleteArticle, article.id)>
```

Prefer route helpers

Do not duplicate your own application-route URLs as hand-written strings. In HTML, `@action(...)` is the preferred form for POST targets and `@href(...)` for links, because the route name and typed parameters remain the source of truth. Built-in runtime endpoints – such as `/__rw/auth/logout` – are naturally not application route helpers.

Do not build a dynamic `href="{{ value }}"` link. A route helper knows both the destination and the parameter type.

Components and layouts

```
layout fn Main(title: String) -> Html {
    html {
        <html>
            <head><title>{{ title }}</title></head>
            <body><main>@content</main></body>
        </html>
    }
}
```

Inside a page:

```
return Ok(html {  
    @layout(Main, article.title) {  
        <article><h1>{{ article.title }}</h1></article>  
    }  
});
```

A layout may contain exactly one `@content` slot; components and layouts can only access their explicit typed parameters.

Chapter 5

Data entry: typed SQL, CRUD, forms, and PRG

A classic web-application cycle is: read data from the database, accept typed input through an HTML form, validate it, mutate state in a transaction, then redirect and read again with GET. This chapter treats that entire cycle as one unit.

5.1 Typed SQL and CRUD

Query

```
query fn articleBySlug(db: Db, slug: Slug)
  -> Result<Article, DbError> sql {
    SELECT id, slug, title, body
    FROM articles
    WHERE slug = :slug
  }
```

The SQL text is static and compiler-owned. A user-provided value may enter only as a typed bind, such as `:slug`.

Mutation in a transaction

```
query fn updateArticle(
  tx: Transaction,
  id: Int,
  title: String,
  body: String
) -> Result<Void, DbError> sql {
  UPDATE articles
  SET title = :title, body = :body
  WHERE id = :id
}
```

The action:

```
action fn update(
  ctx: ActionContext,
  db: Db,
```

```
    id: Int,
    title: String,
    body: String
) -> Result<Redirect, PageError> {
  transaction db {
    updateArticle(tx, id, title, body)?;
  }
  return Ok(redirect("/admin/articles"));
}
```

Success commits; failure rolls back. A `Transaction` capability cannot escape the block.

Portable database schema

V1 aims for patterns that can remain portable across SQLite, PostgreSQL, and MariaDB. Canonical text may be used as the portable representation for `Date`, `DateTime`, `Uuid`, and `Decimal`.

Migration

The server does not migrate automatically at startup. The recommended flow is:

```
migrate verify
migrate apply
migrate verify
```

Keep migration credentials separate from normal application-runtime credentials.

5.2 Forms, validation, and PRG

Named form schema

```
form ArticleForm {
  title<String>
  body<String>
  published<Bool>
  validate title length 2 200 body length 1 200000
}
```

Route:

```
route articleCreate POST "/admin/articles"
  form ArticleForm
  auth role Editor
  => create;
```

The handler parameter order and types should follow the form fields.

Invalid submission

For a named form, a missing required field, a field-level typed decode failure, or a declared validation failure results in 422 *Unprocessable Content*. The runtime refills field values in

escaped form, attaches field errors, and inserts a CSRF token into the form. An unknown or duplicate field is instead a 400 because it violates the request schema.

This intentionally differs from inline form or JSON schemas: there a typed decode/validation failure is a request-contract violation and returns 400. Use a named form when you want to report the error on the same user-facing form with field-level feedback.

Successful POST and PRG

After a successful mutation, return a redirect so the browser loads the destination with GET. This Post/Redirect/Get pattern eliminates the common accidental resubmission problem.

A complete simple form-action pattern

```
action fn create(
  ctx: ActionContext,
  db: Db,
  title: String,
  body: String,
  published: Bool
) -> Result<Redirect, PageError> {
  let articleSlug = slug(title);
  transaction db {
    insertArticle(tx, articleSlug, title, body)?;
  }
  return Ok(redirect("/admin/articles"));
}
```

For a named form, a missing checkbox-like Bool field is interpreted as **false**.

Chapter 6

Database deep dive: transactions, migrations, and concurrency

The previous chapter introduced typed SQL and the basic CRUD pattern. A real wiki, CMS, or line-of-business application soon reaches a point where the question is no longer merely *how to run a `SELECT`*, but how to preserve data integrity across concurrent requests, multiple server instances, and multiple releases.

This chapter provides that developer model. The goal is to treat the database not as a hidden global service but as an explicit capability, and to treat the schema as a versioned release component.

6.1 Three separate responsibilities

In an RWLang application it is useful to separate three layers:

application code declares static typed SQL queries, supplies binds, and requests transactions;
runtime pools connections, binds parameters, applies timeouts and result limits, then validates returned rows against the declared type;
operator owns database credentials, TLS, database capacity, backups, and the migration process.

The application therefore does not open arbitrary sockets to the database or build its own connection pool. **Db** and **Transaction** are the capabilities through which database work occurs.

The current V1 data-layer secure defaults include, among other things, a maximum pool size of 32 connections, a 5-second connection-acquire timeout, a 10-second query timeout, a 10,000-row result limit, and a 16 MiB result-size limit. Do not treat these as application logic: correct queries should still be short, paginated, and indexable.

With multiple server instances, the operator must budget total database connections across all instances:

```
total DB connections ~= number of RWLang instances
                      * pool per instance
                      + admin/maintenance reserve
```

6.2 The query return type is a contract

Query cardinality is visible in the function type:

```
query fn articleById(db: Db, id: Int)
  -> Result<Article, DbError> sql {
    SELECT id, slug, title, body, version
    FROM articles
    WHERE id = :id
  }

query fn articleBySlug(db: Db, slug: Slug)
  -> Result<Article?, DbError> sql {
    SELECT id, slug, title, body, version
    FROM articles
    WHERE slug = :slug
  }

query fn recentArticles(db: Db, limit: Int, offset: Int)
  -> Result<List<Article>, DbError> sql {
    SELECT id, slug, title, body, version
    FROM articles
    ORDER BY id DESC
    LIMIT :limit OFFSET :offset
  }
```

The three forms mean:

Type	Expected result	Cardinality violation
Article	exactly one row	0 rows: 404; multiple rows: DB error
Article?	zero or one row	multiple rows: DB error
List<Article>	zero or more rows	runtime result limits apply

This is more than convenient notation: result shape is part of the program contract and is enforced at runtime. **Article?** does not mean “take the first row if one exists.” If the query returns two or more rows, the invariant is broken and the runtime fails closed with a DB error. Similarly, an **Article** query returns 404 Not Found for zero rows and does not arbitrarily choose a row if there are several.

Make row shape explicit

The names and order of **SELECT** fields should match the model. Do not use **SELECT *** in application queries.

```
model Article {
  id: Int
  slug: Slug
  title: String
  body: String
  version: Int
}

// good: the same five fields, in the same order
SELECT id, slug, title, body, version
```

```
FROM articles
```

Explicit projection matters for two reasons. First, the compiler can verify the relationship between model and query. Second, later schema expansion does not silently alter an old query's result shape.

6.3 Bind values; do not build SQL

Client-derived values enter SQL as typed binds:

```
query fn findByEmail(db: Db, email: String)
  -> Result<User?, DbError> sql {
    SELECT id, email, display_name
    FROM users
    WHERE email = :email
  }
```

`:email` is not string substitution. The runtime translates it to a backend-specific placeholder and binds it as a parameter.

Do not think in terms of:

```
// wrong approach:
// "SELECT ... ORDER BY " + userOrder
// "WHERE " + userFilter
```

Column names, `ORDER BY` direction, SQL operators, and entire `WHERE` fragments must not come from user input. If several sort modes are supported, choose among a small number of static queries in application code.

6.4 Read and mutation authority boundaries

A read query receives a `Db` capability. A mutating query receives a `Transaction`:

```
query fn renameArticle(
  tx: Transaction,
  id: Int,
  title: String
) -> Result<Void, DbError> sql {
  UPDATE articles
  SET title = :title
  WHERE id = :id
}
```

The caller opens an explicit transaction block:

```
action fn rename(
  ctx: ActionContext,
  db: Db,
  id: Int,
  title: String
) -> Result<Redirect, PageError> {
  transaction db {
```

```

        renameArticle(tx, id, title)?;
    }
    return Ok(redirect("/admin/articles"));
}

```

A successful block commits; an error rolls back. **Transaction** cannot escape the block, so a closed transaction handle cannot accidentally be used later.

One business operation should be one transaction

Do not split related mutations across separate actions or separate transactions:

```

transaction db {
    Article.publishChanged(tx, id, version)?;
    audit Article id action publish;
        from article.status to ArticleStatus.Published
}

```

Here publishing and the business audit trail commit together. If the audit record cannot be written, publishing rolls back too. This prevents a state where the business change happened but its required audit record is missing.

6.5 Optimistic locking: do not overwrite someone else's work

In a CMS or wiki, two editors may open the same record. A simple

```
UPDATE articles SET title = :title WHERE id = :id
```

makes the last writer win and can silently discard the first editor's changes.

Use a version column and **Changed**:

```

model Article {
    id: Int
    title: String
    version: Int
}

query fn updateArticle(
    tx: Transaction,
    id: Int,
    title: String,
    version: Int
) -> Result<Changed, DbError> sql {
    UPDATE articles
    SET title = :title,
        version = version + 1
    WHERE id = :id AND version = :version
}

```

The edit form sends the **version** value the user saw when opening the page. The **Changed** contract is strict: the mutation must affect **exactly one row**. One changed row is success; zero changed rows become **409 Conflict**; two or more rows become a DB error because the

query violated its own invariant. **Changed** is only valid as the return type of a mutating query, and in V1 it cannot be combined with a **RETURNING** clause.

With optimistic locking, if another editor saved in the meantime, **WHERE id = :id AND version = :version** matches zero rows and the request becomes a conflict. The correct UX is not an automatic retry. Tell the user that the data changed, load the new version, and let the user decide how to merge.

Typical uses include:

- wiki pages and CMS articles;
- tickets or customer records;
- business objects with workflow state;
- rarely changed administrator configuration.

It is generally unnecessary for append-only events or database-side atomic counters.

6.6 Design indexes for access patterns

An index is useful not because there are many of them, but because it serves a real query.

Natural CMS access patterns might justify:

```
CREATE UNIQUE INDEX articles_slug_uq
  ON articles(slug);

CREATE INDEX articles_status_id_idx
  ON articles(status, id);
```

If the public site loads by slug, make slug unique/indexed. If the admin list filters by status and pages by identifier, a matching composite index may be appropriate.

Some rules:

- paginate every large list;
- index frequently used lookup fields;
- align index order with **WHERE/ORDER BY** patterns;
- do not index every column automatically: indexes cost writes and storage;
- investigate performance with the real backend's **EXPLAIN** plan.

RWLang typed SQL protects query shape; it does not replace the database optimizer.

6.7 Migration: schema is part of the release

In production, the server does not run migrations automatically at startup. This is deliberate: a schema change is a separate operator action with separate credentials and a separate failure boundary.

A typical directory is:

```
migrations/
  0001_create_articles.sql
  0002_articles_slug_index.sql
  0003_articles_add_version.sql
```

File names use:

```
NNNN_name.sql
```

Do not modify an already-applied migration. The CLI records a SHA-256 checksum and stops on mismatch.

Production commands using the installed CLI:

```
/usr/local/bin/rwlang-cli migrate status \  
--dir /srv/myapp/current/migrations \  
--db-url-file /run/secrets/migration-db-url  
  
/usr/local/bin/rwlang-cli migrate verify \  
--dir /srv/myapp/current/migrations \  
--db-url-file /run/secrets/migration-db-url  
  
/usr/local/bin/rwlang-cli migrate apply \  
--dir /srv/myapp/current/migrations \  
--db-url-file /run/secrets/migration-db-url \  
--lock-timeout-secs 30
```

Migration credentials may have more privileges than application DML credentials. The server process should therefore not receive them.

What does the migration CLI guarantee?

- applied file name and checksum are verified;
- an old version number cannot silently receive a new migration later;
- a database-level migration lock is acquired;
- migration state advances only after successful application.

Backend-specific differences remain. PostgreSQL uses a transaction per migration; SQLite keeps the batch in a transaction using a `BEGIN IMMEDIATE` writer lock; MariaDB DDL is not transactional in every case, so partial DDL failure may require operator repair.

6.8 Expand/contract: preserve rollback options

A fast application rollback is straightforward only when the new schema remains compatible with the old application.

For larger changes, use expand/contract:

1. **expand**: add a new nullable/defaulted field or a new table while keeping the old application functional;
2. deploy the new application;
3. perform a controlled backfill if necessary;
4. in a later release, **contract**: remove the old field or constraint after the rollback window has closed.

Immediate rename/drop, a new mandatory field, or an incompatible type change can create a forward-only release. Treat that as an explicit release-review decision.

6.9 Handling database errors

`DbError` is a technical error. Do not expose driver messages, SQL text, host names, or connection strings to the end user.

At application level distinguish at least:

record not found normal domain state, often handled with an optional query;

409 concurrent modification optimistic-lock conflict; the user resolves it by reloading/-comparing;

database unavailable infrastructure failure; readiness may report it and the server log can be searched by request ID;

constraint/domain failure preferably prevent it with validation, while keeping the database constraint as the final integrity boundary.

Do not wrap an entire mutating action in an unlimited automatic retry loop. Repeating a request may repeat side effects, and a transaction conflict is not necessarily transient.

6.10 SQLite, PostgreSQL, or MariaDB?

RWLang V1 supports all three backends, but that does not make every SQL dialect feature portable.

SQLite excellent for simple or single-host deployments; back up a live application database using an SQLite-aware method rather than plain copying the live file.

PostgreSQL a strong default for larger or multi-instance production systems; use TLS for remote connections.

MariaDB also supported, but account for DDL transactional differences especially during migrations.

Portable applications should use a common SQL subset. Use backend-specific features only deliberately, and document the backend dependency in the project.

6.11 Backup from a database developer's perspective

As a database developer, remember one key point here: migration and backup are separate lifecycles. Migration describes how schema changes; backup/restore proves that persistent state can be recovered. For PostgreSQL, MariaDB, and SQLite, use the backend's own consistency-aware backup method and do not assume that a plain file copy is automatically a valid backup.

Full application state – database, AppFs, local-auth store, recovery manifest, RPO/RTO, and restore drills – is covered in Chapter 19. It matters here because schema-change planning must already know what state is available for rollback or restoration.

6.12 A sample CMS database layer

A small CMS might start with:

Article	
id	
slug	UNIQUE
title	
body	


```
status
hero_image
version
created_at
updated_at

migrations/
  0001_create_articles.sql
  0002_article_indexes.sql
  0003_add_version.sql
```

Application patterns:

- public read: `Article?` by slug;
- admin list: paginated `List<Article>`;
- edit: `Changed + version`;
- publish/archive: business mutation + audit in one transaction;
- schema change: separate CLI migration;
- before deploy: migration verify + backup/restore readiness;
- after deploy: readiness, logs, metrics, and smoke test.

With this model the database layer is not a separate collection of SQL snippets; it becomes part of the web application's type, concurrency, release, and recovery contract.

6.13 Chapter checklist

Before considering a database-backed feature complete, verify:

- every user value enters SQL as a typed bind;
- there is no `SELECT *`; projections match models;
- large lists are paginated;
- important lookup/sort patterns have appropriate indexes;
- every mutation runs with a `Transaction` capability;
- related mutations share one transaction;
- human-concurrently-edited records use optimistic locking;
- already-applied migrations are immutable;
- migration credentials are not available to the application server;
- destructive schema changes have a known rollback consequence;
- DB errors do not leak SQL or credentials to the client;
- backups have regularly proven restores.

Chapter 7

Authentication, sessions, and authorization

Authentication proves who the user is; authorization decides what that user may do. After a successful sign-in, the session carries that identity across requests. Authentication is therefore not merely a login form. In a production web application, the identity backend, session lifecycle, second factor, authorization, CSRF protection, rate limiting, audit, and proxy trust form one system. RWLang separates these responsibilities: the `.rw` application declares route policy, while identity and session policy come from server configuration.

7.1 The complete login path

The built-in HTML login endpoints are:

```
GET  /__rw/auth/login
POST /__rw/auth/login
POST /__rw/auth/logout
```

These are server endpoints, not routes defined by the application. The GET login page issues a CSRF token bound to the session. The POST login form expects exactly the `_csrf`, `username`, `password`, and `totp` fields.

At a high level, a successful login:

1. validates form structure and the CSRF token;
2. canonicalizes the username;
3. applies the login rate limit;
4. verifies local-auth or LDAP credentials;
5. if required, verifies TOTP or, for local auth, a recovery code;
6. rotates to a new authenticated session;
7. writes an audit event;
8. redirects to the root with 303 See Other.

Session rotation matters: after successful authentication, the anonymous session identifier is not kept. This is a basic defense against session fixation.

7.2 Local auth: a separate identity database

A single-node application may use the local-auth SQLite store. It is deliberately separate from the application database. RWLang typed SQL queries therefore cannot access credential tables.

For example, the secret file may be:

```
/run/secrets/rwlang/local-auth-db-url
```

with contents:

```
sqlite:///srv/rwlang/auth/users.db?mode=rwc
```

Initialize it with:

```
/usr/local/bin/rwlang-cli auth init \  
  --db-url-file /run/secrets/rwlang/local-auth-db-url
```

When creating a user, do not pass the password as a command-line argument; read it from a file:

```
printf '%s\n' 'a-very-long-random-password' \  
  > /run/secrets/rwlang/new-user-password  
chmod 600 /run/secrets/rwlang/new-user-password  
  
/usr/local/bin/rwlang-cli auth user-add \  
  --db-url-file /run/secrets/rwlang/local-auth-db-url \  
  --username alice \  
  --password-file /run/secrets/rwlang/new-user-password \  
  --role User \  
  --role Editor
```

The local store uses Argon2id password hashing with a unique random salt. The current implementation accepts passwords between 12 and 1024 characters. Do not confuse this with application-side form validation: credential policy belongs to the identity backend.

7.3 Account lifecycle through the operator CLI

User state does not need to be modified with direct SQL. The CLI provides explicit operations:

```
rwlang-cli auth password-set --db-url-file ... \  
  --username alice --password-file ...  
  
rwlang-cli auth disable  --db-url-file ... --username alice  
rwlang-cli auth enable   --db-url-file ... --username alice  
rwlang-cli auth roles-set --db-url-file ... --username alice \  
  --role User --role Publisher
```

Disabling an account does not delete it. From the outside, failed login does not reveal whether the cause was a wrong password, nonexistent user, or disabled account, because such distinctions can become account-enumeration channels.

7.4 Session invalidation: `auth_generation`

With local auth, credential and authorization changes increment the user's `auth_generation`. A session records the generation that was current when it was created.

On the next request, the server compares the session generation with the current value in the local-auth store. If they differ, the old session is invalidated and a fresh anonymous session is created.

Consequently, the following operator actions affect existing sessions too, not merely the next login:

- password change;
- role change;
- account disable/enable;
- TOTP enrollment or disable.

This is particularly important for role revocation: there is no need to wait for the old session to expire naturally.

7.5 Session store: memory or Redis

An in-memory session store may be used for development and testing. In production, especially with multiple server instances, Redis should be the shared session backend.

```
[redis]
url_file = "/run/secrets/rwlang/redis-url"
allow_insecure = false
```

Redis may hold several kinds of short-lived infrastructure state in RWLang:

- shared sessions;
- TOTP replay protection;
- login rate limiting;
- public cache.

Redis is not a business source of truth. After disaster recovery, an empty session/cache state is acceptable: users sign in again and caches rebuild.

7.6 Cookie policy

In normal production mode the session cookie is named:

```
__Host-rw_session
```

The server uses `Path=/`, `HttpOnly`, and `Secure`. The default is `SameSite=Lax`; with credentialed cross-origin configuration it becomes `SameSite=None`; `Secure`.

The explicit development escape hatch `-insecure-dev-cookies` changes the cookie name to `rw_session` and omits `Secure`. Never use this for an Internet-facing production deployment.

7.7 Logout

Logout is intentionally a POST operation:

```
<form method="post" action="/_rw/auth/logout">
  <input type="hidden" name="_csrf" value="{ { csrfToken } }">
  <button type="submit">Logout</button>
</form>
```

The server verifies the CSRF token, invalidates the old session, creates a fresh anonymous session, sends a new cookie, and audits the logout event. Logout is therefore not simply a client-side cookie deletion.

7.8 LDAP / LDAPS for enterprise identity

For shared or enterprise identity, an LDAP backend may be used. Use an LDAPS URL in production, with service-bind credentials loaded from secret files.

Example configuration:

```
[auth]
ldap_url = "ldaps://ldap.example.com:636"
ldap_search_base = "ou=people,dc=example,dc=com"
ldap_username_attribute = "uid"
ldap_service_bind_dn_file = "/run/secrets/rwlang/ldap-bind-dn"
ldap_service_bind_password_file = "/run/secrets/rwlang/ldap-bind-password"
require_totp = true
```

Do not configure local auth and LDAP as simultaneous primary identity backends in one process. That avoids ambiguous backend priority and username collisions.

With LDAP, role mapping should come from trusted server-side configuration. Client-supplied `role`, `username`, or similar fields are never authorization authority.

7.9 TOTP and recovery codes

Enroll TOTP for a local-auth user with:

```
/usr/local/bin/rwlang-cli auth totp-enroll \
  --db-url-file /run/secrets/rwlang/local-auth-db-url \
  --username alice
```

The CLI prints the secret/URI data required for enrollment and the recovery codes. Recovery codes remain in the local-auth database only as hashes, and a successfully used code is consumed.

Disable TOTP with:

```
/usr/local/bin/rwlang-cli auth totp-disable \
  --db-url-file /run/secrets/rwlang/local-auth-db-url \
  --username alice
```

If `require_totp = true`, a user with a correct password still cannot sign in without a usable second factor.

The login form's `totp` field may contain either a six-digit TOTP or, with local auth, a recovery code. With Redis, replay state is shared, so multiple server instances see the same TOTP

replay protection.

7.10 Route authorization

Applications commonly declare three route-level policies:

```
route account GET "/account" auth user => account;
route admin GET "/admin" auth role Admin => admin;
route security GET "/security" auth mfa => security;
```

Their meanings are:

- `auth user`: any authenticated principal;
- `auth role Admin`: an authenticated principal with the trusted `Admin` role in the session;
- `auth mfa`: an authenticated principal that has successfully proven a second factor.

For an HTML route, an unauthenticated request may redirect to the login page. For a JSON API, the same condition appears as `401 Unauthorized` rather than an HTML redirect, which is more predictable for API clients.

7.11 Using the principal on a page

An authenticated page receives trusted session state:

```
page fn account(ctx: PageContext) -> Result<Html, PageError> {
  return Ok(html {
    <main>
      <h1>Account</h1>
      <p>Signed in as {{ authPrincipal }}</p>
      <form method="post" action="/__rw/auth/logout">
        <input type="hidden" name="_csrf" value="{{ csrfToken }}">
        <button type="submit">Logout</button>
      </form>
    </main>
  });
}

route account GET "/account" auth user => account;
```

`authPrincipal`, session roles, and MFA state are server-side trusted state. Do not reconstruct them from hidden form fields or query parameters.

7.12 A route role is not always enough

A route-level role answers whether a user may enter an operation. Object-level authorization may require more.

For example, an editor may be allowed to modify only their own draft. In that case `auth role Editor` alone is insufficient: the action must compare the owner of the record loaded from the database with the trusted session principal.

The core rule is:

identity from the session, object state from the database, decision on the server

Client-supplied `owner`, `actor`, or `role` fields are data at most, never authority.

The typed object-authorization guard

The `authorize` guard may be used only on an already-loaded, **non-optional** model record. The compiler also verifies that the `owner` field exists and has type `String`. A `Result<Article?, DbError>` must therefore be handled explicitly first; authorization does not interpret a “no record” state.

```
let article = loadArticle(db, id)?;  
authorize article owner authorUsername or role Publisher;
```

A handler containing this guard cannot be exposed through a public route, and public cache cannot sit in front of object authorization. The owner or one of the listed trusted roles is allowed; every other authenticated actor receives `403 Forbidden`.

7.13 Login rate limiting

Login has its own rate-limit policy. For example:

```
[auth]  
login_max_attempts = 8  
login_window_secs = 300
```

After trusted-proxy processing, the server uses the effective client IP together with the canonical username as part of the limiter key. This is why `trusted_proxy_cidrs` is also an authentication-security setting behind a reverse proxy.

A failed credential or second factor does not produce detailed failure information for the client. Use `429 Too Many Requests` for rate limiting and `503 Service Unavailable` when the authentication backend is unavailable.

7.14 CSRF, Origin, and Fetch Metadata

With session-cookie authentication, the browser sends the cookie automatically, so state-changing requests require CSRF protection. RWLang uses defense in depth:

- a session-bound CSRF token;
- same-origin Origin/Referer policy;
- Fetch Metadata controls;
- strict request parsing;
- trusted-proxy policy.

CORS does not replace CSRF protection. If you allow credentialed CORS for a separate frontend origin, that is a distinct security policy and requires HTTPS.

7.15 Authentication audit

Login and logout events go to the audit log. Useful fields include request ID, actor, outcome, and effective client IP. The purpose of the audit log is event reconstruction, not credential debugging.

Never log:

- passwords;
- TOTP secrets or recovery codes;
- session IDs or full Cookie headers;
- CSRF tokens;
- LDAP bind or DB/Redis credentials.

A login result may distinguish success, invalid credentials, invalid second factor, or rate limiting internally, while the client-facing message remains intentionally less detailed.

7.16 Local auth or LDAP?

	Local auth	LDAP/LDAPS
Identity store	separate SQLite DB	enterprise directory
Typical deployment	single-node	shared/enterprise identity
User administration	RWLang CLI	directory admin workflow
Role source	local-auth store	trusted mapping
TOTP	local enrollment + recovery	server-side TOTP policy/secrets
Multi-instance identity	not the target use case	yes

If the application needs public self-registration, email password reset, or self-service MFA enrollment, do not bolt a thin route onto the operator CLI. Those features require token lifecycle, email security, rate limiting, and their own recovery threat model.

7.17 Rich text in an authenticated CMS

An authenticated CMS editor interface still does not justify storing raw HTML. Editor authorization and content sanitization are separate defenses. Store Markdown as content and render it with the safe renderer:

```
<article class="article-body">
  @markdown(article.body)
</article>
```

`@markdown` does not permit raw HTML and filters link targets through an allowlist.

7.18 Production checklist

Before enabling authentication in production, verify:

- exactly one primary identity backend is active;
- the local-auth DB is separate from the application DB;
- passwords and service credentials come only from secret files;
- production sessions use Redis when multiple server instances run;
- cookies are Secure/HttpOnly and `-insecure-dev-cookies` is not enabled;
- `trusted_proxy_cidrs` covers only the real Apache/Nginx proxy addresses;
- the login limiter is configured;
- state-changing forms use CSRF protection;
- object-level authorization supplements route roles where necessary;
- credentials and session secrets never enter logs;
- account disable, role changes, and TOTP changes have been tested for session revocation.

Chapter 8

Files, uploads, and the media library

In a web application, file handling is not simply a matter of copying a client-provided filename into a directory. Uploading touches request processing, resource limits, filesystem security, the database, authorization, backup, and – for images – HTML output as well.

RWLang therefore provides two distinct levels. The general **Upload** value carries upload metadata for a file streamed into AppFs, while **Image** is a stricter validated image type that is preferable for CMS and wiki interfaces.

8.1 The mental model: three different things

Keep these three concepts separate during upload handling:

1. **client metadata**: original filename and client-declared MIME type;
2. **storage**: the server-side file selected by the runtime inside AppFs;
3. **business reference**: the value the application stores in the database and uses later.

The client filename is not a storage path, and the client's **Content-Type** header is not security evidence. Treat both at most as display or audit metadata.

8.2 General file upload: Upload

The route declaration tells the runtime that a multipart file is expected and names the static AppFs subdirectory that receives it:

```
route uploadFile POST "/upload"
  upload file<Upload> to "uploads"
  => uploadFile;
```

The action does not receive the whole file in memory. `Upload` exposes upload metadata:

```
action fn uploadFile(ctx: ActionContext, file: Upload)
  -> Result<Redirect, PageError> {
    let savedPath = file.path;
    let originalName = file.filename;
    let contentType = file.contentType;
    let byteCount = file.bytes;
    return Ok(redirect("/uploaded"));
  }
```

This is an important resource-protection property: the runtime streams multipart content, so a large file does not automatically become an equally large VM heap allocation.

8.3 AppFs: the application's writable filesystem

AppFs is the restricted filesystem area assigned to the application. Give it a dedicated data root in production; do not mix it with the program binary, static assets, or other writable system areas.

Example server configuration:

```
[storage]
data_root = "/srv/rwlang/data"
fs_mode = "rwc"
max_upload_bytes = 16777216
max_image_pixels = 40000000
```

The short capability letters mean:

- r** read;
- w** write;
- c** create and the operations required to commit an upload.

Uploads typically require **rwc**. The byte limit is an operator hard limit; application code cannot raise it arbitrarily for one route.

On Linux, AppFs confinement is intended to prevent an application from escaping the assigned root through path traversal, symlinks, or magic-link tricks. Normal Unix ownership and permissions are still required: the service user should have write access only to directories it actually needs.

8.4 What happens during an upload?

From a developer perspective, the safe upload path is:

1. the runtime checks request-size limits and the route upload schema;
2. it streams the file into a confined staging location;
3. the server generates the storage key rather than deriving it from the client filename;
4. typed uploads perform additional server-side content validation;
5. after successful processing the file can be committed atomically;
6. on handler/runtime failure the runtime attempts cleanup.

Do not build your own “sanitize the filename and write it somewhere” mechanism beside AppFs. That would bypass the security boundary provided by the language/runtime.

8.5 Image upload: prefer the `Image` type

For images in a CMS, wiki, news site, or admin interface, use `Image` rather than the general Upload surface:

```
route saveHero POST "/admin/hero"
  upload hero<Image> to "media"
  auth role Editor
  => saveHero;
```

`Image` uses the same streaming multipart and AppFs path, but the server also validates actual file content. V1 accepts PNG and JPEG. The client-declared MIME type is not authority.

SVG is intentionally not a general image-upload format because it can carry active content and script/XSS surface. Do not assume GIF, WebP, or AVIF support unless the documentation for the runtime version explicitly states it.

8.6 Byte limits and pixel limits are different

Images have two distinct resource risks. A file may be small in bytes but decode into an enormous number of pixels. The pixel ceiling (`max_image_pixels`) is therefore separate from the upload-byte ceiling (`max_upload_bytes`).

```
max_upload_bytes = 16777216
max_image_pixels = 40000000
```

The second limit caps `width * height`. Set both consciously for the application’s requirements; a “16 MB upload limit” alone is not image-decode protection.

8.7 Image references in the database

`Image` is a first-class scalar, so it can be stored as a model field and in the database using a canonical representation. The application does not need – and should not attempt – to assemble or parse internal storage-path components manually.

A typical business model:

```
model Article {
  id: Int
  slug: Slug
  title: String
  body: String
  hero: Image
}
```

The important point is that the database stores a *reference*, not a client filename and not a hand-built public URL. Storage and HTTP-serving policy therefore remain under runtime control.

8.8 Safe rendering in HTML

A validated image is rendered using the dedicated renderer rather than direct string interpolation:

```
page fn article(ctx: PageContext, article: Article)
  -> Result<Html, PageError> {
    return Ok(html {
      <article>
        <h1>{{ article.title }}</h1>
        @image(article.hero, article.title)
        @markdown(article.body)
      </article>
    });
  }
```

`@image(image, alt)` generates HTML from the validated image reference. It escapes the alt text, uses validated dimensions, and targets the built-in read-only media endpoint. Direct `{ image }` interpolation of an `Image` is a compiler error, so the typed reference cannot accidentally become a raw URL or markup value.

8.9 The built-in media endpoint

The runtime reserves this URL prefix:

```
/__rw/media/
```

Applications cannot declare conflicting routes. The media endpoint does not make the entire `data_root` browsable over HTTP; it serves images only from AppFs directories declared by an `upload ...<Image> to "..."` route in the program.

When serving media, the server re-detects content based on the image detector rather than trusting the client's original MIME type. Responses receive security and cache headers; media serving does not run an application route and does not create a session.

8.10 CMS example: uploading a hero image

An editor workflow normally consists of two business steps:

1. the editor uploads a validated image;
2. the application associates the resulting typed image reference with the article.

The route should at least require authentication; an admin/editor function normally also deserves role policy:

```
route saveHero POST "/admin/articles/:id<Int>/hero"
  upload hero<Image> to "media"
  auth role Editor
  => saveHero;
```

Do not assume that “it is only an image, so it is harmless.” An upload route is still a state-changing POST: authentication, CSRF/origin protection, request limits, and an auditable business operation may all apply.

8.11 Database and AppFs may form one business state

If a database row references an image stored in AppFs, the two stores together form the state that must be recoverable. This directly affects backup strategy.

For example, a database dump taken in the morning and an unrelated data-root archive taken in the afternoon are not automatically a consistent recovery point. A reference may have been created or deleted between the two snapshots and exist in only one of them.

As a production baseline, treat a coordinated application DB + local-auth DB + AppFs snapshot taken under controlled stop/drain as one recovery point. Redis session/cache/rate-limit state, by contrast, is not business source of truth.

8.12 Static assets and uploaded files are different

CSS, JavaScript, icons, and other versioned assets shipped with the program have a different lifecycle from user uploads. The static root should be a read-only deployment artifact; the AppFs data root should be a separate, controlled writable runtime area.

This separation prevents several classes of mistakes:

- deployment cannot overwrite user media;
- upload cannot modify application code or versioned assets;
- backup clearly distinguishes business data from reinstallable release artifacts;
- reverse-proxy rules can be designed more cleanly.

8.13 What not to do

- Do not use the client filename directly as a filesystem path.
- Do not trust multipart **Content-Type**.
- Do not expose the entire AppFs data root through an Apache/Nginx alias.
- Do not manually interpolate an **Image** into a URL or HTML.
- Do not treat DB and AppFs as unrelated recovery states when references exist between them.
- Do not bypass operator upload/pixel limits from application code.

8.14 Current media-surface limits

The V1 media surface is intentionally small. Do not build on undocumented assumptions. The current baseline does not promise an automatic thumbnail/resize pipeline, WebP/AVIF conversion, EXIF normalization or metadata stripping, a complete admin media browser, S3/object-storage backends, or general Markdown image syntax.

If an application needs one of these, treat it as a separate architectural feature with its own resource, security, and lifecycle rules.

8.15 Developer upload checklist

- | | |
|-----------------------------------|--|
| 1. Auth/authorization correct? | 2. AppFs root separate and non-public? |
| 3. Reasonable byte limit? | 4. Pixel limit for images too? |
| 5. Using Image for images? | 6. Filename/MIME treated only as metadata? |
| 7. Typed reference in DB? | 8. @image in HTML? |
| 9. DB + AppFs backed up together? | 10. No reverse-proxy alias for data root? |

Chapter 9

JSON API, CORS, and browser client integration

RWLang does not force you to choose between classic server-rendered HTML and a JSON API. The same application can contain pages returning `Html`, actions processing forms, and typed JSON endpoints. This is useful both when a small JavaScript component needs some data and when a separate frontend application uses the RWLang server as an API.

This chapter cleanly separates three operating models:

1. **same-origin HTML + API**: the browser loads the page and calls the API from the same origin;
2. **cross-origin public API**: another origin calls the API, but there is no session cookie;
3. **cross-origin session auth**: a separate frontend origin uses cookie-based sessions, so CORS and CSRF policy are both required.

The most important rule is: **CORS is neither authentication nor CSRF protection**. CORS is a browser-side access policy; authentication decides who the user is, while CSRF protection prevents an authenticated session from being abused for unauthorized state changes from another origin.

9.1 The first typed JSON endpoint

A JSON endpoint handler returns `Json`. `json(...)` converts a supported runtime value into a JSON representation.

```
page fn statusApi(ctx: PageContext) -> Result<Json, PageError> {
  let healthy = true;
  return Ok(json(healthy));
}

route statusApi GET "/api/status" => statusApi;
```

A successful response is JSON, for example:

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8

true
```

`json(value)` is not an arbitrary object serializer. Supported values are constrained: scalar values, model records, optional models, and lists of models. This matches RWLang's typed-boundary philosophy: the response shape does not emerge from an arbitrary runtime-changing data structure.

9.2 JSON lists from the database

Typed SQL and JSON responses connect directly. The query still returns an exact model shape, and the JSON layer represents that typed value.

```
model Product {
  id: Int
  name: String
  price: Int
}

query fn listProducts(db: Db, limit: Int, offset: Int)
  -> Result<List<Product>, DbError> sql {
  SELECT id, name, price
  FROM products
  ORDER BY id
  LIMIT :limit OFFSET :offset
}

page fn productsApi(ctx: PageContext, db: Db,
  limit: Int, offset: Int)
  -> Result<Json, PageError> {
  let products = listProducts(db, limit, offset)?;
  return Ok(json(products));
}

route productsApi GET "/api/products"
  query limit<Int> offset<Int>
  validate limit range 1 100 offset range 0 1000000
  => productsApi;
```

The important point is not the JSON but the fact that input remains typed and validated just as it is for an HTML route. A client cannot bypass route policy by supplying an unbounded `limit`.

9.3 Typed JSON POST body

A route declares `json` input mode for a JSON request body.

```
action fn echoApi(ctx: ActionContext,
                  name: String,
                  age: Int,
                  active: Bool)
  -> Result<Json, PageError> {
  return Ok(json(name));
}

route echoApi POST "/api/echo"
  json name<String> age<Int> active<Bool>
  validate name length 1 100 age range 0 150
  auth user
  => echoApi;
```

A valid request is:

```
POST /api/echo HTTP/1.1
Content-Type: application/json
Accept: application/json
X-CSRF-Token: <session CSRF token>

{"name":"Alice","age":42,"active":true}
```

The JSON input schema is **closed**. Unknown or missing keys, duplicate keys, wrong scalar types, and unsupported nested objects, arrays, floats, or `null` values are errors. These are request-contract violations and therefore return **400 Bad Request**. A missing or incorrect `Content-Type` on a JSON route returns **415 Unsupported Media Type**. Field and request-size limits apply just as they do to other request bodies.

The `json`, `form`, and `upload` body modes are mutually exclusive on the same POST route, and the compiler enforces this. JSON expects `application/json`. Simple forms use URL-encoded form data; uploads use multipart form data. Each endpoint should have one unambiguous request contract.

9.4 Content negotiation and 406

The client can use the `Accept` header to state what representation it can receive. For a JSON route, supported examples include:

```
Accept: application/json
Accept: application/*
Accept: */*
```

If a handler would return JSON but the client accepts only `text/html`, the server returns **406 Not Acceptable**. This is preferable to silently returning a representation the client explicitly rejected.

9.5 API errors

On JSON routes, runtime-classified application errors use a simple common envelope: a stable **error** code, while detailed diagnosis remains in server-side logs. Clients should not parse the English text of error messages; they should use the HTTP status and stable error code.

Once a route is known to return JSON, errors use a JSON envelope, for example:

```
HTTP/1.1 401 Unauthorized
Content-Type: application/json; charset=utf-8

{"error": "unauthorized"}
```

Typical error codes include:

- `bad_request`;
- `unauthorized`;
- `forbidden`;
- `csrf_failed`;
- `unsupported_media_type`;
- `database_unavailable`;
- `resource_limit`;
- `internal_error`.

This is especially important for browser clients. An authenticated JSON route returns **401** to an anonymous request rather than an HTML login redirect. The frontend can therefore decide based on status code without having to interpret HTML as an API error.

9.6 Same-origin: the default and simplest model

If HTML and API are served from the same scheme + host + port combination, they are same-origin from the browser's point of view.

```
https://app.example/
https://app.example/api/products
```

No CORS configuration is normally needed. A simple client is:

```
const response = await fetch("/api/products?limit=20&offset=0", {
  headers: { Accept: "application/json" }
});

if (!response.ok) {
  throw new Error(`HTTP ${response.status}`);
}

const products = await response.json();
```

Prefer this architecture when there is no real business need for a separate frontend origin. It means fewer trust boundaries, a simpler cookie policy, and less deployment configuration.

9.7 CORS: exact-origin allowlist

Cross-origin browser access is denied by default. Trusted server configuration must enumerate exact origins; there is no wildcard origin.

```
[web]
cors_origins = ["https://frontend.example"]
cors_allow_credentials = false
```

The CLI equivalent is:

```
rwlang-server --cors-origin https://frontend.example ...
```

Use multiple explicit allowlist entries for multiple origins. Do not build a policy that blindly reflects the request's `Origin` value.

9.8 Preflight

Before some cross-origin requests, the browser sends an `OPTIONS` preflight. RWLang allows the preflight only when:

- the `Origin` is configured;
- the target `GET/POST` route exists;
- the requested request headers are supported.

Supported browser-API headers include `Content-Type`, `Accept`, and `X-CSRF-Token`. Preflight is therefore not a generic “allow everything” `OPTIONS` endpoint; it is part of the real route and origin policy.

9.9 Credentialed CORS with a session cookie

When a frontend on a different origin uses RWLang session authentication, credentialed CORS is required:

```
[web]
cors_origins = ["https://frontend.example"]
cors_allow_credentials = true
```

In production this is allowed only over `HTTPS`. With this configuration the session cookie uses `SameSite=None; Secure`, because the browser must send it on cross-site requests.

The JavaScript `fetch` call also needs explicit credential mode:

```
const response = await fetch("https://api.example/api/account", {
  credentials: "include",
  headers: { Accept: "application/json" }
});
```

`credentials: "include"` does not grant permission. It only tells the browser that matching cookies may be sent for a request allowed by the CORS policy.

9.10 CSRF on a JSON API

With cookie-based sessions, the browser manages the session cookie automatically, so state-changing JSON requests need CSRF protection just like HTML forms.

For form POST, the token is a body field:

```
_csrf=<token>
```

For JSON POST, use a header:

```
X-CSRF-Token: <token>
```

For a separate frontend origin, an authenticated token endpoint can be defined:

```
page fn csrfApi(ctx: PageContext) -> Result<Json, PageError> {  
  return Ok(json(csrfToken));  
}  
  
route csrfApi GET "/api/csrf" auth user => csrfApi;
```

The token belongs to the same session. A CORS allowlist does not replace this check.

9.11 Complete browser POST flow

A session-auth frontend running from a separate `frontend.example` origin typically does the following:

1. the user authenticates and the browser receives the session cookie;
2. the frontend fetches the CSRF token belonging to that session;
3. POST is sent with `credentials: "include"`;
4. the JSON body conforms to the route's closed typed schema;
5. `X-CSRF-Token` contains the session token;
6. the runtime verifies origin, authentication, CSRF, and input validation;
7. only then does the handler perform the business operation.

Example client:

```
const csrf = await fetch("https://api.example/api/csrf", {  
  credentials: "include",  
  headers: { Accept: "application/json" }  
}).then(async r => {  
  if (!r.ok) throw new Error(`CSRF HTTP ${r.status}`);  
  return r.json();  
});  
  
const result = await fetch("https://api.example/api/echo", {  
  method: "POST",  
  credentials: "include",  
  headers: {  
    "Content-Type": "application/json",  
    Accept: "application/json",  
    "X-CSRF-Token": csrf  
  },  
  body: JSON.stringify({ name: "Alice", age: 42, active: true })  
});
```

```
}).then(async r => {
  if (!r.ok) throw new Error(`API HTTP ${r.status}`);
  return r.json();
});
```

9.12 Reverse proxy and origin

Behind Apache or Nginx, public scheme/host and trusted-proxy configuration must agree. The application must not decide what origin is safe based on arbitrary forwarded headers supplied by the client. The installation chapter explains the trusted-proxy boundary; the full `server.toml` key reference appears near the end of the book.

Always place the **frontend origin as seen by the browser** in the CORS allowlist, for example:

```
https://frontend.example
```

not an internal proxy or container address.

9.13 What not to do

- Do not enable CORS merely because “this is an API”.
- Do not reflect arbitrary request `Origin` values.
- Do not treat CORS as authentication or CSRF protection.
- Do not send session credentials over HTTP in production.
- Do not mix form, JSON, and upload body modes on the same POST route.
- Do not leave list-query parameters unbounded; API input is validated input too.
- Do not build JSON-client logic around an HTML login redirect; handle 401/403 statuses.

9.14 Which architecture should you choose?

Situation	Recommended model
server-rendered page + small JS	same-origin HTML + JSON API
SPA on the same domain	same-origin API, no CORS
separate public frontend, no session	exact-origin CORS without credentials
separate frontend + session auth	exact-origin credentialed CORS + HTTPS + CSRF
server-to-server client	CORS is irrelevant; auth and network policy matter

9.15 Runnable example

The complete repository example is `examples/json-api/app.rw`.

Try it first from the same origin. Introduce a separate frontend origin and CORS only when the deployment genuinely requires it. This keeps it clear which problem is solved by the JSON API and which is solved by the browser’s cross-origin policy.

Chapter 10

Static assets and frontend integration

A web application produces more than dynamic HTML and JSON responses. CSS, icons, images, fonts, source maps, or artifacts from a separate frontend build have a different lifecycle: they arrive as part of a release, ideally remain immutable, and can be cached aggressively by the browser.

RWLang therefore serves static assets from a separate, **read-only static root**. This is not the same area as the writable AppFs data root, and it is not the same concept as user-uploaded media.

This chapter answers four questions:

1. where CSS, images, fonts, and other build artifacts should live;
2. how they can have a long cache lifetime without trapping clients on old versions;
3. when RWLang should serve them and when Apache/Nginx may serve them directly;
4. what the V1 CSP model means for JavaScript.

10.1 Three separate file lifecycles

In production, keep at least three file categories separate:

	Example	Writable?	Lifecycle
release	CSS, font, logo	read-only	replaced on deploy
runtime data	export, generated file	according to AppFs policy	changes while the app runs
user media	uploaded image	controlled writable	lives with business data

A typical directory structure is:

```
/usr/local/bin/rwlang-server
/usr/local/bin/rwlang-cli
/srv/myapp/releases/2026-09-04/
  main.rw
  migrations/
  public/
    css/
    img/
```

```

    fonts/
/srv/myapp/data/
/run/secrets/rwlang/

```

Treat `public/` as a deployment artifact. `data/`, by contrast, is runtime state. If they are mixed, an upload may overwrite a release file, or a rollback may accidentally roll user data backward.

10.2 Configuring the static root

Static namespace configuration lives in a separate server-config section:

```

[static_assets]
root = "/srv/myapp/current/public"
url_prefix = "/assets/"
max_asset_bytes = 8388608
max_age_secs = 300
immutable_max_age_secs = 31536000
precompressed = true

```

The same options are available through the CLI. Important flags include:

```

--static-root
--static-url-prefix
--max-static-asset-bytes
--static-max-age-secs
--static-immutable-max-age-secs

```

The static prefix is a reserved route namespace. If the prefix is `/assets/`, the application must not declare a conflicting dynamic or static route. The server should fail closed on collision: startup failure is better than ambiguous route selection per request.

10.3 Fingerprints: the key to long caching

Tie static content and URL together. For example:

```

public/
  css/app.01234567.css
  img/logo.a91f028c.svg
  fonts/ui-f1e2d3c4.woff2

```

RWLang treats a filename as fingerprinted when it contains a token of at least eight hexadecimal characters delimited by a dot, hyphen, or underscore. Such a file may receive a long immutable cache policy:

```
Cache-Control: public, max-age=31536000, immutable
```

For a non-fingerprinted file the safer default is short and revalidatable:

```
Cache-Control: public, max-age=300, must-revalidate
```

The key rule is: **do not replace content behind the same URL while using a one-year cache**. A new CSS build should get a new fingerprinted filename. Old files may remain in the release long enough for old HTML representations to age out.

10.4 A minimal asset build pipeline

RWLang does not prescribe a frontend build system. The pipeline only needs to produce deterministic output into `public/`.

For a small application, source might look like:

```
assets-src/  
  app.css  
  logo.svg  
public/
```

A release build may:

1. create a clean `public/` directory;
2. build or copy CSS/fonts/images;
3. compute a content hash;
4. create fingerprinted filenames;
5. optionally produce Brotli/gzip siblings;
6. use the fingerprinted URLs from the application.

The hash is not a security token. Its purpose is to identify the content version in the filename.

10.5 ETag and conditional GET

Every static representation receives its own ETag. The browser can later send:

```
If-None-Match: "rw-..."
```

If the representation has not changed, the server may return `304 Not Modified` without a body. `HEAD` is supported alongside `GET`.

A static request does not create a session and does not send a session cookie. This matters for both performance and security: downloading CSS or a font should not become a session-lifecycle event.

10.6 Precompressed Brotli and gzip

The RWLang server does not compress static files during the request. A release pipeline may prebuild:

```
app.01234567.css  
app.01234567.css.br  
app.01234567.css.gz
```

Based on `Accept-Encoding`, the server chooses Brotli, then gzip, then the original representation. A compressed response includes headers such as:

```
Content-Encoding: br
Vary: Accept-Encoding
```

This avoids per-request compression CPU cost and does not expose dynamic compression work as part of the web request resource surface.

10.7 MIME, nosniff, and path confinement

The server maps known web extensions to MIME types and uses `application/octet-stream` for unknown extensions. Responses always include:

```
X-Content-Type-Options: nosniff
```

Static paths are intentionally strict. Valid:

```
/assets/css/app.01234567.css
```

Rejected forms include `..`, encoded traversal, backslashes, empty path components, and control characters. V1 does not allow percent-encoding in static paths; the build pipeline should use URL-safe filenames.

On Linux, static-root access follows the same confinement philosophy as AppFs: serving assets must not become arbitrary host-filesystem reads through symlinks or traversal.

10.8 CSS, images, and fonts in RWLang HTML

The V1 CSP allows same-origin style, image, font, and media assets. A normal release may therefore contain:

```
/assets/css/app.01234567.css
/assets/img/logo.a91f028c.svg
/assets/fonts/ui-f1e2d3c4.woff2
```

HTML should always reference the fingerprinted URL that belongs to the release. Templates must not construct asset URLs from user input; the asset name is build data, not request data.

10.9 Important V1 boundary: JavaScript may be served, but not executed in RWLang HTML

The static server knows the MIME types for `.js` and `.mjs`, so those build artifacts can physically be served. This does **not** mean that HTML rendered by RWLang may execute scripts.

In the V1 HTML security model:

- CSP does not grant a `script-src` permission;
- the compiler rejects `<script>` markup;
- there is no inline-script or “unsafe” escape hatch.

This is an intentional security boundary. JavaScript `fetch` examples in the previous chapter therefore have two valid interpretations:

1. a separate frontend application runs on its own origin and calls an RWLang JSON API;
2. the code illustrates browser-API client behavior, not that a script tag can be embedded in V1 RWLang HTML.

For a classic RWLang server-rendered application, build around HTML + CSS + typed form/action. If a full JavaScript frontend is required, deploy it separately and use RWLang as a typed JSON backend.

10.10 Should RWLang serve assets, or Apache/Nginx?

Two deployment models work.

1. RWLang-owned static namespace

This should be the default. The reverse proxy sends every request to RWLang, which handles the `/assets/` namespace itself:

```
Internet
|
Apache / Nginx :443
|
127.0.0.1:8080 RWLang
|-- application routes
`-- /assets/*
```

The advantage is that fingerprint, ETag, precompressed, nosniff, and path policy stay in one place. This is usually more than sufficient for small and medium applications.

2. The reverse proxy serves assets directly

At higher static traffic volumes, Apache or Nginx may serve the read-only release `public/` directory directly. If you do this, **you must reproduce** the important contract yourself:

- separate read-only static root;
- traversal/symlink-safe path handling;
- correct MIME + `nosniff`;
- long immutable cache for fingerprinted files;
- short/revalidatable cache for non-fingerprinted files;
- correct `Vary` for Brotli/gzip;
- no namespace collision with application routes.

This is an operator optimization, not a requirement for application correctness.

10.11 Nginx example for direct static serving

A simple starting point:

```
server {
    listen 443 ssl;
    server_name app.example;

    location /assets/ {
        alias /srv/myapp/current/public/;
        try_files $uri =404;
        add_header X-Content-Type-Options nosniff always;
    }

    location / {
        proxy_pass http://127.0.0.1:8080;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-Proto https;
    }
}
```

This is only a topology baseline. Production configuration must align cache policy, precompressed files, and symlink/release rules with the same immutable deployment model.

10.12 Apache example for direct static serving

The idea is the same with Apache:

```
Alias /assets/ /srv/myapp/current/public/

<Directory /srv/myapp/current/public/>
    Require all granted
    Options -Indexes
    AllowOverride None
    Header always set X-Content-Type-Options "nosniff"
</Directory>

ProxyPass          / http://127.0.0.1:8080/
ProxyPassReverse   / http://127.0.0.1:8080/
```

If Apache serves assets directly, the `Alias` rule must take effect before the proxy catch-all, and cache headers must be configured explicitly.

10.13 Deploy application code and assets as one release

A common asset failure is deploying HTML that references the new CSS filename while the static root still points to the old release – or vice versa.

Use this order:

```
build application
-> build/fingerprint assets
-> optional .br/.gz
-> immutable release directory
-> preflight
-> atomic current switch
-> controlled restart/reload
```

Application code and the `public/` directory should be part of the same release artifact. Rollback then restores both together.

10.14 Checking with curl

After deployment, check at least:

```
curl -I https://app.example/assets/css/app.01234567.css
curl -I -H 'Accept-Encoding: br' \
  https://app.example/assets/css/app.01234567.css
curl -I -H 'If-None-Match: "rw-..."' \
  https://app.example/assets/css/app.01234567.css
```

Verify `Content-Type`, `Cache-Control`, `ETag`, `X-Content-Type-Options`, and, for compressed representations, `Content-Encoding` and `Vary`.

10.15 Practical decision rule

Situation	Recommended approach
server-rendered RWLang page	RWLang static root + CSS/image/font
small/medium production	proxy forwards everything to RWLang
high static traffic	Apache/Nginx direct static with equivalent policy
full JavaScript frontend	separate frontend deployment + RWLang JSON API
user upload	AppFs/media pipeline, not static root

The most important rule is: **a static asset is a deployment artifact, not application data**. Preserving that boundary simplifies caching, rollback, backup, and security at the same time.

Chapter 11

Organizing larger applications

In an application with only a few routes, it may still be convenient to keep every model, query, and handler in a single `main.rw`. In a wiki, CMS, or line-of-business system, however, *structure* soon matters more than the syntax of individual language features. RWLang provides two complementary tools: `mod` organizes source files, while `object` defines the namespace of a domain API.

11.1 Three separate questions

Treat the structure of a larger application as three distinct layers:

Tool	Responsibility
<code>mod</code>	In which file or source module does the code live?
<code>object</code>	Which business concept owns the model, query, page, or action?
<code>route</code>	What HTTP surface and what auth/cache/rate-limit policy expose it?

This separation is deliberate. A domain object does not automatically receive a database, current user, filesystem, or network access, and a module does not imply HTTP policy.

11.2 `mod` is not textual include

A small multi-file project might look like:

```
myapp/  
|-- main.rw  
|-- models.rw  
|-- queries.rw  
|-- components.rw  
|-- pages.rw  
|-- actions.rw  
|-- migrations/  
`-- public/
```

The entry point:

```
mod models;  
mod queries;  
mod components;
```

```
mod pages;  
mod actions;
```

`mod` is not C-style textual inclusion and it does not inject declarations into a global namespace. The compiler loads, validates, and compiles the explicit source graph into one program. A module owns a namespace: declarations from `queries.rw`, for example, are referenced as `queries::name` across module boundaries. Module paths are application-root relative, so nested source files must be loaded explicitly with paths such as `mod article::admin;`.

11.3 When should you switch to feature-based structure?

Splitting by technical type – `models.rw`, `queries.rw`, `pages.rw` – is a good first step, but in a larger application it can mean opening five or six files for one feature change. At that point, organize by business area instead:

```
main.rw  
article.rw  
article/  
|-- admin.rw  
`-- components.rw  
account.rw  
account/  
`-- profile.rw  
invoice.rw  
invoice/  
`-- admin.rw  
routes.rw  
migrations/  
public/
```

Then `main.rw` becomes mostly application composition:

```
mod article;  
mod article::admin;  
mod article::components;  
mod account;  
mod account::profile;  
mod invoice;  
mod invoice::admin;  
mod routes;
```

The goal is not to maximize file count. The goal is for one business change to remain mostly inside one feature directory.

11.4 Domain object: a business namespace, not an OOP class

object groups related business names inside a domain namespace. For example:

```

object Article {
  model {
    id: Int
    slug: Slug
    title: String
    body: String
    authorUsername: String
  }

  query fn bySlug(db: Db, slug: Slug)
    -> Result<Article, DbError> sql {
    SELECT id, slug, title, body, authorUsername
    FROM articles
    WHERE slug = :slug
  }

  page fn show(ctx: PageContext, db: Db, slug: Slug)
    -> Result<Html, PageError> {
    let article = Article.bySlug(db, slug)?;
    return Ok(html {
      <article>
        <h1>{{ article.title }}</h1>
        @markdown(article.body)
      </article>
    });
  }
}

```

Usage becomes expressive:

```
let article = Article.bySlug(db, slug)?;
```

Short domain names become natural too, for example `Invoice.approve`, `Invoice.cancel`, and `UserAccount.disable`.

What may an object contain?

A V1 domain object can group operations around one model. The following is only a structural sketch, not standalone compilable RWLang source:

```

object Article {
  model { ... }
  query fn bySlug(...) ...
  page fn show(...) ...
  action fn publish(...) ...
  component fn Card(...) ...
  layout fn EditorLayout(...) ...
}

```

Exactly one model block belongs to an object. An object is not a classic object-oriented class: there is no inheritance, mutable `self`, constructor magic, virtual dispatch, or reflection.

11.5 Dependencies remain explicit

The domain namespace does not hide capabilities. This is good:

```
query fn byId(db: Db, id: Int)
  -> Result<Article, DbError> sql {
    SELECT id, slug, title, body, authorUsername
    FROM articles
    WHERE id = :id
  }
```

An action receives the capabilities it needs just as explicitly. The following is intentionally abbreviated structural code; the concrete typed state-changing query is shown in the CRUD and business-audit sections:

```

action fn publish(ctx: ActionContext, db: Db, id: Int)
  -> Result<Redirect, PageError> {
  let article = Article.byId(db, id)?;
  authorize article owner authorUsername or role Publisher;
  // ... state transition inside a transaction ...
  return Ok(redirect("/admin/articles"));
}

```

The handler signature still reveals database access. The same principle applies to AppFs, auth, and later network egress: a domain object must not become a hidden service locator.

11.6 Why should routes remain top-level?

Routes deliberately do not disappear entirely into objects:

```

route articleShow GET "/article/:slug<Slug>"
  cache public ttl 60
  => article::Article.show;

route articlePublish POST "/admin/article/:id<Int>/publish"
  auth role Publisher
  invalidate cache articleShow
  => article::Article.publish;

```

A route registry or code review can then see in one place:

- URL and HTTP method;
- authentication/authorization requirement;
- cache policy and invalidation;
- rate limit or resource profile;
- the global route name used in logs and audit.

A domain member name may be short and locally meaningful, such as `Article.publish` inside its own module; from a separate `routes.rw`, use `article::Article.publish`. The route name should remain globally descriptive, such as `articlePublish`.

11.7 Recommended naming convention

Element	Form	Example
object	PascalCase business noun	<code>Article</code> , <code>Invoice</code> , <code>UserAccount</code>
member	camelCase, short inside domain	<code>bySlug</code> , <code>publish</code> , <code>approve</code>
route	globally auditable name	<code>articleShow</code> , <code>invoiceApprove</code>
module	feature or technical unit	<code>article</code> , <code>account</code> , <code>routes</code>

11.8 One possible CMS structure

A medium-size CMS might use:


```
main.rw
routes.rw
article.rw          # Article object + base queries
article/
|-- admin.rw        # edit/publish
`-- components.rw   # Card, lists
media.rw
media/
`-- admin.rw
account.rw
migrations/
public/
`-- app.css
```

Public and admin HTTP policy remain in the route layer. The article feature keeps the article business model and its operations together.

11.9 What not to do

Do not create one file per declaration

Overly small modules create more navigation than structure. A fifty-line feature does not need eight files.

Do not build a hidden “service” layer

If an operation uses DB, AppFs, or auth context, keep that visible in the typed signature. One of RWLang’s strengths is precisely that capability boundaries are explicit.

Do not duplicate route policy inside handlers

Route-level auth, cache, rate limit, and resource policy are declarative contracts. Domain-specific object authorization naturally remains in the handler because it depends on the business record.

Do not use free-form String for business state

If a domain has Draft, Review, Published, or Cancelled states, use an enum. Project structure helps most when the domain contract is typed too.

11.10 Refactoring path: from one file to a larger application

You do not need to design the final directory hierarchy on day one. A good evolutionary path is:

1. Start with one `main.rw` while the application is small.
2. Separate model/query/page/action groups when the file becomes hard to navigate.
3. If one feature is scattered across many technical files, switch to feature-based modules.
4. If too many global business names appear, group them under `object` namespaces.

Part III

Practical applications

Chapter 12

Practical applications: wiki and CMS

The next two examples use the same foundations but demonstrate different product needs. The minimal wiki combines stable slugs, Markdown content, and authenticated editing; a more mature wiki can later add optimistic locking and an audit trail. The CMS example adds a multi-file project structure, editor authorization, and public cache, then lists the additional layers a production CMS usually needs.

The minimal wiki is also checked as a compilation fixture in `examples/wiki/main.rw`; the CMS is the multi-file `examples/news-site/` project. These examples are therefore not merely book pseudocode.

12.1 A simple wiki

A minimal wiki domain consists of pages: stable identifier, slug, title, and Markdown body. The following example focuses on connecting the language elements.

Model

```
model WikiPage {
  id: Int
  slug: Slug
  title: String
  body: String
}
```

Query

```
query fn wikiBySlug(db: Db, slug: Slug)
-> Result<WikiPage, DbError> sql {
  SELECT id, slug, title, body
  FROM wiki_pages
  WHERE slug = :slug
}
```

Public page

```
page fn wikiShow(ctx: PageContext, db: Db, slug: Slug)
  -> Result<Html, PageError> {
    let page = wikiBySlug(db, slug)?;
    return Ok(html {
      @layout(Main, page.title) {
        <article>
          <h1>{{ page.title }}</h1>
          @markdown(page.body)
        </article>
      }
    });
  }

route wikiShow GET "/wiki/:slug<Slug>" => wikiShow;
```

Editor form

```
form WikiForm {
  title<String>
  body<String>
  validate title length 2 200 body length 1 300000
}
```

The mutating route should require authentication, for example `auth role Editor`. The write query should receive a `Transaction` capability. Do not blindly regenerate and overwrite the slug from the title on every edit if preserving stable old URLs matters.

Canonical slug and aliases

A more mature wiki can maintain old names in a separate slug-alias/history table. The runtime can produce a 301 redirect from a canonical route, so the application does not need to build arbitrary redirect URLs.

What should a real wiki add?

- optimistic locking for concurrent edits;
- a business audit trail for changes;
- object-level authorization if not every page has identical access rules;
- backup/restore policy for the database and any AppFs state.

12.2 CMS and news site

The repository's `examples/news-site/` project already demonstrates a small CMS core: model, typed SQL, layout, Markdown, editor action, and public cache.

Project structure

```
main.rw
models.rw
queries.rw
components.rw
```

```
pages.rw
actions.rw
```

Article model

```
model Article {
  id: Int
  slug: Slug
  title: String
  body: String
}
```

Editor input gets a separate named form schema:

```
form ArticleForm {
  title<String>
  body<String>
  validate title length 2 200 body length 1 300000
}
```

Public article page

```
page fn article(ctx: PageContext, db: Db, slug: Slug)
  -> Result<Html, PageError> {
  let article = articleBySlug(db, slug)?;
  return Ok(html {
    @layout(Main, article.title) {
      <article>
        <h1>{{ article.title }}</h1>
        @markdown(article.body)
      </article>
    }
  });
}

route article GET "/article/:slug<Slug>"
  cache public ttl 60
  => article;
```

Public cache is appropriate only for a page that does not depend on session or user state.

Editor-side creation

```
action fn create(ctx: ActionContext, db: Db,
  title: String, body: String)
  -> Result<Redirect, PageError> {
  let articleSlug = slug(title);
  transaction db {
    insertArticle(tx, articleSlug, title, body)?;
  }
  return Ok(redirect("/admin/articles"));
}
```

```
route articleCreate POST "/admin/articles"  
  form ArticleForm  
  auth role Editor  
  invalidate cache article  
  => create;
```

The invalidation above changes the route-level generation: it does not delete just one cache key for the new slug, but makes the previous generation of the `article` route unreachable. This is correct and predictable in a simple example; in a high-traffic CMS you should deliberately decide which public routes a given mutation really needs to invalidate.

From CMS to production system

The following additions form the backbone of a practical production CMS:

- separate validation for publication and domain invariants;
- draft/published state as an enum;
- canonical slug history;
- optimistic locking during editing;
- business audit trail;
- media library and AppFs for images;
- role-based editor/publisher policy;
- rate limits and resource profiles for expensive routes;
- backups, restore drills, and controlled deployment.

In a CMS, original Markdown should remain the source of truth; do not store raw rendered HTML produced by the application.

Part IV

Advanced integration and quality

Chapter 13

External services and egress policy

A real web application rarely remains completely self-contained. It may need to reach a payment provider, directory service, mapping API, internal corporate service, or another HTTPS endpoint. This is a different security boundary from an incoming HTTP route: here the application itself initiates a network connection.

13.1 First, the V1 boundary

V1 has **no general RWLang HttpClient or Secrets language API**. Application code therefore does not receive arbitrary network access. The following is intentionally not RWLang syntax; it illustrates a shortcut you must not assume exists:

```
// There is NO general V1 surface like this:  
// let r = http.get(user_supplied_url);
```

The repository's `integrations` crate does contain a trusted Rust adapter layer with:

- a policy-controlled outbound HTTPS client;
- named egress targets;
- host, CIDR, port, and TLS restrictions;
- send/receive size limits and timeouts;
- a file-based `SecretsStore`.

This is an important design decision: **if a capability is not in the language surface, do not build an application-level bypass**. External integration should be implemented as a trusted adapter or as a later, separately designed language/runtime feature.

13.2 Why is an arbitrary URL dangerous?

Suppose a route could directly fetch a URL supplied by the user. The application could easily be tricked into contacting internal or local services:

```
http://127.0.0.1:...  
http://169.254.169.254/...  
http://internal-db.example/...  
http://[::1]/...
```


This is classic SSRF (Server-Side Request Forgery). The correct model is therefore not “the request tells us where to connect,” but “trusted configuration predefines a named target.”

13.3 Named target: configuring a network capability

A target may look like:

```
[[target]]
name = "payments"
hosts = ["api.payments.example"]
cidrs = ["203.0.113.0/24", "2001:db8:1200::/48"]
ports = [443]
tls_required = true
max_dns_answers = 8
max_sent_bytes = 262144
max_received_bytes = 2097152
connect_timeout_ms = 5000
total_timeout_ms = 15000
```

The name – here **payments** – is the capability known to the application/integration. Hostnames and network ranges are trusted operator policy.

The same pattern is available in `examples/egress/policy.toml`.

13.4 Policy checks multiple layers

Egress policy does more than check a hostname.

Host

The requested host must appear in the target’s explicit **hosts** list. Use a named host that can be validated by TLS rather than an IP-literal hostname.

Port and TLS

The port must be on the allowed list. The current adapter requires TLS; the usual target is HTTPS on port 443.

DNS and CIDR

Every A and AAAA answer returned by DNS is checked. The rule is fail-closed: if even one resolved address falls outside the allowed CIDRs, the entire connection is rejected.

This matters because a hostname allowlist by itself does not protect against DNS rebinding or incorrect DNS configuration.

Peer check after connect

After the TCP connection is established, the adapter checks the actual peer IP again. It must match the selected address and still satisfy the CIDR policy.

TLS identity

Certificate verification uses the original hostname. CIDR policy does not replace TLS identity verification; the two checks reinforce each other.

13.5 IPv4 and IPv6 belong to the same policy

Outbound policy is dual-stack. A target can list both IPv4 and IPv6 CIDRs:

```
cidrs = [
    "203.0.113.0/24",
    "2001:db8:1200::/48"
]
```

The runtime normalizes IPv4-mapped IPv6 addresses. For example, `::ffff:203.0.113.9` is checked as IPv4, so it cannot bypass the IPv4 rule.

Loopback, link-local, ULA, multicast, and unspecified IPv6 ranges receive no implicit trust. If access to such a range is genuinely required, it must also be allowed by explicit CIDR policy.

13.6 Size and time limits

An upstream service can be broken or hostile just like an incoming client. Target policy therefore limits more than destination address:

Setting	What does it limit?
<code>max_dns_answers</code>	number of acceptable DNS answers
<code>max_sent_bytes</code>	maximum request size sent upstream
<code>max_received_bytes</code>	maximum response size accepted
<code>connect_timeout_ms</code>	duration of one connection attempt
<code>total_timeout_ms</code>	total outbound-operation time

The principle is the same as elsewhere in RWLang: **untrusted application code must not choose its own resource limits**.

13.7 A secret is not a configuration string

External APIs often require credentials. The trusted adapter can use a `SecretsStore` that reads by name from a dedicated secret root.

For example:

```
/run/secrets/rwlang/
  payments-bearer
```

A secret name cannot contain paths, `..`, or arbitrary separators. The adapter does not follow symlinks, limits file size, and on Unix verifies that the secret is not world-accessible.

`SecretString` has a redacted debug representation, and its stored bytes are overwritten on drop. Even so, the rule remains: credentials must not be written to logs, error messages, or audit payloads.

13.8 GET and JSON POST in the trusted adapter

The current Rust adapter provides two common operations: HTTPS GET and JSON POST with an optional bearer secret. This is **trusted Rust code**, not `.rw` syntax.

```
let policy = EgressPolicy::from_toml_file(policy_path)?;
let client = OutboundHttpClient::new(policy);
let secrets = SecretsStore::new("/run/secrets/rwlang"?);
let token = secrets.get("payments-bearer")?;

let response = client
    .post_json(
        "payments",
        "api.payments.example",
        443,
        "/v1/charge",
        body,
        Some(&token),
    )
    .await?;
```

Both the `payments` target and concrete host must satisfy trusted policy. An arbitrary host derived from the request cannot become a new target.

13.9 Error model

For external integration, distinguish these classes of failure:

- **Policy**: the requested target or resolved/peer address is not allowed;
- **Secret**: a credential cannot be read or violates secret-store rules;
- **DNS**: name resolution fails;
- **Connect/TLS**: network or identity failure;
- **Timeout**: upstream is too slow;
- **Protocol**: malformed HTTP response;
- **ResponseTooLarge/SendTooLarge**: a policy size limit is exceeded.

Do not collapse all of these automatically into the same business error. A policy denial is a configuration/security event; an upstream 503 is an availability problem.

13.10 Retries and idempotency

The current adapter does not provide a general automatic retry layer. That is the right default: retrying a POST can repeat a side effect.

Before adding retries to a later integration, answer:

1. is the operation idempotent;
2. does the upstream support an idempotency key;
3. which failures are retryable;
4. what is the total timeout budget;
5. how will retry storms be avoided?

For payments or webhook-like operations, an idempotency key is business data, not merely network tuning.

13.11 Synchronous request or background work?

The V1 surface has no general background-job language API. Do not hide a long external workflow behind a normal page/action request merely because a trusted adapter can technically perform it.

A short, well-bounded integration may fit into the request/response cycle. A long or reliably retryable workflow needs a separate explicit job/outbox feature; do not emulate that with ad-hoc threads or subprocesses.

13.12 What does this mean for an RWLang application developer?

The daily rules are simple:

1. Do not design outbound features around arbitrary user-supplied URLs.
2. Do not read secrets through raw filesystem or environment-variable shortcuts.
3. Make the integration destination a named capability.
4. Keep host/CIDR/port/TLS/timeout/size rules in trusted policy.
5. Let user input choose business payload, not network destination.
6. If the integration is not in the V1 language surface, use a trusted adapter or design a reviewed language/runtime feature.

13.13 Minimum testing

A happy path is not enough for egress policy. Prove at least:

- allowed host + allowed IPv4;
- allowed host + allowed IPv6;
- fail-closed behavior when a mixed A/AAAA answer contains one forbidden address;
- forbidden port;
- too many DNS answers;
- peer-IP revalidation;
- TLS hostname mismatch;
- IPv4-mapped IPv6 normalization;
- send/receive limits and timeouts;
- negative secret traversal/symlink/permission cases.

Repository unit tests may prove policy decisions, but release evidence should also include a real dual-stack network integration test.

13.14 The essence of this chapter

External HTTPS is not just a utility; it is a new trust boundary. RWLang V1 intentionally does not grant unrestricted network capability to application code. The current trusted adapter demonstrates the intended model: **named target + explicit network policy + bounded resources + separate secret store + fail-closed validation**.

Chapter 14

Testing, error models, and developer diagnostics

A web application is not correct merely because it compiles. The RWLang compiler intentionally rejects many mistakes before execution, but it cannot prove that a business workflow is correct, that the database is actually reachable, that a migration is compatible, that authorization policy matches the product requirements, or that the production topology works correctly.

Testing in RWLang is therefore best treated as a set of layers that build on one another:

1. **source check:** is the program valid in the language and policy sense;
2. **negative compile test:** is a dangerous or forbidden program actually rejected;
3. **application/database test:** does the business operation create the expected state;
4. **integration test:** do the real DB, Redis, LDAP, TLS, AppFs, or other dependencies work together correctly;
5. **system/smoke test:** does the installed system work over HTTP with the real configuration.

The goal is not for every test to prove the same thing, but for every class of failure to break at the cheapest possible layer.

14.1 The first gate: `rwlang-cli check`

The fastest development check is:

```
rwlang-cli check main.rw
```

On success, the CLI summarizes the recognized program, for example the number of models, queries, pages, actions, and routes. The check does not start an HTTP server or mutate a database; it validates the source contract.

Among other things, the compiler can catch here:

- type mismatches;
- mismatches between query result shape and `model`;
- invalid route or form declarations;
- forbidden HTML/script constructions;
- unsafe SQL construction or bind errors;
- incompatible auth/cache/policy combinations;

- certain violations of capability boundaries.

This is one of the key differences from a general-purpose scripting approach: a significant class of errors should be found before any HTTP request is made.

14.2 A negative test is as valuable as a positive test

For a security invariant, it is not enough to test that the correct program compiles. You must also prove that the forbidden program *does not* compile.

The repository release checks therefore run fixtures such as:

```
# This must compile successfully:
rclang-cli check examples/auth/app.rw

# This must fail:
rclang-cli check \
  examples/security/sql-injection-rejected.rw
```

Typical negative tests cover:

- dynamic SQL suitable for SQL injection;
- raw/script-like XSS attempts;
- optional values used without a presence check;
- invalid form/schema validation;
- user-dependent routes with public cache;
- public routes using object-level authorization guards;
- forbidden upload paths or mixed JSON/form body modes.

A security rule is strong when the project contains **both positive and negative evidence** for it.

14.3 Four different worlds of failure

When debugging, first decide which layer failed. The same user-visible symptom may require a completely different fix depending on the layer.

Compile-time error The source contract is invalid. `rclang-cli check` or the build stops. Fix the code.

Startup/config error The program is valid, but server configuration, a secret, TLS key, or hard policy is invalid. Use `-check-config`, server logs, and effective config.

Request/policy error The server runs, but a request is rejected by input, authentication, authorization, rate-limit, or content-negotiation policy. Use the HTTP status, request ID, access log, and audit log.

Dependency/runtime error DB, Redis, LDAP, filesystem, network, or a resource limit causes the failure. Use readiness, metrics, server logs, and real integration tests.

A bad debugging reflex is to start rewriting source code for every 500/503 symptom. Identify the layer first.

14.4 Test configuration separately

Source checking does not replace server-configuration validation:

```
rwlang-server \  
  --config /usr/local/etc/rwlang/server.toml \  
  --check-config
```

To inspect the fully resolved values before deployment:

```
rwlang-server \  
  --config /usr/local/etc/rwlang/server.toml \  
  --print-effective-config
```

`-check-config` is not a dependency integration test. A syntactically valid DB URL can still point to an unavailable database; that is why readiness and deployment smoke tests exist separately.

14.5 Migration: status, verify, apply, verify

When the database changes, migration state is part of the test process:

```
rwlang-cli migrate status \  
  --dir migrations \  
  --db-url-file /run/secrets/rwlang/migration-db-url  
  
rwlang-cli migrate verify \  
  --dir migrations \  
  --db-url-file /run/secrets/rwlang/migration-db-url
```

A good deployment order is:

```
source/config check  
-> migration verify  
-> migration apply  
-> migration verify  
-> application rollout  
-> readiness  
-> smoke test
```

Do not modify an already-applied migration merely to make a test green. Put new changes into a new migration.

14.6 Business tests: prove state, not implementation

A CRUD or CMS action test should not primarily ask whether a certain internal function was called. Assert the externally observable state.

For article publishing, for example:

1. create a draft article;
2. authenticate with the Publisher role;
3. submit the publish action;
4. verify the redirect;
5. read the article back and verify the **Published** state;
6. verify the business-audit record;

7. verify that the public page now shows the new state;
8. for a cached route, verify invalidation too.

This test remains valuable even if queries or module structure are later reorganized internally.

14.7 Database tests: SQLite does not prove everything

A separate temporary SQLite database is convenient for fast development tests. But if production uses PostgreSQL or MariaDB, test against the real backend at least in the release/integration suite.

Particularly sensitive areas include:

- transaction behavior;
- constraints and unique conflicts;
- concurrent mutation and optimistic locking;
- indexes/query plans;
- migration DDL;
- timeouts and connection availability.

A backend-independent typed query contract rules out many mistakes, but it does not make backend behavior identical.

14.8 Authentication and authorization test matrix

For a protected action, test at least:

Case	Expected result
No session	login/auth rejection
Valid user, missing required role	authorization deny
Required role present	successful operation
MFA-required route, no MFA	MFA rejection
Expired or revoked session	new authentication required
Invalid CSRF token on state change	request rejected
Object not owned by user	object-authorization deny
Admin/allowed actor	successful operation

When using LDAP/LDAPS or Redis, include real dependency integration tests. A mock does not prove TLS hostname validation, network timeout behavior, or Redis fail-closed semantics.

14.9 File, media, API, and cache tests

For these features, happy-path coverage is especially insufficient.

Upload and media

Test at least:

- a valid small file;
- byte-limit overflow;
- image pixel-limit overflow;
- malicious/invalid filename;

- path traversal or symlink escape rejection;
- interrupted-upload cleanup.

JSON/CORS

In addition to valid JSON, test incorrect content type, duplicate keys, unknown fields, `Accept` negotiation, preflight, and the interaction of CSRF and CORS for credentialed state changes.

Cache and rate limiting

Test that user-dependent pages cannot become public cache, that mutations invalidate the right cache generation, that shared state is actually shared across instances, and that Redis failure follows the chosen security fail-closed behavior.

14.10 An HTTP error code is not the same thing as a program bug

The client-visible status is part of the contract. Typical cases are:

Situation	HTTP
Malformed input or request-contract violation	400
No valid authentication on JSON/API route	401
Actor/policy does not allow the operation	403
Resource not found	404
Wrong HTTP method	405
Representation not acceptable	406
Concurrent modification / zero rows for Changed	409
Request/upload too large	413
Wrong or missing body content type	415
Host/trust-boundary mismatch	421
Named-form field error with rerender	422
HTTPS required	426
Rate limit	429
Request header too large	431
Internal unclassified server failure	500
Dependency or request resource temporarily unavailable	503

Never send SQL, secrets, physical filesystem paths, session tokens, or stack traces to the client. Detailed diagnosis stays in server-side logs and is correlated by request ID.

Status and representation are separate contracts

The same business situation may have a different representation on HTML and JSON surfaces. This is not inconsistency: status expresses the error category, while representation expresses the client-facing surface contract.

Situation	HTML route	JSON route
No session	303 See other to built-in login page	401 + JSON error envelope
Authorization denied	403	403 + JSON error envelope

Situation	HTML route	JSON route
Optimistic-lock conflict	409 conflict page, no-store	409 + JSON error=conflict
Named-form validation	422, form rerendered with field errors	not a JSON body mode; body contracts are exclusive
Record not found	404	404 + JSON error envelope

A successful mutating HTML action typically returns **303 See Other** for Post/Redirect/Get. A **303** is therefore not an error; it is part of browser control flow.

14.11 Debugging with request IDs

When an HTTP request fails, the best first diagnostic key is the request ID generated by the server. Starting from the client response or access log:

```
grep 'rw-...' /var/log/rwlang/access.log
grep 'rw-...' /var/log/rwlang/server.log
grep 'rw-...' /var/log/rwlang/audit.log
```

The three logs answer different questions:

- access: what arrived and what status ended the request;
- server: what technical/runtime event occurred;
- audit: whether a security or user-policy decision occurred.

Preserve this diagnostic chain in automated test failure output too.

14.12 Deployment smoke test

A green CI run does not prove that the freshly installed service actually started. After rollout, check at least:

```
curl --fail --silent \
  https://example.test/health/live

curl --fail --silent \
  https://example.test/health/ready

curl --fail --silent \
  https://example.test/
```

Keep smoke tests small and non-destructive. If you also test a state-changing endpoint, use isolated test data and idempotent cleanup; do not experiment on production content.

14.13 What does the repository `verify.sh` mean?

Inside the RWLang source tree, the full release-evidence entry point is:

```
./verify.sh
```

This is much more than an application source check. It runs Rust workspace tests, positive and negative RWLang fixtures, migration/auth/storage/runtime checks, and release-artifact invariants.

If you modify the RWLang implementation itself, this is the release minimum. If you only develop your own `.rw` application, the daily loop can be smaller, but your release process should still include source check, migration/config validation, relevant integration tests, and post-deployment smoke testing.

14.14 Recommended development cycle

A good baseline for everyday feature development is:

```
1. design domain/model change
2. write migration if needed
3. rwlang-cli check main.rw
4. fast application test
5. negative/security cases
6. migration verify
7. relevant integration test
8. rwlang-server --config server.toml --check-config
9. deploy to staging
10. readiness + HTTP smoke
11. production rollout
12. readiness + smoke + metrics/log review
```

The principle is that fast deterministic failures should happen early; expensive environment-dependent tests should run only after them.

14.15 Debugging decision order

When “it does not work,” proceed consistently:

1. Does it compile? Run `rwlang-cli check`.
2. Is the config valid? Run `-check-config`.
3. Did the service start? Inspect service and server logs.
4. Is the process alive? Check liveness.
5. Are dependencies reachable? Check readiness.
6. Does the request reach the route? Inspect access log and request ID.
7. Is policy rejecting it? Inspect audit log and trust-boundary status (for example forwarding 400, 401/403/421/426/429).
8. Is runtime/dependency failing? Inspect server log and metrics.
9. Is the business result wrong? Reproduce with isolated test data and inspect persistent state.

This is much faster than simultaneously changing code, proxy configuration, and database state.

14.16 Minimum application release checklist

- `rwlang-cli check` is green;
- migration history verify is green;
- server config check is green;

- positive and negative tests for critical business flows are green;
- auth/authorization/security regression cases are green;
- integration tests against production-equivalent DB/Redis/TLS are green;
- backup/restore readiness is known;
- staging readiness and smoke are green;
- after rollout, readiness, smoke, logs, and metrics are reviewed.

The goal of testing is not 100 percent line coverage. It is repeatable evidence for the system's important contracts: **what we allow, what we deny, what state we create, and how we behave when dependencies fail.**

Part V

Production and operations

Chapter 15

Cache, Redis, rate limiting, and multi-instance operation

With a single RWLang server instance, much state can appear naturally local. As soon as two or more instances serve the same application, three questions arise immediately: where does the session live, who counts the rate limit, and which instance's cache is authoritative? The production answer is not to let application code manipulate Redis keys directly. RWLang uses shared infrastructure as a runtime capability, while the application requests declarative policy.

This chapter separates four related topics:

- **session and MFA replay state**: user security state;
- **route rate limiting**: traffic-control state;
- **public page cache**: reproducible performance state;
- **multi-instance deployment**: deciding which state must live in a shared backend.

The current common production backend is Redis, but the application receives no general Redis API. That is an important security and portability boundary.

15.1 Why not a general Redis client?

V1 has no language operation that lets a program read or write arbitrary Redis keys. Redis is compiler/runtime-owned infrastructure:

Use	Who owns the key and rules?
session	runtime/auth layer
TOTP replay	runtime/auth layer
login rate limit	runtime/auth layer
route rate limit	runtime + trusted policy config
public page cache	compiler/runtime
business data	not Redis; application DB/AppFs

Do not store wiki articles, CMS workflow state, or billing records in Redis. Those are business source of truth and belong in the application DB accessed through typed SQL. After Redis loss, the system should rebuild from durable state: sessions may disappear, cache may rebuild, and rate-limit windows may restart.

15.2 Production Redis configuration

The Redis URL is a secret and should come from a file:

```
[redis]
url_file = "/run/secrets/rwlang/redis-url"
allow_insecure = false
```

The file may contain a TLS URL such as:

```
rediss://rwlang:SECRET@redis.internal.example/0
```

Keep `allow_insecure = false` as the production baseline. Network reachability to Redis is not an authorization model by itself: also restrict it with network ACL/firewall, and make credentials readable only by the RWLang service user.

The runtime uses separate namespaces for separate purposes. Public cache, for example, lives under an `rw-cache` namespace; the application does not choose arbitrary key prefixes.

15.3 Shared sessions across server instances

An in-memory session store is useful for development, but would be a broken production model with multiple application instances. Consider two servers:

```

      +--> RWLang A --+
Browser -> proxy |           |--> Redis
      +--> RWLang B --+
```

If the session lived only in A's memory, a later request sent to B would lose the user's authentication. Sticky sessions can partially hide this but are fragile for failover, rollout, and load balancing. The correct solution is a shared session backend.

In production, Redis shares auth state including:

- sessions;
- TOTP replay protection;
- login rate-limit state.

The same browser therefore does not need to hit the same server instance every time.

The important local-auth exception

Sessions can be shared, but the V1 local-auth identity store is SQLite and targets single-node identity. For multiple application instances, the current shared identity backend is LDAP/LDAPS. Do not confuse these two layers:

Redis session tells the server which trusted principal and auth state an already-authenticated request belongs to;

identity backend verifies credentials and account/role state during login.

Multiple instances still need a common identity source even with shared Redis sessions; in V1, LDAP is the production path for that.

15.4 Public page cache

Public cache avoids repeating the same query and HTML render for every request to a public, user-independent GET page.

Declaration:

```
route frontPage GET "/"
  cache public ttl 60
  => frontPage;
```

The developer declares only the caching intent and route TTL. The runtime constructs a canonical hashed key from, among other things, the route name, route generation, path, sorted typed query input, and representation.

Application code therefore cannot do this:

```
cache.set("homepage:" + userInput, rendered)
```

There is no general cache API. This prevents key-naming drift, accidental PII in keys, and a proliferation of incompatible invalidation strategies.

15.5 What may go into public cache?

The name `cache public` is literal. Only GET routes whose responses do not depend on user session or request-specific security state should be cached.

The compiler forbids dependencies such as:

```
csrfToken
authPrincipal
authMfaVerified
```

This is critical because a successful cached response may also be public-cacheable at the HTTP layer:

```
Cache-Control: public, max-age=<ttl>
```

Account pages, admin interfaces, personalized navigation, and pages consuming flash messages therefore do not belong in public cache.

15.6 Operator-controlled cache boundaries

A route cannot request an arbitrarily large TTL. Trusted server configuration sets hard ceilings:

```
[cache]
max_ttl_secs = 3600
max_entries = 10000
max_bytes = 67108864
allow_memory = false
```

`max_ttl_secs` caps any route-requested TTL. Entry and byte limits bound the in-memory development fallback; memory cache is not a production scaling strategy.

If the application uses public cache but Redis is not configured, the server does not start under the production default. Memory cache can be enabled explicitly for development, but do not let that escape into production unnoticed.

15.7 Cache invalidation: generations, not key search

A state-changing route can declare which cached routes become invalid:

```
route publish POST "/publish"
  invalidate cache frontPage article
  => publish;
```

Invalidation does not run Redis `SCAN` and does not delete a discovered key list. The runtime increments a route generation. New requests then build keys from the new generation, making old entries unreachable to new traffic.

This has two important consequences:

- invalidation is effectively $O(1)$, independent of how many old cache keys exist;
- with shared Redis, every instance observes the same new generation, so invalidation is cluster-safe.

Old entries may remain physically in Redis until TTL expiry, but logically they are no longer part of the active cache namespace.

15.8 Stampede protection and its limits

After a cache miss, many identical requests may arrive at once. On a single instance, RWLang uses single-flight-like behavior so only one request rebuilds a given cache key; others wait and then read cache again.

The boundary matters: V1 has no distributed single-flight lease. Two different RWLang instances may rebuild the same expired key concurrently. Therefore a cacheable page query must still be reasonably cheap and well indexed. Cache is not an excuse for an uncontrolled expensive database operation.

15.9 Redis failure during cached requests

A public-cache backend failure must not silently switch the application to a different consistency model. When the cache backend is unavailable, RWLang may return a service failure such as `cache_unavailable` for cached routes instead of implicitly allowing every request to fall through to the database without bound.

This fail-closed production behavior is useful: a Redis outage should not silently trigger a DB thundering herd. Readiness can return 503 for DB/Redis dependency failures so the load balancer can remove the instance from traffic.

15.10 Route rate limiting

A route does not embed numeric limits in source; it requests a trusted policy by name:

```
route api GET "/api/status"
  rate publicApi
```

```
=> api;
```

Concrete numbers live in operator configuration:

```
[policy.publicApi]
limit = 120
window_secs = 60
scope = "ip_route"
```

This is a deliberate separation. The application can say that an endpoint needs the `publicApi` policy, but cannot arbitrarily override production resource and abuse policy.

15.11 Rate-limit scopes

V1 scopes are:

Scope	Bucket subject
<code>ip</code>	effective client IP
<code>route</code>	one shared route bucket for all clients
<code>ip_route</code>	effective client IP + route
<code>user</code>	authenticated canonical principal
<code>user_route</code>	canonical principal + route

`user` and `user_route` on a public route are startup errors because there is no trusted authenticated principal from which to derive the bucket.

For IP scopes, RWLang does not blindly trust `X-Forwarded-For`. It first determines the effective client IP using trusted-proxy policy. Correct proxy-CIDR configuration is therefore also a rate-limit security concern behind Apache/Nginx.

15.12 Fixed-window model across instances

The route limiter uses fixed-window buckets. In Redis, all instances atomically increment the same bucket, so the limit is global rather than per instance.

For example with two servers:

```
limit = 120 / 60 sec

not: 120 requests on A + 120 requests on B
but: 120 requests in the shared bucket
```

The subject appears in the Redis key only as a stable hash, so raw usernames or IP addresses are not embedded directly as keys.

When the limit is exceeded:

```
HTTP/1.1 429 Too Many Requests
Retry-After: 60
```

If the Redis limiter fails, policy fails closed:

```
503 rate_limiter_unavailable
```

This is safer than automatically opening the endpoint without limits during a Redis failure.

15.13 Memory backends are development exceptions

The trusted production baseline is:

```
[rate_limit]
policies_file = "/usr/local/etc/rwlang/rate-limits.toml"
allow_memory = false

[cache]
allow_memory = false
```

Memory rate limiting and memory cache are explicit development escape hatches. With multiple processes or hosts, each instance would see separate memory and neither limits nor cache invalidation would have the same global semantics as Redis.

If you test locally with memory backends, always validate the production configuration before deployment:

```
rwlang-server --config /usr/local/etc/rwlang/server.toml --check-config
```

15.14 A typical multi-instance topology

```

      +---> RWLang A  --+---> application DB
Internet -> proxy --|      |
      +---> RWLang B  --+---> Redis
              |
              +-----> shared AppFs (*)
```

The asterisk matters: if the application uses uploaded files/AppFs as business state, multi-instance infrastructure must make that data root consistently available to every instance. The V1 AppFs API provides confinement and safe file operations; infrastructure still decides how the data root is shared correctly.

Component roles are:

- **reverse proxy/load balancer:** TLS, routing, instance selection;
- **RWLang instance:** near-stateless application process with bounded local runtime state;
- **application DB:** durable structured business source of truth;
- **AppFs:** durable file/media business state when used;
- **Redis:** rebuildable shared session/cache/rate-limit/replay infrastructure.

15.15 Rolling restart and cache generation

A new release rollout does not require manually deleting old cache keys. Route generation is part of the cache key, so a new release/compiler generation creates a new namespace. Explicit

business invalidation still comes from state-changing route declarations.

During a rolling restart, the proxy should send traffic only to ready instances. Use SIGTERM graceful drain. The orchestrator's termination grace period should exceed the server's configured shutdown grace period.

Sticky sessions are not required when the shared session backend is configured correctly.

15.16 What should you monitor?

Cache and rate limiting are not invisible optimizations. Minimum production monitoring includes:

- `rw_cache_hits_total` and `rw_cache_misses_total`;
- 429 responses and `rw_rate_limit_denials_total`;
- `rate_limiter_unavailable` events;
- readiness 503 events on Redis/DB dependency failures;
- Redis latency, connection, and memory state in infrastructure monitoring;
- application DB latency during cache-miss periods.

Do not use raw IP, username, session ID, tenant, request ID, or full URL as metric labels. The bounded-cardinality rule from the observability chapter applies here too: metric labels must not grow without bound per user or request.

15.17 Redis in recovery

Redis is runtime state in this architecture: sessions, cache, rate-limit, and replay-protection state may live there, but it is not business source of truth. Recovery therefore focuses not on restoring cache, but on ensuring that the system rebuilds this state safely and predictably after Redis loss.

The full backup/restore order and which state must actually be backed up are covered in Chapter 19.

15.18 Practical example: cached news front page with publishing

The public front page is user-independent and can be cached:

```
page fn frontPage(ctx: PageContext, db: Db)
  -> Result<Html, PageError> {
    let articles = latestArticles(db)?;
    return Ok(html {
      <main>
        @for article in articles {
          <a @href(article, article.slug)>
            {{ article.title }}
          </a>
        }
      </main>
    });
  }

route frontPage GET "/"
```

```
cache public ttl 60
=> frontPage;
```

Publishing is an authenticated state change and is not cacheable, but it invalidates the front page:

```
route publish POST "/admin/publish/:id<Int>"
  auth role Publisher
  invalidate cache frontPage
=> publish;
```

The mental model remains simple:

1. the database is the source of truth for articles;
2. the public HTML cache is only a fast copy;
3. after the publish transaction, the new generation causes the next GET to build a fresh page;
4. any RWLang instance may perform that rebuild;
5. the resulting response goes into shared Redis cache.

15.19 Checklist

Before a multi-instance production deployment, verify:

- Redis URL comes from a secret file and TLS/security policy is appropriate;
- shared sessions eliminate the need for sticky sessions;
- local-auth SQLite is not treated as a shared multi-instance identity store;
- public cache exists only on truly user-independent GET routes;
- every relevant state-changing route declares explicit cache invalidation;
- cache TTL does not exceed the operator maximum;
- production uses `allow_memory = false` for cache and rate limiter;
- rate-limit scope derives from trusted identity/effective client IP;
- 503/readiness behavior has been tested for Redis outage;
- cache hit/miss and rate-limit denial metrics are monitored;
- Redis state is not treated as a required business backup component.

Chapter 16

Security hardening: connecting the trust boundaries

Earlier chapters taught authentication, object-level authorization, CSRF and CORS protection, typed SQL, AppFs, shared runtime state, and outbound egress policy separately. The detailed production configuration key reference comes later; here we summarize the security boundaries that configuration must enforce. In production handoff, these are not independent features but one security model.

Security hardening therefore does not introduce new syntax. Its purpose is to verify that every external value and every sensitive operation crosses a clear trust boundary and that no shortcut bypasses compiler, runtime, or server policy.

16.1 The five most important trust boundaries

A typical RWLang application should treat at least these boundaries separately:

Boundary	Untrusted side	Authority / validation
HTTP input	path, query, header, form, JSON	typed route/input, parser limits, CSRF/CORS
Identity	cookie, login fields, client role	trusted auth backend, session, MFA
Data	client value, record identifier	typed query, bind, transaction, authorization
Filesystem	filename, MIME, upload bytes	AppFs confinement, server-owned destination, limits
External network	URL, DNS, peer, response	named egress target, CIDR/port/TLS/timeout policy

The most important mental rule is: **data does not become trusted merely because it already passed through an earlier layer**. An authenticated user's form data is still user input; a valid record identifier does not imply authorization; a host resolved by DNS does not prove that the actual peer IP is allowed.

16.2 HTTP edge: Host, proxy, and parser

With direct HTTPS, the RWLang server itself is the public HTTP trust boundary. Behind Apache or Nginx there are two layers: the public proxy and the RWLang listener.

In production, forwarding headers may be accepted only from explicitly trusted proxy networks. A directly client-supplied `X-Forwarded-For`, `X-Forwarded-Proto`, or similar header must not become authority.

The public host should also come from trusted configuration. Do not construct redirect targets from an arbitrary client `Host` header.

The RWLang HTTP parser is intentionally strict and rejects ambiguous request forms. Examples include duplicate critical headers, obs-fold, and bodies on GET/HEAD. Request `Transfer-Encoding` and unknown connection tokens are also rejected. The developer rule is simple: **do not attempt to loosen edge-parser invariants from application code**.

Fail-closed decisions and security audit

A trust-boundary denial should not exist only as a client response. The server uses the same server-generated request ID for access logs and, where a security decision occurred, a structured `security_audit` event. Important classifications are:

Boundary	HTTP	Audit category/outcome
forwarding metadata	400	proxy/invalid_forwarding
Host pinning	421	host/mismatch
HTTPS policy	426	transport/https_required
missing auth on API	401	auth/unauthorized
CSRF	403	csrf/denied
CORS	403	cors/denied
Origin/Fetch Metadata	403	origin/denied
route/object policy	403	policy/forbidden
rate limit	429	rate_limit/denied

Do not put tokens, cookies, credentials, full URLs, or request bodies into the security audit. Category/outcome says *which trust boundary decided*; use the request ID to correlate access/server logs for detailed request forensics.

16.3 TLS and secret handling

Private keys, database credentials, Redis URLs, LDAP credentials, and other secrets must not live in:

- source code;
- the Git repository;
- command-line arguments;
- plaintext TOML secret values;
- access/server/audit logs.

The production pattern is a `*_file` reference:

```
[tls]
cert_file = "/run/secrets/tls/fullchain.pem"
```

```
key_file = "/run/secrets/tls/privkey.pem"

[database]
url_file = "/run/secrets/rwlang/database-url"
```

Configuration states *where* the secret is; the secret value itself remains behind a separate permission boundary. Secret files should be readable by the service user and as inaccessible as practical to everyone else.

16.4 Identity: authentication is not authorization

Successful login proves only that a trusted principal exists. It does not prove that the principal may access a particular article, invoice, or admin operation.

The correct layers are:

1. authentication: who is the user;
2. route authorization: may the user enter this handler;
3. object authorization: may the user access this particular record;
4. business invariant: is this operation allowed in the current state.

Client-supplied role, owner, tenant, or price is never authority. Roles come from the trusted auth backend, owner comes from a trusted record, and business decisions come from server-side state.

A typical object-level mutation order is:

```
let article = Article.byId(db, id)?;
authorize article owner authorUsername or role Publisher;

transaction db {
  Article.publishChanged(tx, id, version)?;
  audit Article id action publish;
    from article.status
    to ArticleStatus.Published
}
```

The authorization guard does not replace optimistic locking, and optimistic locking does not replace authorization.

16.5 Session, CSRF, and CORS together

A session cookie is not CSRF protection by itself. State-changing form or JSON requests require session-bound CSRF protection; JSON POST may carry the token in a dedicated header.

Treat browser-side protection as defense in depth:

```
session-bound CSRF token
+ same-origin Origin/Referer check
+ Sec-Fetch-Site cross-site deny
```

Open CORS only to an explicit exact-origin allowlist. Credentialed CORS must never use a wildcard. CORS is not authentication or authorization; it only controls which browser origins

can read a response.

16.6 SQL and business mutation

Compiler-owned SQL plus typed bind parameters are among V1's strongest security foundations. User input must not enter SQL string concatenation, dynamic column names, or dynamic table names.

Mutations run inside transactions. A transaction is not only a consistency tool: it also matters for security because multiple writes performed after authorization should succeed or fail together, and a business audit should belong to the same state transition.

The runtime DB credential should follow least privilege. Migration credentials are a separate operator capability; the web server should not run continuously with schema-modification privileges.

16.7 Uploads, media, and AppFs

Every byte of an uploaded file is untrusted. Client filename and MIME type are metadata only; do not use them as storage keys, paths, or file-type authority.

The production baseline is:

- server-owned destination directory;
- byte limit;
- for images, pixel/decode limit and byte-based format validation;
- AppFs confinement;
- no symlink/magic-link escape;
- runtime data separated from release/static files;
- only the required `r/w/c` capability.

Do not use the same directory for public static root and user uploads merely because both need to be served over HTTP. Their lifecycle, trust, and cache policies are different.

16.8 Redis and shared runtime state

Redis is runtime-owned infrastructure, not business source of truth. Sessions, TOTP replay state, rate limits, and public cache may live there; business records should not move there simply because Redis is “faster.”

With multiple instances, missing shared session/rate-limit state can produce different security decisions depending on which node receives a request. Shared-state policy is therefore part of the production topology.

When Redis fails, security-relevant functions should not become automatically permissive. Fail closed for rate limits, replay protection, and session validation.

16.9 Outbound egress and SSRF

The external network is a separate trust boundary. V1 has no unrestricted application-side `HttpClient`; the trusted integration adapter uses named targets.

Target policy constrains host, address range, port, TLS, timeouts, and response size together. After DNS resolution, every A/AAAA address is checked, then the actual peer address is validated again after connection.

Forbidden pattern:

```
user input URL -> generic server-side fetch
```

Recommended pattern:

```
user/business choice
    |
    v
named integration target
    |
    v
trusted host/CIDR/port/TLS policy
```

The user may choose a business option, but not an arbitrary network destination.

In V1, egress is a capability of the trusted Rust integration layer, not a general `.rw` HTTP API. Therefore a fail-closed `IntegrationError::Policy` denial is not automatically a webserver `security_audit` event: the adapter/operator integration must log the target name and minimal outcome if forensic evidence is required. Never log secrets, credentials, arbitrary full URLs, or response bodies there either.

16.10 Resource limits are security controls too

Security is not only confidentiality and authorization. A web application is also vulnerable if one request can consume unbounded memory, CPU, DB time, or upload bandwidth.

The defensive layers work together:

- request/parser hard limits;
- upload and response size limits;
- query/result limits and timeouts;
- rate limits;
- runtime resource profiles;
- process/systemd/cgroup hard limits.

Do not request a larger resource profile “just in case.” Larger budgets should be a production policy decision and an auditable exception.

16.11 Log hygiene and audit integrity

Logs are themselves a data-exfiltration surface. Do not log credentials, cookies, CSRF/TOTP secrets, or entire form/JSON bodies.

Security events need minimal structured fields: request ID, route, actor when appropriate, decision type, and status. Business audit should be transactional and tied to the application state change.

A request ID is a correlation identifier, not authority. The server should generate its own; do not trust a client-supplied identifier.

16.12 Process and filesystem hardening

Run the production service as an unprivileged user. Release directories and configuration should be runtime read-only; only genuinely writable areas should be writable.

Recommended separation:

```
/usr/local/bin/rwlang-server
/usr/local/bin/rwlang-cli
/usr/local/etc/rwlang/      # trusted config, read-only
/srv/rwlang/releases/      # immutable releases
/srv/rwlang/data/          # declared persistent app data
/run/secrets/rwlang/       # secret files
/var/log/rwlang/           # logs
```

Systemd hardening, cgroup limits, and RWLang resource policy are not alternatives; they are separate defensive layers.

16.13 Dependency and build supply chain

The deployed RWLang server is a binary, so the dependency graph is release state too. A committed `Cargo.lock` and `-locked` builds make the dependency surface more reproducible and reviewable.

After an intentional dependency update, do more than compile:

```
./tools/refresh-lock.sh
./verify.sh
./tools/dependency-audit.sh
```

RustSec audit provides evidence about known advisories; it does not replace code review or upstream changelog review. Major/minor migrations of parser, TLS, crypto, auth, or database dependencies should not be hidden inside unrelated feature diffs.

16.14 Production security gate

Minimum checks before production handoff:

1. HTTPS and public-host policy are verified.
2. Reverse-proxy trust is open only to explicit CIDRs.
3. Secrets enter the process only through file references.
4. Runtime DB credentials are least privilege; migration credentials are separate.
5. Session/auth backend matches the single-node or multi-instance topology.
6. Every state-changing browser operation has CSRF protection.
7. CORS uses exact allowlists; no credentialed wildcard exists.
8. Object authorization uses trusted principal/role and stable owner authority.
9. SQL is compiler-owned statement + typed binds only.
10. Upload uses a server-owned destination and restricted AppFs capability.
11. No business source of truth would disappear if Redis were empty after restore.
12. Outbound integrations reach the network only through named egress policy.
13. Request, upload, DB, runtime, and process resource limits are configured.
14. Logs contain no secrets or full sensitive request payloads.
15. Security-negative tests, config check, and deployment smoke tests are green.

16.15 A code-review thinking pattern

Five questions catch many security problems early in every new feature:

1. **What comes from outside?** – Which field, header, file, record identifier, or external response is untrusted?
2. **Where does it become typed data?** – Route parser, form/JSON schema, model, or trusted adapter?
3. **Who is the authority?** – Session backend, database record, trusted config, or egress policy?
4. **What happens on failure?** – Fail closed, rollback, cleanup, 4xx/5xx, audit?
5. **What limits the resource?** – Size, time, count, rate, or process budget?

If these questions do not have short, unambiguous answers, the feature's security model is probably not finished.

Chapter 17

Logging, audit, metrics, and troubleshooting

A production web application is not operable merely because it “prints something to the console” when something goes wrong. Developers and operators need to correlate the same events: which HTTP request ran, which route it reached, what status it returned, whether a security denial occurred, and whether a durable business audit record was created.

RWLang provides four separate surfaces for this:

- structured **server log** for technical and lifecycle events;
- structured **access log** for HTTP requests;
- structured **audit log** for security and user activity;
- a separate-listener **Prometheus metrics** endpoint.

Applications may additionally store a transactional **business audit trail** in their database. The audit log and business audit are not the same thing.

17.1 The role of the three log classes

Recommended production configuration:

```
[observability]
metrics_listen = "127.0.0.1:9090"
allow_public_metrics = false
access_log = true

[logging]
server_file = "/var/log/rwlang/server.log"
access_file = "/var/log/rwlang/access.log"
audit_file = "/var/log/rwlang/audit.log"
stderr = true
```

Each file serves a different purpose.

server.log Startup, shutdown, dependency failures, technical and internal runtime events.

Check this first if the server does not start, readiness fails, or a background component reports trouble.

access.log One record per HTTP request. This is the traffic and latency trace: method, route, status, duration, client IP, input/output bytes, request ID.

audit.log Security and user activity: login/logout, auth or MFA failure, policy denial, rate-limit denial, resource-profile startup audit, and authenticated state-changing actions.

`access_log = false` disables only the access log. Security/user audit remains active.

17.2 Request ID: following one request end to end

Every successfully parsed application, static, or health request receives a server-generated identifier:

```
X-Request-Id: rw-...
```

An access-log entry may look like:

```
{"event":"http_request","request_id":"rw-...",  
  "method":"GET","route":"articleShow","status":200,  
  "duration_ms":7,"client_ip":"203.0.113.5",  
  "bytes_in":0,"bytes_out":812}
```

A client-provided `X-Request-Id` is not authority. RWLang generates its own request ID so an attacker cannot force the server to use an attacker-chosen correlation identifier.

The basic troubleshooting workflow is therefore:

```
# 1. find the request ID in the access log  
grep 'rw-...' /var/log/rwlang/access.log  
  
# 2. search for the same ID in the technical log  
grep 'rw-...' /var/log/rwlang/server.log  
  
# 3. for security cases, search the audit log too  
grep 'rw-...' /var/log/rwlang/audit.log
```

Together the three logs provide the complete technical picture.

17.3 What may and may not be logged?

A log is not a debug-data dumping ground. In production, sensitive request contents especially must not be logged automatically.

RWLang logs must not contain:

- Authorization headers;
- Cookie headers or session IDs;
- CSRF tokens;
- TOTP secrets or recovery codes;
- DB, Redis, or LDAP credentials;
- secret-file contents;
- complete request bodies or form payloads.

This is a privacy and security invariant. If a business event requires durable information, do not dump the request body; use explicit narrow business-audit fields.

17.4 Security audit and HTTP denials

For trust-boundary denials, RWLang emits a dedicated `security_audit` event with the same request ID. This includes invalid/untrusted forwarding metadata returning 400, security decisions returning 401/403/421/426/429, and more precise categories for CSRF/CORS/origin denials. This lets operators distinguish, for example:

- `proxy/invalid_forwarding`: malformed or untrusted forwarding metadata;
- `auth/unauthorized`: missing authentication on an API route;
- `csrf/denied`, `cors/denied`, `origin/denied`: browser trust-boundary denial;
- `policy/forbidden`: route- or object-authorization denial;
- `host/mismatch`, `transport/https_required`: Host/TLS boundary failure;
- `rate_limit/denied`: limiter denial.

A key developer distinction is that 403 does not necessarily mean a program bug. It may be the correct result of security policy. Always inspect the audit event and route policy too.

17.5 Business audit: durable business history

Access and security logs are operational event streams. Publishing an article, approving an invoice, or changing an order state is business state change and is better recorded transactionally in the application database.

Example:

```
enum ArticleStatus {
    Review
    Published
}

transaction db {
    Article.publishChanged(tx, id, version)?;
    audit Article id action publish;
        from article.status
        to ArticleStatus.Published
}
```

The audit runs in the same DB transaction as the mutation. If the audit record cannot be written, the business change rolls back too. This is stronger than writing a separate log line.

RWLang writes to the `_rw_business_audit` runtime table fields such as:

- runtime-generated event ID and UTC timestamp;
- HTTP request ID;
- canonical authenticated actor;
- source action;
- object type and object identifier;
- static business-action name;
- optional previous and new domain state.

A business audit statement does not replace authorization:

```
let invoice = Invoice.byId(db, id)?;
authorize invoice owner ownerUsername or role Accountant;

transaction db {
  Invoice.approve(tx, id, version)?;
  audit Invoice id action approve;
    from invoice.status
    to InvoiceStatus.Approved
}
```

The correct order is: **identity -> authorization -> mutation -> audit in the same transaction.**

17.6 Prometheus metrics on a separate management socket

Metrics are not an application route. They run on a separate listener:

```
[observability]
metrics_listen = "127.0.0.1:9090"
allow_public_metrics = false
```

Query from the same host:

```
curl http://127.0.0.1:9090/metrics
```

The listener serves only GET/HEAD `/metrics`. It does not run application routes, sessions, or auth handlers.

Typical metrics include:

```
rw_requests_total
rw_responses_5xx_total
rw_auth_failures_total
rw_csrf_failures_total
rw_policy_denials_total
rw_request_timeouts_total
rw_runtime_budget_exceeded_total
rw_rate_limit_denials_total
rw_readiness_failures_total
rw_request_bytes_in_total
rw_response_bytes_out_total
rw_active_connections
rw_log_fallback_total
rw_request_duration_ms_bucket
rw_route_requests_total{route="..."}
rw_route_5xx_total{route="..."}
rw_route_duration_ms_sum{route="..."}
```

`rw_policy_denials_total` tracks security-policy denials: in addition to 403/421/426 decisions, a 400 explicitly classified as forwarding-metadata failure counts too. A normal malformed-request 400 does not.

The most important rule is **bounded cardinality**. The `route` label is a compiler-owned route name or fixed reserved name. Raw URLs, path parameters, IP addresses, usernames, tenants, session IDs, and request IDs must never become metric labels. Otherwise an attacker could create an unbounded number of time series.

17.7 Who scrapes metrics behind Apache or Nginx?

The public web proxy and the metrics listener are separate trust boundaries. A typical layout is:

```
Internet
  |
  v
Apache/Nginx :443
  |
  v
127.0.0.1:8080  RWLang server

Prometheus agent
  |
  v
127.0.0.1:9090  RWLang metrics
```

Do not automatically proxy the metrics port out through the public domain. If a remote Prometheus server must reach it, use a private management network, ACL, or separate authenticated monitoring proxy.

17.8 Health: liveness and readiness are different

Built-in server endpoints are:

```
GET  /health/live
HEAD /health/live

GET  /health/ready
HEAD /health/ready
```

Liveness only proves that the server event loop is alive. It does not call DB, Redis, or LDAP.

Readiness reports whether configured runtime dependencies required for service are reachable. DB or Redis failure/timeout may produce **503 Service Unavailable**.

Therefore in a load balancer:

- do not remove an instance based on **liveness** because of a brief DB outage;
- but an instance may be removed temporarily based on **readiness**.

A health endpoint is not a business monitoring API; it deliberately exposes minimal status.

17.9 Logrotate and SIGHUP

RWLang writes file logs in append mode. Let the operating system handle log rotation in production. The repository includes `examples/logrotate/rwlang`.

Simplified example:

```
/var/log/rwlang/server.log /var/log/rwlang/access.log {
    daily
    rotate 14
    compress
    delaycompress
    missingok
    notifempty
    create 0640 rwlang rwlang
    sharedscripts
    postrotate
        systemctl kill -s HUP --kill-who=main rwlang.service || true
    endscript
}
```

The meaning of SIGHUP is deliberately bounded. In behind-proxy mode it reopens log files and transactionally rebuilds domain/application hosting state; it is not a general process-level `server.toml` reconfiguration mechanism. Application source changes normally use the automatic source-reload supervisor instead.

For configuration changes:

```
/usr/local/bin/rwlang-server \
    --config /usr/local/etc/rwlang/server.toml \
    --check-config

systemctl restart rwlang.service
```

This is more auditable and predictable than a partially successful hot reload.

17.10 What if the log file is not writable?

File logging uses a bounded writer queue. If the queue fills, a file sink fails, or the writer is unavailable, RWLang falls back to stderr and increments:

```
rw_log_fallback_total
```

Treat increases as alertable conditions. Possible causes include:

- permission or filesystem failure;
- full disk;
- broken logrotate/reopen;
- sustained log-processing pressure.

The parent directory of a log path must exist. On Unix, the runtime opens log files with mode 0640 and without following symlinks; the operator must still configure directory permissions correctly.

17.11 Practical production troubleshooting workflow

When investigating 5xx responses or “the page is slow,” do not guess. Use a reproducible order.

1. **Liveness:** is the process itself running?
2. **Readiness:** are DB/Redis reachable?
3. **Access log:** which route, status, duration, and request ID are involved?
4. **Server log:** does the same request ID show a dependency/runtime error?
5. **Audit log:** did security policy or rate limiting reject the request?
6. **Metrics:** is this isolated or a trend in 5xx/latency?
7. **Reverse proxy:** did the failure occur before, inside, or after Apache/Nginx?

Quick checks:

```
curl -i https://www.example.com/health/live
curl -i https://www.example.com/health/ready
# For proxy deployments, also test the internal upstream:
curl -i http://127.0.0.1:8080/health/ready
curl -s http://127.0.0.1:9090/metrics | grep rw_responses_5xx_total
journalctl -u rqlang.service --since "10 min ago"
```

If the loopback RWLang endpoint is healthy but the public domain is not, inspect Apache/Nginx, TLS, or forwarded-header policy next. If loopback readiness is also 503, investigate backend dependencies first.

17.12 Minimum monitoring baseline

Even a small production deployment should alert on at least:

- readiness persistently not ready;
- sudden growth in 5xx rate/count;
- meaningful request-latency degradation;
- unusual spikes in auth/policy/rate-limit denials;
- increases in `rw_runtime_budget_exceeded_total`;
- increases in `rw_log_fallback_total`;
- process restart loops or unexpected shutdown.

Thresholds should not be hardcoded by the language. Operators set them according to traffic, SLOs, and business criticality.

17.13 Retention, SIEM, and audit protection

Retention is an organizational and legal policy. RWLang does not impose one universal retention period.

Local `audit.log` is an important operational trace, but by itself it is not a strong tamper-evident archive because a host administrator or actor with sufficient filesystem rights can modify it. For higher assurance, forward it to a centralized access-controlled or append-only log/SIEM system.

The business-audit database table is likewise not protected against direct modification by a DB administrator. Together the two layers provide a strong application trace, but compliance environments still benefit from an independent external log archive.

17.14 Checklist

Before production startup, verify:

- separate server/access/audit log paths exist;
- log-directory permissions are restrictive;
- request bodies and secrets are not logged;
- the metrics listener is not accidentally public;
- Prometheus labels have bounded cardinality;
- liveness and readiness are treated separately;
- logrotate triggers SIGHUP, while config changes trigger restart;
- `rw_log_fallback_total` is monitored;
- business workflows write business audit in the same DB transaction;
- audit-log forwarding and retention have a documented decision.

Chapter 18

Production deployment and service lifecycle

This chapter does not repeat the configuration reference or observability details. From a production perspective, it summarizes how a verified release becomes a running service, how release switching works, and how the process lifecycle remains controlled. Complete backup, restore, and disaster recovery are covered separately in the next chapter.

18.1 Release path

Recommended order:

```
./verify.sh
-> cargo build --locked --release
-> immutable release directory
-> rwlang-server --config ... --check-config
-> rwlang-cli migrate verify
-> approved rwlang-cli migrate apply
-> rwlang-cli migrate verify
-> atomic current symlink switch
-> controlled restart
-> /health/live
-> /health/ready
-> log / metric / audit review
```

The built `rwlang-server` and `rwlang-cli` binaries, application source, and migrations should all be attributable to the same release identifier. Do not modify files one by one inside a live `/srv/rwlang/current` directory. Prepare a new immutable release directory and switch the symlink atomically.

18.2 systemd service

`rwlang-server` is a long-lived service process. `rwlang-cli` is not a daemon: it starts for a migration, auth administration, or another operator task and exits when the task is complete.

```
[Service]
User=rwlang
Group=rwlang
```

```
ExecStart=/usr/local/bin/rwlang-server \\  
  --config /usr/local/etc/rwlang/server.toml  
Restart=on-failure  
NoNewPrivileges=true  
PrivateTmp=true
```

Run the service under an unprivileged account. Under systemd, let systemd own the process-level cgroup ceilings such as `MemoryMax`, `CPUQuota`, and `TasksMax`; do not simultaneously configure RWLang to write the same cgroup controls. The goal is one clear authority for each resource boundary.

For direct TLS on ports 80/443, grant only the bind-service capability (`CAP_NET_BIND_SERVICE`). Behind a reverse proxy on 127.0.0.1:8080, omit it.

18.3 Controlled restart and readiness

After changing process-level configuration, run `-check-config` first, then perform a controlled restart. In behind-proxy mode, `SIGHUP` reopens logs and transactionally rebuilds domain/application hosting state, but listener, database/Redis/auth connections, cgroup policy, logging sinks, and other process-level settings still require restart.

Application-source deployment has a lighter path. The shared source-reload supervisor watches each loaded application entrypoint and its complete transitive module graph using `mtime + size`. After a configurable debounce, it compiles and validates a candidate runtime and atomically swaps only that domain on success. A broken upload leaves the previous generation active and is logged as a reload rejection. The logical entrypoint path is watched too, so an atomic `current` symlink switch activates a new immutable release. Successful code reload also advances public-cache generations so old rendered content does not survive merely because of its previous TTL.

A traffic director should send requests only to a ready instance. During a rolling deployment, start the new instance, wait until it is ready, and only then remove the old one. Health endpoints and their diagnostics are covered in detail in the observability chapter.

18.4 Migration and rollback are not the same operation

Rolling back an application binary or source release may be simple; rolling back a database schema is not. Prefer expand/contract migrations in production: first introduce a backward-compatible schema, then remove obsolete structures only in a later release after the rollback window has closed.

Do not tie automatic reverse SQL to switching the application symlink back. If a migration transformed or deleted data, the real recovery point may be the verified backup.

18.5 Recovery gate before deployment

Before a destructive or schema-compatibility-sensitive release, require a known recovery point and recent restore evidence. Backup contents, DB/AppFs consistency, RPO/RTO, and the rollback-versus-restore decision tree are covered in the next chapter. Deployment is responsible for ensuring that this gate is satisfied before the release switch.

18.6 Minimum post-deploy checks

1. the new process is running and ready;
2. the public home page or a smoke route responds through the real edge path;
3. there is no new error spike in the server log;
4. the access-log status distribution looks normal;
5. the audit log is writable;
6. DB/Redis dependency metrics have not degraded;
7. the release hash and database migration state are known for a rollback decision.

Chapter 19

Backup, restore, and disaster recovery

The purpose of a backup is not merely to keep a copy somewhere. It is to make the application **provably restartable from a known recovery point**. An RWLang system may also have more than one durable state store: the application database, the local-auth identity store, and AppFs may together form business state.

The right question is therefore not “do we back up the database?” but **what is the complete recovery state, under what consistency boundary is it captured, and do we regularly prove that it can be restored?**

19.1 Start with a state inventory

Before production, list everything that belongs to application state. Typically:

- the application database;
- the local-auth SQLite database, if local auth is used;
- the AppFs `data_root`, including uploaded media and other runtime data;
- the immutable release identifier and artifact hash;
- the migration set and checksums;
- the version or hash of trusted configuration and policy files;
- backup time, application version, and database backend.

Secret files belong to a separate secret-management boundary. The recovery runbook must know where the required DB, Redis, TLS, or LDAP credentials can be restored from, but **secret values must never be written into a plaintext release or backup manifest**.

19.2 What is not business source of truth?

In V1, Redis stores session, rate-limit, and public-cache state. Do not treat that state as part of the business backup contract.

An empty Redis is a safe and acceptable disaster-recovery baseline:

- sessions disappear, so users sign in again;
- the public cache rebuilds;
- rate-limit windows restart.

If the same Redis cluster stores another application's business data, backing up that data belongs to that other system's recovery contract.

19.3 RPO and RTO are business decisions before they are technical settings

Record two values before implementing the backup mechanism:

RPO – Recovery Point Objective the maximum acceptable data-loss window;

RTO – Recovery Time Objective the maximum acceptable time to restore service.

If the RPO is 24 hours, a daily backup may be sufficient in principle. If the RPO is measured in minutes, database-level WAL/binlog/PITR or equivalent infrastructure may be required. The RWLang runtime cannot invent organizational RPO/RTO values; operators must translate those requirements into backup technology.

19.4 Safe V1 baseline: a controlled maintenance backup

The database and AppFs may jointly represent business state. A DB row may, for example, refer to an uploaded image. A DB dump and AppFs archive taken at unrelated times are therefore not automatically a consistent recovery point.

The simplest auditable V1 baseline is:

```
traffic drain
-> SIGTERM / graceful shutdown
-> stopped state verified
-> application DB backup
-> local-auth DB backup, if used
-> AppFs data_root snapshot
-> recovery manifest + hashes
-> service start
-> readiness
```

This introduces a short write pause but gives a clear consistency boundary. An online backup should be called consistent only when the DB/storage infrastructure can provide snapshots tied to the same recovery point, or when the application data model demonstrably tolerates different snapshot times. The runtime itself does not promise a distributed snapshot.

19.5 Application DB: use the backend's native backup tool

RWLang does not attempt to hide native database backup mechanisms. Operators should use the supported tooling of the selected backend, preferably with a dedicated backup credential.

PostgreSQL

A typical logical backup:

```
pg_dump --format=custom --no-owner --no-acl \\  
--file app.dump "$DATABASE_URL"
```

For an isolated restore test:

```
createdb rwlang_restore_test
pg_restore --no-owner --no-acl \\\
--dbname rwlang_restore_test app.dump
```

Do not put production secrets in shell history. For large databases, physical backup and PITR may be the better choice; their runbooks belong to the PostgreSQL platform.

MariaDB

For a transactional InnoDB schema, a typical logical backup is:

```
mariadb-dump --single-transaction \\\
--routines --triggers app > app.sql
```

Restore into an isolated database:

```
mariadb rwlang_restore_test < app.sql
```

`--single-transaction` does not magically make non-transactional tables consistent. For a mixed schema, keep the controlled-maintenance baseline.

SQLite

Do not call a plain `cp` of a live SQLite database a backup. Use SQLite's backup mechanism or take a filesystem snapshot while the application is stopped.

```
sqlite3 /srv/rwlang/data/app.db \\\
".backup '/backup/app.db'"
```

Always restore into a separate test path. Do not discover whether a backup works by restoring over production.

19.6 The local-auth DB is a security-sensitive backup

When local auth is enabled, the identity store is a separate SQLite boundary. It may contain password hashes, TOTP secrets, and recovery hashes, so treat its backup **as a secret**.

```
sqlite3 /srv/rwlang/auth/users.db \\\
".backup '/backup/local-auth.db'"
```

Store the backup encrypted, with narrow access and documented retention. After restore, verify file and parent-directory ownership and permissions. The identity store cannot be reconstructed from the application database.

19.7 AppFs: the entire data root may be durable state

Do not back up only the media subdirectory that is published over HTTP. The entire declared AppFs `data_root` may belong to the recovery state.

With the service stopped, for example:

```
tar -C /srv/rwlang -cpf data-root.tar data
```

After restore, preserve or restore ownership and permissions. The service account should be able to write only the roots declared writable by configuration; do not weaken AppFs confinement with symlinks.

19.8 Every recovery point needs a manifest

Backup objects alone do not identify which release and migration state they belong to. Create a secret-free manifest:

```
backup_id=2026-09-04T180000Z
release_id=2026-09-04.1
server_sha256=...
app_source_sha256=...
migrations_sha256=...
config_sha256=...
db_backend=postgresql
app_db_backup_sha256=...
local_auth_backup_sha256=...
data_root_backup_sha256=...
```

The manifest should contain hashes and identifiers, not credential values. The included `deploy/release-record.sh` helps record release/source/migration/config state in read-only mode.

19.9 Restore drill: prove the backup

A successful backup job does not prove recoverability. Regularly run a restore drill in an isolated environment.

```
new empty restore DB
-> application DB restore
-> local-auth restore, if used
-> AppFs restore into separate root
-> select exact immutable release
-> restore-test config on isolated host/port
-> rwlang-server --check-config
-> rwlang-cli check main.rw
-> rwlang-cli migrate verify
-> start rwlang-server on isolated listener
-> /health/live + /health/ready
-> application-specific smoke test
-> log and audit verification
```

`deploy/restore-verify.sh` automates part of the non-mutating config/source/migration checks. It does not start a production listener and does not run `migrate apply`.

Record the drill result with date, backup ID, and release ID. “The backup completed” is not a release gate; you need recent restore evidence appropriate to the organization’s RPO/RTO requirement.

19.10 Make a recovery decision before an upgrade

A controlled production upgrade should follow an order such as:

```
immutable release artifact + hashes
-> migration verify
-> compatibility decision
-> consistent backup + recent restore evidence
-> migration apply
-> migration verify
-> atomic release switch
-> controlled restart
-> live + ready + smoke
-> logs + metrics + audit review
```

Record migration compatibility at release level. The key question is whether the previous application can still run against the new schema.

19.11 Expand/contract keeps the fast rollback window open

If the new schema is backward-compatible, a faulty application release can be switched back to the previous immutable artifact without restoring the database.

Recommended pattern:

1. **expand**: add a nullable/defaulted column, table, or index while keeping the old application working;
2. deploy the new application;
3. perform controlled data backfill if needed;
4. only in a later release **contract**: remove the old column or constraint after the rollback window has closed.

A destructive rename/drop or immediate incompatible shape change can close the app-only rollback option. Mark that explicitly before release.

19.12 Rollback and restore are different operations

Faulty application, backward-compatible schema

```
traffic drain
-> previous immutable release
-> controlled restart
-> live + ready + smoke
```

There is no database restore, so business data created after the faulty deployment is preserved.

Faulty application, but a forward fix is possible

Prefer a new immutable release or a forward migration. Do not tie automatic reverse SQL to switching the release symlink back.

Schema or data is corrupted

This is disaster recovery:

```
stop writes
-> choose and record recovery point
-> restore DB + auth DB + AppFs consistently
-> select matching release and config
-> verify + smoke
-> controlled return to traffic
```

Writes after the selected recovery point may be lost. This is a business incident decision, not a technical footnote: the data-loss window must be explicitly accepted and documented.

19.13 Forbidden shortcuts

- calling a plain `cp` of a live SQLite DB a backup;
- backing up only the application DB when AppFs also contains business state;
- considering backup successful without a restore drill;
- tying an automatic down migration to application rollback;
- testing restore on the production database;
- giving the normal application service the migration credential;
- writing secret values into release or backup manifests;
- making Redis session/cache restoration a prerequisite for disaster recovery.

19.14 Production recovery checklist

A system is recovery-ready only if at least the following are true:

1. the state inventory is documented;
2. RPO and RTO are recorded;
3. backup encryption, retention, and access policy are known;
4. there is an identifiable recent successful restore drill;
5. migration compatibility for the release is documented;
6. the app-only rollback procedure has been exercised;
7. the owner and approval path for destructive restore are known;
8. the effect of losing Redis is accepted;
9. the recovery runbook identifies the source of release/config/secret restoration;
10. live, ready, smoke, log, and audit checks are performed after an incident.

The core principle of recovery is: **a backup is not a file; it is a proven ability to restore.** Without restore drills, RPO and RTO are assumptions.

Part VI

Reference and operator quick guide

Chapter 20

The RWLang CLI and the daily developer/operator workflow

RWLang installs two programs: the long-lived `rwlang-server` process and the short-lived `rwlang-cli` operator tool. Treat them consistently as separate responsibilities. The server serves HTTP and enforces runtime policy; the CLI checks source, manages migrations, and performs local-auth administration.

This chapter introduces no new language features. It shows how daily work becomes a repeatable command flow from editing through production verification.

20.1 Which tool for which task?

Task	Tool	Notes
Check RWLang source	<code>rwlang-cli check</code>	Compiler-level check; does not start an HTTP server.
Migration status/verify/apply	<code>rwlang-cli migrate</code>	DB URL comes from a secret file.
Local-auth store administration	<code>rwlang-cli auth</code>	Operator action on the SQLite identity store.
Server configuration preflight	<code>rwlang-server</code>	<code>-check-config</code> : validates config, app, and local policy/resource files.
Inspect effective configuration	<code>rwlang-server</code>	<code>-print-effective-config</code> ; secret values remain redacted.
Run the web application	<code>rwlang-server</code>	Long-lived process behind a proxy or direct HTTPS.
Workspace release build	<code>cargo build</code>	Build both binaries with <code>-locked -release</code> .
Verify the whole platform	<code>./verify.sh</code>	Part of RWLang repository release evidence, not a command every application must run.

Production examples install the programs here:

```
/usr/local/bin/rwlang-cli
/usr/local/bin/rwlang-server
```

With a normal shell `PATH`, the examples below can use the short program names.

20.2 The CLI is not an alternative server

`rwlang-cli` does not open a listener, serve routes, or maintain session state. It performs one command and exits. This is useful from a security perspective: migrations and user administration do not require an administrative HTTP endpoint.

Conversely, `rwlang-server` does not run migrations automatically at startup. Schema changes remain explicit operator decisions.

20.3 The first command: `rwlang-cli check`

During daily development, make this the first gate after every meaningful source change:

```
rwlang-cli check main.rw
```

On success, the CLI prints a short summary of models, queries, pages, actions, and routes seen by the compiler. On failure, do not start the server yet; fix the compile-time contract first.

`check` catches issues such as:

- unknown types or invalid language syntax;
- query projection and model-shape mismatch;
- invalid typed bind/decode contracts;
- route/policy or resource-profile reference errors that are provable at compile time;
- forbidden raw HTML/SQL-style constructs.

It does not prove that the DB or Redis is reachable over the network and does not replace HTTP integration tests.

20.4 Running from the source tree versus using the installed CLI

When developing the RWLang repository itself, the same program can be run through Cargo:

```
cargo run --locked -p rwlang-cli -- check main.rw
```

This is a developer convenience. In production runbooks, use the immutable installed release binary:

```
/usr/local/bin/rwlang-cli check /srv/rwlang/app/main.rw
```

Do not mix the two forms in one deployment. A production operation should be clearly attributable to one release artifact.

20.5 Migration workflow

The migration command provides three operations:

status shows applied and pending migrations;

verify validates migration history and checksum contracts;

apply acquires the migration lock and applies pending migrations.

The DB URL is not a plaintext command-line secret. Put it in a one-line secret file:

```
/run/secrets/rwlang/db-url
```

Typical verification order:

```
rwlang-cli migrate status \\  
  --dir migrations \\  
  --db-url-file /run/secrets/rwlang/db-url  
  
rwlang-cli migrate verify \\  
  --dir migrations \\  
  --db-url-file /run/secrets/rwlang/db-url
```

For an approved schema change:

```
rwlang-cli migrate apply \\  
  --dir migrations \\  
  --db-url-file /run/secrets/rwlang/db-url \\  
  --lock-timeout-secs 30  
  
rwlang-cli migrate verify \\  
  --dir migrations \\  
  --db-url-file /run/secrets/rwlang/db-url
```

`--lock-timeout-secs` is a positive integer. If the migration lock cannot be acquired within that time, fail the operation; do not allow parallel schema changes.

--allow-insecure-db is not a production convenience switch

The migration CLI follows the server's secure-by-default model. The `--allow-insecure-db` flag is only for deliberate, controlled environments. In production, use proper TLS/database security rather than a permanent bypass.

20.6 Local-auth administration

If the application uses local auth, the CLI also owns identity-store administration. Initialize the store first:

```
rwlang-cli auth init \\  
  --db-url-file /run/secrets/rwlang/local-auth-db-url
```

Create a user:

```
rwlang-cli auth user-add \\  
  --db-url-file /run/secrets/rwlang/local-auth-db-url \\  
  --username alice \\  
  --password-file /run/secrets/rwlang/bootstrap-password \\  
  --role Editor
```

If no `-role` is supplied, a new user receives the `User` role. Supply multiple `-role` options to assign multiple roles.

Change a password or account state:

```
rwlang-cli auth password-set \\
--db-url-file /run/secrets/rwlang/local-auth-db-url \\
--username alice \\
--password-file /run/secrets/rwlang/new-password

rwlang-cli auth disable \\
--db-url-file /run/secrets/rwlang/local-auth-db-url \\
--username alice

rwlang-cli auth enable \\
--db-url-file /run/secrets/rwlang/local-auth-db-url \\
--username alice
```

Replace roles:

```
rwlang-cli auth roles-set \\
--db-url-file /run/secrets/rwlang/local-auth-db-url \\
--username alice \\
--role Editor --role Publisher
```

`roles-set` treats the supplied role list as the new complete state. Do not treat it as an incremental “add one role” operation.

20.7 TOTP administration is especially sensitive

Enroll TOTP:

```
rwlang-cli auth totp-enroll \\
--db-url-file /run/secrets/rwlang/local-auth-db-url \\
--username alice \\
--recovery-count 8
```

The command prints the TOTP secret, the `otpauth` URI, and one-time recovery codes. Do not run it in CI logs, shell transcripts, or shared terminals. The operator process must securely hand off and store the secret and recovery codes immediately.

Disable TOTP:

```
rwlang-cli auth totp-disable \\
--db-url-file /run/secrets/rwlang/local-auth-db-url \\
--username alice
```

20.8 A secret file is also an input contract

The CLI does not read DB-URL and password files without limits. The current secret-URL file contract requires:

- a regular file;

- not a symlink;
- at most 16 KiB;
- exactly one non-empty line.

A password file must also be a regular non-symlink file; it is limited to 4096 bytes, contains one line, and the password itself must be 12–1024 bytes.

Do not try to bypass the contract with process substitution, symlinks, or multiline secret templates. The CLI boundary is deliberately narrow.

20.9 Server preflight follows CLI checks

After source and migration checks, validate the production configuration with the server:

```
rwlang-server \\  
  --config /usr/local/etc/rwlang/server.toml \\  
  --check-config
```

This reads the trusted TOML, compiles the application, checks route-rate/resource policy references and TLS configuration, and validates local preconditions for declared storage/static roots.

To audit precedence or effective policy:

```
rwlang-server \\  
  --config /usr/local/etc/rwlang/server.toml \\  
  --print-effective-config
```

The output is intentionally secret-redacted. It is still operator information, so retain automation logs only when justified.

20.10 Daily development cycle

A good baseline for a small or medium application is:

```
edit .rw source  
-> rwlang-cli check main.rw  
-> rwlang-cli migrate verify  
-> if schema changed: migrate status/apply/verify on dev DB  
-> rwlang-server --config server.dev.toml --check-config  
-> start rwlang-server  
-> HTTP/form/JSON smoke tests  
-> automated application tests
```

The point of the order is to catch cheaper, more deterministic failures first. There is little value in running HTTP smoke tests against source that already fails at compiler level.

20.11 Application release workflow

A minimal production operator flow is:

```
immutable release selected  
-> rwlang-cli check deployed main.rw
```

```
-> rwlang-server --check-config
-> rwlang-cli migrate status
-> rwlang-cli migrate verify
-> backup/recovery gate
-> approved rwlang-cli migrate apply, if pending
-> rwlang-cli migrate verify
-> controlled service restart/switch
-> /health/live
-> /health/ready
-> application smoke test
-> logs + metrics + audit review
```

Migration `apply` is the only schema-mutating step in this sequence; treat it as a separate approval point.

20.12 The RWLang platform release workflow is different

When building the RWLang repository itself, Rust workspace release evidence is part of the process:

```
./verify.sh
cargo build --locked --release \\\
  -p rwlang-server -p rwlang-cli
```

The resulting binaries are:

```
target/release/rwlang-server
target/release/rwlang-cli
```

Install them with:

```
sudo install -m 0755 target/release/rwlang-server \\\
  /usr/local/bin/rwlang-server
sudo install -m 0755 target/release/rwlang-cli \\\
  /usr/local/bin/rwlang-cli
```

Your own RWLang web application's daily workflow does not need to verify the language's entire Rust workspace every time. Your own source, migration, configuration, and integration tests are the relevant gates.

20.13 Automation: separate read-only and mutating steps

In CI/CD, separate safely repeatable checks from state-changing operations.

Read-only/preflight:

```
rwlang-cli check main.rw
rwlang-cli migrate status \\\
  --dir migrations --db-url-file /run/secrets/rwlang/db-url
rwlang-cli migrate verify \\\
  --dir migrations --db-url-file /run/secrets/rwlang/db-url
rwlang-server --config /usr/local/etc/rwlang/server.toml \\\
  --check-config
```

```
rwlang-server --config /usr/local/etc/rwlang/server.toml \\  
  --print-effective-config  
health/readiness GET
```

Mutating/operator-approved:

```
rwlang-cli migrate apply \\  
  --dir migrations --db-url-file /run/secrets/rwlang/db-url  
rwlang-cli auth init \\  
  --db-url-file /run/secrets/rwlang/local-auth-db-url  
# Other auth commands also receive their explicit --db-url-file  
# plus their own --username/--password-file/--role options.
```

Do not normally put auth-administration commands into the automatic application deployment pipeline. They are identity-administration operations with a separate audit and access boundary.

20.14 Common failures

check succeeds but the server does not start The compile-time contract is valid, but configuration, TLS, filesystem, or policy startup validation failed. Run `-check-config` and inspect the server log.

-check-config succeeds but readiness is red Local configuration is valid, but a runtime dependency such as DB or Redis is unreachable or unhealthy.

migrate verify fails Do not reflexively run `apply`. Resolve the history/checksum discrepancy first.

The auth CLI cannot open the DB Verify that the URL points to the local-auth SQLite store and that the store has been initialized.

Secret-file error Do not relax the file contract; provide a regular, non-symlink, one-line secret file.

20.15 One-page operator runbook

If you remember only the minimum order:

```
# source  
rwlang-cli check main.rw  
  
# schema  
rwlang-cli migrate status ...  
rwlang-cli migrate verify ...  
  
# config  
rwlang-server --config /usr/local/etc/rwlang/server.toml \\  
  --check-config  
  
# approved change  
rwlang-cli migrate apply ...  
rwlang-cli migrate verify ...  
  
# service  
systemctl restart rwlang
```

```
# end-to-end health through the public TLS edge
curl -fsS https://www.example.com/health/live
curl -fsS https://www.example.com/health/ready

# proxy topology: optional local upstream probe
curl -fsS http://127.0.0.1:8080/health/ready

# evidence
review server/access/audit log + metrics
```

The exact systemd unit name is deployment-specific; `rwlang` is only an example. The important invariant is the order: **source -> schema -> config -> approved mutation -> service -> health -> evidence**.

Chapter 21

The `server.toml` configuration reference

Earlier chapters used server configuration in a task-oriented way. This chapter gives the complete V1 production configuration from an operator's perspective in one place. The reference is based on the actual parser and `config/server.toml.sample`; if they ever differ, parser behavior is authoritative.

21.1 Loading, precedence, and fail-closed rules

Recommended production startup:

```
rwlang-server \\  
  --config /usr/local/etc/rwlang/server.toml
```

Precedence is:

```
built-in defaults < server.toml < explicit CLI override
```

The TOML file is trusted configuration, but parsing remains fail-closed:

- unknown section or key: startup error;
- duplicate key or table: TOML parse error;
- the config file must be a regular non-symlink file;
- maximum size is 1 MiB;
- filesystem paths coming from TOML must be absolute;
- secrets must not appear in plaintext fields: DB/Redis/LDAP/local-auth credentials are supplied only through `*_file` references.

Validate without starting a listener:

```
rwlang-server \\  
  --config /usr/local/etc/rwlang/server.toml \\  
  --check-config
```

Inspect the secret-free effective view:

```
rwlang-server \\  
  --check-config
```

```
--config /usr/local/etc/rwlang/server.toml \\  
--print-effective-config
```

21.2 How to read the tables

Built-in default means the value that applies with no configuration. The **production sample** is the recommended starting value in `config/server.toml.sample`. They are not always identical: the sample deliberately opts into larger production budgets in several places.

21.3 [server] and [tls]

Key	Built-in default	Meaning / production note
<code>server.app</code>	none	Absolute path to the application's <code>main.rw</code> . Sample: <code>/srv/rwlang/current/main.rw</code> .
<code>server.listen</code>	127.0.0.1:8080	Listener address. Usually loopback behind a reverse proxy; may be public for direct TLS.
<code>server.insecure_dev_cookies</code>	false	Development HTTP only. Keep false in production.
<code>tls.cert_file</code>	none	Absolute path to the TLS certificate chain.
<code>tls.key_file</code>	none	Absolute path to the TLS private key.
<code>tls.handshake_timeout_ms</code>	5000	TLS handshake timeout ceiling.
<code>tls.http_redirect_listen</code>	none	Optional HTTP listener used only for HTTPS redirects.
<code>tls.public_host</code>	none	Canonical/public host for redirect and Host/Origin validation; also required in trusted HTTPS reverse-proxy mode.

Behind a reverse proxy

Behind Apache or Nginx, RWLang typically does not listen on a public interface:

```
[server]  
app = "/srv/rwlang/current/main.rw"  
listen = "127.0.0.1:8080"  
insecure_dev_cookies = false  
  
[tls]  
public_host = "www.example.com"
```

The proxy terminates public TLS. RWLang certificate/key settings are omitted, but `tls.public_host` is still required for trusted Host/Origin pinning. The `web.trusted_proxy_cidrs` list defines which upstreams may supply authoritative forwarding metadata. In this production mode, a non-TLS RWLang listener is allowed only on loopback.

21.4 [database], [redis], and [auth]

Key	Default	Meaning
<code>database.url_file</code>	<code>none</code>	Secret file containing the DB URL. There is no plaintext <code>database.url</code> .
<code>database.allow_insecure</code>	<code>false</code>	Explicitly permits an unencrypted remote DB connection; normally do not use in production.
<code>redis.url_file</code>	<code>none</code>	Redis URL from a secret file.
<code>redis.allow_insecure</code>	<code>false</code>	Explicitly permits insecure Redis.
<code>auth.ldap_url</code>	<code>none</code>	LDAP/LDAPS endpoint.
<code>auth.ldap_search_base</code>	<code>none</code>	LDAP search base.
<code>auth.ldap_username_attribute</code>	<code>uid</code>	LDAP attribute used as the login name.
<code>auth.ldap_service_bind_dn_file</code>	<code>none</code>	Service bind DN from a secret file.
<code>auth.ldap_service_bind_password_file</code>	<code>none</code>	Bind password from a secret file.
<code>auth.totp_secrets_file</code>	<code>none</code>	TOTP secret-store file.
<code>auth.roles_file</code>	<code>none</code>	External role-mapping file.
<code>auth.local_auth_db_url_file</code>	<code>none</code>	Local-auth identity-store URL from a secret file.
<code>auth.require_totp</code>	<code>false</code>	Whether TOTP is required after a successful first factor.
<code>auth.login_max_attempts</code>	<code>8</code>	Maximum login attempts in one rate-limit window.
<code>auth.login_window_secs</code>	<code>300</code>	Login rate-limit window in seconds.

Do not configure LDAP and local auth as two parallel primary identity sources in one deployment. Keep identity authority unambiguous. Redis contains session, cache, and rate-limit runtime state; it is not the user source of truth.

21.5 [web]

Key	Default	Meaning
<code>web.trusted_proxy_cidrs</code>	empty list	Only these source addresses may provide authoritative forwarded client/protocol metadata.
<code>web.allow_missing_origin</code>	<code>false</code>	Policy for unsafe requests without Origin. The production default is fail-closed.
<code>web.cors_origins</code>	empty list	Exact-origin allowlist. Wildcards are not valid for credentialed CORS.

<code>web.cors_allow_credentials</code>	<code>false</code>	Allows cross-origin session cookies/credentials; must be considered together with CSRF policy.
---	--------------------	--

21.6 [storage] and [static_assets]

Key	Built-in default	Meaning
<code>storage.data_root</code>	<code>none</code>	AppFs runtime-data root. Sample: <code>/srv/rwlang/data</code> .
<code>storage.fs_mode</code>	<code>rwc</code>	AppFs capability mode. Grant only the operations that are required.
<code>storage.max_upload_bytes</code>	16 MiB	Maximum bytes for one upload.
<code>storage.max_image_pixels</code>	40,000,000	Decoded image pixel ceiling.
<code>static_assets.root</code>	<code>none</code>	Read-only static root. Sample: <code>/srv/rwlang/current/public</code> .
<code>static_assets.url_prefix</code>	<code>/assets/</code>	URL prefix for the static namespace.
<code>static_assets.max_asset_bytes</code>	8 MiB	Maximum size of one static asset.
<code>static_assets.max_age_secs</code>	300	Cache lifetime for non-fingerprinted assets.
<code>static_assets.immutable_max_age_secs</code>	31536000	Cache lifetime for fingerprinted immutable assets.
<code>static_assets.precompressed</code>	<code>true</code>	Use pre-generated Brotli/gzip variants.

Do not use the same directory for `data_root` and the static root: one is mutable application state, the other is a release-owned read-only artifact.

21.7 [lifecycle], [observability], and [logging]

Key	Default	Meaning
<code>lifecycle.health_live_path</code>	<code>/health/live</code>	Process liveness endpoint.
<code>lifecycle.health_ready_path</code>	<code>/health/ready</code>	Dependency-aware readiness endpoint.
<code>lifecycle.health_dependency_timeout_ms</code>	1000	Timeout for readiness dependency probes.
<code>lifecycle.shutdown_grace_ms</code>	30000	Graceful-shutdown drain window.
<code>observability.metrics_listen</code>	<code>none</code>	Prometheus listener; sample: <code>127.0.0.1:9090</code> .
<code>observability.allow_public_metrics</code>	<code>false</code>	Explicitly allows a public metrics listener.
<code>observability.access_log</code>	<code>true</code>	Emit HTTP access events.
<code>logging.server_file</code>	<code>none</code>	Error/system log file.
<code>logging.access_file</code>	<code>none</code>	Access log file.
<code>logging.audit_file</code>	<code>none</code>	Security/user-activity audit log.

<code>logging.stderr</code>	<code>true</code>	Stderr log output.
-----------------------------	-------------------	--------------------

The operator creates the parent log directory. An existing log target must not be a symlink or non-regular file. In behind-proxy mode `SIGHUP` reopens logs and transactionally rebuilds domain/application hosting state; process-level settings still require restart.

21.8 [rate_limit] and [cache]

Key	Default	Meaning
<code>rate_limit.policies_file</code>	<code>none</code>	Route rate-limit policy file. Sample: <code>/usr/local/etc/rwlang/rate-limits.toml</code> .
<code>rate_limit.allow_memory</code>	<code>false</code>	Explicitly allows in-memory fallback. Keep <code>false</code> for multi-instance production.
<code>cache.max_ttl_secs</code>	<code>3600</code>	Maximum route TTL for public cache.
<code>cache.max_entries</code>	<code>10000</code>	In-memory cache entry ceiling.
<code>cache.max_bytes</code>	<code>64 MiB</code>	Total in-memory cache byte ceiling.
<code>cache.allow_memory</code>	<code>false</code>	Explicitly allows memory cache without Redis.

When multiple `rwlang-server` instances run, shared session/cache/rate-limit state must be carried by Redis. Memory fallback is appropriate only deliberately, usually in a single-instance environment.

21.9 [limits]

Request and runtime limits are defense-in-depth controls. The sample sets several values explicitly so the operator makes an auditable production-budget decision.

Key	Built-in default	Sample / meaning
<code>limits.max_header_bytes</code>	<code>16 KiB</code>	Sample uses the same value.
<code>limits.max_body_bytes</code>	<code>256 KiB</code>	Same in sample; uploads also have a separate storage limit.
<code>limits.max_connections</code>	<code>4096</code>	Concurrent-connection ceiling.
<code>limits.max_requests_per_connection</code>	<code>100</code>	Keep-alive request ceiling.
<code>limits.read_timeout_ms</code>	<code>5000</code>	Request read timeout.
<code>limits.request_timeout_ms</code>	<code>15000</code>	Total request budget.
<code>limits.write_timeout_ms</code>	<code>5000</code>	Response write timeout.
<code>limits.max_header_count</code>	<code>64</code>	Header-count ceiling.
<code>limits.max_form_fields</code>	<code>64</code>	Maximum number of form fields.
<code>limits.max_form_field_bytes</code>	<code>8 KiB</code>	Byte ceiling for one form field.
<code>limits.max_instructions</code>	<code>100000</code>	Sample: <code>5000000</code> . Interpreter budget; increases should be audited.

<code>limits.max_runtime_alloc_bytes</code>	32 MiB	Sample: 256 MiB. Runtime allocation ceiling.
<code>limits.session_ttl_secs</code>	28800	8 hours.
<code>limits.max_sessions</code>	100000	Session ceiling.
<code>limits.max_process_memory_bytes</code>	none	Sample: 1 GiB; process address-space limit.
<code>limits.resource_profiles_file</code>	none	Sample: <code>/usr/local/etc/rwlang/resource-profiles.toml</code> .

Why separate request, runtime, and process limits?

`max_body_bytes` limits network input, `max_runtime_alloc_bytes` limits interpreter allocations for one request, and `max_process_memory_bytes` limits the entire process address space. None replaces the others.

21.10 [cgroup]

Cgroup controls add a Linux production defense-in-depth layer above process-level limits.

Key	Default	Meaning
<code>cgroup.dir</code>	none	Absolute path to an already prepared cgroup directory.
<code>cgroup.memory_max_bytes</code>	none	Cgroup memory ceiling; the systemd sample does not set this.
<code>cgroup.memory_swap_max_bytes</code>	none	Optional swap ceiling for delegated-cgroup deployments.
<code>cgroup.cpu_percent</code>	none	Optional CPU budget percentage for delegated-cgroup deployments.
<code>cgroup.pids_max</code>	none	Optional PID/thread ceiling for delegated-cgroup deployments.

Creating and delegating the cgroup directory is an operator task; the server is not a system-administration tool. In the repository's systemd baseline, the service manager is the cgroup authority (`MemoryMax`, `CPUQuota`, `TasksMax`), so the `[cgroup]` section is intentionally not enabled. Use RWLang-side cgroup writes only with a separately delegated writable cgroup v2 directory. Do not configure both authorities at once.

21.11 Production baseline

A typical minimum configuration behind a reverse proxy:

```
[server]
app = "/srv/rwlang/current/main.rw"
listen = "127.0.0.1:8080"
insecure_dev_cookies = false

[tls]
public_host = "www.example.com"

[database]
```

```
url_file = "/run/secrets/rwlang/database-url"

[redis]
url_file = "/run/secrets/rwlang/redis-url"

[web]
trusted_proxy_cidrs = ["127.0.0.1/32", "::1/128"]
allow_missing_origin = false

[storage]
data_root = "/srv/rwlang/data"

[logging]
server_file = "/var/log/rwlang/server.log"
access_file = "/var/log/rwlang/access.log"
audit_file = "/var/log/rwlang/audit.log"
stderr = true

[limits]
resource_profiles_file = "/usr/local/etc/rwlang/resource-profiles.toml"
```

This is only a baseline. Auth, static assets, metrics, rate limiting, TLS, and cgroup settings must match the deployment topology.

21.12 Automatic source reload

The global `[reload]` section defines `enabled`, `poll_interval_ms`, and `debounce_ms`. Each multi-domain entry may override them in `[domains.reload]`. The default policy enables metadata polling. A single shared supervisor watches the compiler-reported source graph with `mtime + size`, including the configured logical entrypoint path for atomic symlink deployments. Failed candidates leave the old generation active; successful commits atomically replace only the affected domain and invalidate its public-cache generations. Use `-no-source-reload` for a process-wide maintenance override.

21.13 Change workflow

For production configuration changes, this book uses one recommended order:

1. modify the version-controlled configuration template;
2. change secrets in separate files, not in TOML;
3. run `rwlang-server -check-config`;
4. inspect the secret-free `-print-effective-config` summary;
5. perform a controlled restart;
6. verify live/ready endpoints, logs, metrics, and audit output.

`SIGHUP` is a bounded hosting reload in behind-proxy mode, not a general process reconfiguration mechanism. Application source changes normally need neither `SIGHUP` nor restart: the `[reload]` supervisor watches the entrypoint and transitive module graph and transactionally swaps a successfully compiled domain generation.

Chapter 22

Developer reference and where to go next

22.1 Where to look for each topic

This English book is the canonical long-form guide. The repository is in the process of making English the default language for all supporting Markdown documentation as well. Where an English topic document already exists, the table points to `docs/`; where only the Hungarian companion currently exists, it is explicitly marked under `docs/hu/`. Do not infer that a missing English Markdown file means the feature is undocumented in this book.

Topic	Detailed project documentation
Routing, input, validation	<code>docs/hu/04-routing-input.md</code>
Database and SQL	<code>docs/hu/05-adatbazis.md</code>
CRUD and transactions	<code>docs/hu/06-crud.md</code>
Auth and Redis	<code>docs/hu/07-auth-redis.md</code>
Route rate limiting	<code>docs/hu/08-rate-limiting.md</code>
JSON API and CORS	<code>docs/hu/05-json-api-cors.md</code>
Static assets	<code>docs/hu/06-static-assets.md</code>
Public page cache	<code>docs/hu/13-cache.md</code>
HTTPS and proxy security	<code>docs/hu/08-https-security.md</code>
Upload and AppFs	<code>docs/hu/09-upload-appfs.md</code>
Migrations	<code>docs/hu/10-database-migrations.md</code>
Forms	<code>docs/hu/12-forms.md</code>
Components and layouts	<code>docs/hu/14-components-layouts.md</code>
Security checklist	<code>docs/hu/15-security-checklist.md</code>
Dependency security	<code>docs/19-dependency-security.md</code>
Server configuration	<code>docs/hu/15-server-config.md</code>
Markdown/rich text	<code>docs/hu/16-markdown-rich-text.md</code>
Media library	<code>docs/hu/17-media-library.md</code>
Object authorization	<code>docs/hu/20-object-authorization.md</code>
Project structure and modules	<code>docs/hu/21-modules-slugs-project-layout.md</code>
Modules and namespaces	<code>docs/21-modules-and-namespaces.md</code>
Arithmetic, F32 math, timing	<code>docs/44-math-and-timing.md</code>
String builtins	<code>docs/46-string-builtins.md</code>

Typed string lists	<code>docs/47-string-lists.md</code>
Typed string dictionaries	<code>docs/48-string-dictionaries.md</code>
Regular expressions	<code>docs/49-regular-expressions.md</code>
Domain objects	<code>docs/hu/22-domain-objects.md</code>
Enums	<code>docs/hu/23-enums.md</code>
Optimistic locking	<code>docs/24-optimistic-locking.md</code>
Business audit	<code>docs/hu/25-business-audit-trail.md</code>
Domain validation	<code>docs/hu/26-domain-validation.md</code>
Canonical URL	<code>docs/hu/27-canonical-url.md</code>
PRG, flash, conflict UX	<code>docs/hu/28-prg-flash-conflict.md</code>
Production deployment	<code>docs/hu/29-production-deployment.md</code>
Backup, restore, upgrade	<code>docs/hu/30-backup-restore-upgrade-rollback.md</code>
V1 developer handoff	<code>docs/hu/31-v1-developer-guide.md</code>
IPv6-ready outbound egress	<code>docs/32-ipv6-egress.md</code>
CLI and daily workflow	<code>docs/hu/33-cli-workflow.md</code>
Server TOML reference	<code>docs/hu/34-server-toml-reference.md</code>
Production checklist and examples	<code>docs/hu/35-production-checklist.md</code>
Debian package and dpkg installation	<code>docs/36-debian-package.md</code>

22.2 Daily development checklist

1. Use typed route/input contracts, and an enum for closed choices.
2. Never place user input into raw SQL or raw HTML.
3. Perform mutations inside a transaction when the business operation must be atomic.
4. After a successful browser POST, use Post/Redirect/Get.
5. Do not public-cache pages that depend on user/session state.
6. Supply secrets only through `*_file` boundaries.
7. Before release, run application checks, configuration preflight, and migration verification; when releasing the RWLang platform itself, also run `./verify.sh`.
8. After deployment, verify live/ready, logs, metrics, audit, and an application smoke path.

Appendix A

Appendix and production checklist

This chapter introduces no new language elements. It is a compact operations aid assembled from the preceding chapters: directory layout, reverse-proxy examples, a systemd unit, logrotate configuration, a pre-release gate, and a troubleshooting map. Treat the examples as starting points; adapt domains, certificates, permissions, and resource limits to your environment.

A.1 Recommended production directory layout

RWLang is locally installed software, so this book consistently uses the `/usr/local` hierarchy for programs and configuration. Application state and logs live separately.

```
/usr/local/bin/
  rclang-server
  rclang-cli

/usr/local/etc/rclang/
  server.toml
  rate-limits.toml
  resource-profiles.toml

/run/secrets/rclang/
  database-url
  redis-url
  local-auth-database-url

/srv/rclang/
  current/                # current application release
  releases/
    2026-09-04.1/
  data/                   # AppFs / durable application data

/var/log/rclang/
  server.log
  access.log
  audit.log
```

Keep release contents read-only where possible. The server should have write permission only where durable state or logs actually need to be written. Secret files must not live inside the application release directory or source control.

A.2 Complete minimal Apache reverse-proxy example

This example assumes one Apache reverse proxy. Apache owns public TLS; RWLang listens only on loopback.

The corresponding minimum RWLang configuration is:

```
[server]
listen = "127.0.0.1:8080"
insecure_dev_cookies = false

[tls]
public_host = "www.example.com"

[web]
trusted_proxy_cidrs = ["127.0.0.1/32", "::1/128"]
```

In reverse-proxy production mode the certificate and key stay on the proxy, but trusted `public_host` is still required for Host/Origin pinning. The proxy must produce authoritative `X-Forwarded-Proto: https` metadata.

```
<VirtualHost *:80>
    ServerName www.example.com
    Redirect permanent / https://www.example.com/
</VirtualHost>

<VirtualHost *:443>
    ServerName www.example.com

    SSLEngine on
    SSLCertificateFile \
        /etc/letsencrypt/live/www.example.com/fullchain.pem
    SSLCertificateKeyFile \
        /etc/letsencrypt/live/www.example.com/privkey.pem

    ProxyPreserveHost On
    ProxyPass      / http://127.0.0.1:8080/
    ProxyPassReverse / http://127.0.0.1:8080/

    RequestHeader set X-Forwarded-Proto "https"
</VirtualHost>
```

Typical Apache modules are `proxy`, `proxy_http`, `headers`, and `ssl`. The RWLang trusted-proxy list should contain only the real proxy. For a loopback proxy this normally means `127.0.0.1/32` and, if needed, `::1/128`. An Internet-sized trusted-proxy range is a configuration error.

A.3 Complete minimal Nginx reverse-proxy example

```
server {
    listen 80;
    server_name www.example.com;
    return 301 https://$host$request_uri;
}
```

```

server {
    listen 443 ssl;
    server_name www.example.com;

    ssl_certificate
        /etc/letsencrypt/live/www.example.com/fullchain.pem;
    ssl_certificate_key
        /etc/letsencrypt/live/www.example.com/privkey.pem;

    location / {
        proxy_pass http://127.0.0.1:8080;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-Proto https;
        proxy_set_header X-Forwarded-For $remote_addr;
    }
}

```

Here Nginx directly sees the client, so it creates the forwarded client address itself. Do not pass through arbitrary Internet-supplied `X-Forwarded-*` values as authority. With multiple proxies, design the trust chain explicitly.

A.4 systemd service example

The repository contains the complete example at `examples/systemd/rwlang.service`. Its essential shape is:

```

[Unit]
Description=RWLang application
Wants=network-online.target
After=network-online.target

[Service]
Type=simple
User=rwlang
Group=rwlang
WorkingDirectory=/srv/rwlang/current
ExecStartPre=/usr/local/bin/rwlang-server \
    --config /usr/local/etc/rwlang/server.toml \
    --check-config
ExecStart=/usr/local/bin/rwlang-server \
    --config /usr/local/etc/rwlang/server.toml
ExecReload=/bin/kill -HUP $MAINPID
Restart=on-failure
RestartSec=2s
TimeoutStopSec=45s

MemoryMax=1G
MemorySwapMax=0
CPUQuota=200%
TasksMax=256
LimitNOFILE=8192
# For direct TLS on 80/443 add only:
# AmbientCapabilities=CAP_NET_BIND_SERVICE
# CapabilityBoundingSet=CAP_NET_BIND_SERVICE

```

```

NoNewPrivileges=yes
PrivateTmp=yes
PrivateDevices=yes
ProtectSystem=strict
ProtectHome=yes
ProtectKernelTunables=yes
ProtectKernelModules=yes
ProtectControlGroups=yes
RestrictSUIDSGID=yes
LockPersonality=yes
ReadWritePaths=/srv/rwlang/data /var/log/rwlang

[Install]
WantedBy=multi-user.target

```

Systemd hard limits and RWLang request/runtime resource policy are complementary defense layers. In the systemd baseline, systemd itself is the process cgroup authority, so do not also enable RWLang's `[cgroup]` writer mode. For direct binds to 80/443 grant only `CAP_NET_BIND_SERVICE`; behind a reverse proxy on a high loopback port, omit it. `TimeoutStopSec` should exceed the configured graceful-shutdown window. In behind-proxy mode `SIGHUP` can transactionally reload domain/application hosting state, while process-level configuration changes still require a controlled restart; application source alone is normally activated by the automatic source-reload supervisor.

A.5 logrotate example

```

/var/log/rwlang/server.log /var/log/rwlang/access.log {
    daily
    rotate 14
    compress
    delaycompress
    missingok
    notifempty
    create 0640 rwlang rwlang
    sharedscripts
    postrotate
        systemctl kill -s HUP --kill-who=main \
            rwlang.service >/dev/null 2>&1 || true
    endscript
}

/var/log/rwlang/audit.log {
    daily
    rotate 90
    compress
    delaycompress
    missingok
    notifempty
    create 0640 rwlang rwlang
    sharedscripts
    postrotate
        systemctl kill -s HUP --kill-who=main \
            rwlang.service >/dev/null 2>&1 || true
    endscript
}

```

Retention is a legal and risk decision. Audit logs usually deserve longer retention and central forwarding; the numbers above are examples, not universal requirements.

A.6 Pre-release production gate

The following list is intentionally short and executable. The earlier chapters provide the rationale.

1. **Source and build:** identify the release commit; pin `Cargo.lock`; build the platform with `cargo build -locked -release -p rqlang-server -p rqlang-cli`; pass `rqlang-cli check` on application source.
2. **Migration:** run `migrate status` and `migrate verify` before production; apply approved schema changes explicitly; verify again afterward.
3. **Configuration:** pass `config-check` on the real production `server.toml`; review the redacted effective configuration.
4. **Secrets:** keep DB/Redis/auth secret files out of the repository, with narrow filesystem permissions, and out of logs and release records.
5. **HTTP edge:** verify domain, TLS, and Host handling; the trusted-proxy list contains only real proxies.
6. **Auth:** production cookie policy is active; local-auth/LDAP configuration is verified; admin/MFA recovery is documented.
7. **CSRF and CORS:** state-changing browser requests are CSRF protected; CORS allowlists are explicit; credentialed CORS has no wildcard origin.
8. **Database:** the application DB user is least privilege; pool/budget matches backend capacity; important query indexes are understood.
9. **Storage/upload:** AppFs confinement is correct; byte/pixel limits are active; the application cannot write release content.
10. **Redis/cache/rate limit:** multi-instance shared security/runtime state lives in Redis; Redis outage behavior is known and tested.
11. **Egress:** only named outbound targets are allowed; host/CIDR/port/TLS/timeout/size policy is explicit; secrets do not appear in URLs or logs.
12. **Resources:** request/runtime limits and process hard limits are consistent; cgroup authority is either systemd or a separately delegated RQLang cgroup, not both.
13. **Observability:** server/access/audit logs are writable; request IDs work; metrics and readiness are reachable from the correct management network; minimum alerts are defined.
14. **Backup/recovery:** recent verified DB + AppFs + identity backups exist; a recovery manifest exists; restore drills are real rather than theoretical.
15. **Deploy/rollback:** release artifact and checksum are known; the previous release can be restored; schema changes have an explicit app-rollback versus full-restore decision.
16. **Post-start evidence:** liveness, readiness, and at least one business smoke request succeed; logs show no startup fallback or unexpected policy failure.

A.7 Quick troubleshooting map

Symptom	First place to look	Next question
Service does not start	config-check, server log	Config/secret/path/policy or dependency readiness?
502/504 at proxy	proxy log, RWLang listener/readiness	Is the process running, loopback port correct, readiness green?
400/415	access log + route body contract	Malformed input, wrong schema, or wrong Content-Type?
401	auth/session audit event	No session, expired, invalidated, or MFA missing?
403	route/object authorization, CSRF/CORS	Who is the actor, and which policy denied access?
404/405/406	route map + request method/Accept	Does the route exist, and is the method/representation acceptable?
409	optimistic-lock/business state	Did the record change, or did a Changed query modify zero rows?
413/431	request hard limits	Did body/upload or headers exceed configured limits?
421/426	Host/proxy/TLS policy	Are public host, trusted proxy, and effective HTTPS correct?
422	named-form rerender	Which field failed typed decode or validation?
429	rate-limit metrics/audit	Which limiter and identity/IP bucket was exhausted?
500	server log by request ID	Was there a real internal invariant failure?
503	readiness + server log	Did DB/Redis/auth/storage or a request resource limit make service temporarily unavailable?
DB error	DB readiness + query context	Connection, timeout, schema drift, lock, or constraint?
File/media error	AppFs path/permissions/limits	Is the storage root writable, with confinement and byte/pixel limits correct?
External API error	egress policy + adapter log	DNS, CIDR, port, TLS, timeout, or response-size denial?
Logs disappear after rotation	<code>rw_log_fallback_total</code> , service log	Did SIGHUP arrive, and are new-file permissions correct?

A.8 What to carry into production

The most important practical rules from this book are:

- define typed data and explicit policy before exposing an HTTP surface;

- keep stable production policy in `server.toml`, not a long service flag list;
- load secret values from files, not the command line or repository;
- treat schema change as an explicit operator step, not a server-startup side effect;
- accept proxy headers as authority only from trusted proxies;
- treat Redis as runtime/security state, not business source of truth;
- remember that DB and AppFs may jointly form one recovery state;
- do not consider a backup proven until an isolated restore drill succeeds;
- check before release, then measure and smoke-test after deployment;
- if external data or a resource is not explicitly bounded, the boundary is not finished.

Closing note

The purpose of this book is to teach the language features as parts of a working web application rather than as isolated syntax. Detailed feature-level reference material remains available in the project's `docs/` directory; this book provides the day-to-day development path.